

A COMPLETE GUIDE · JAVA EDITION

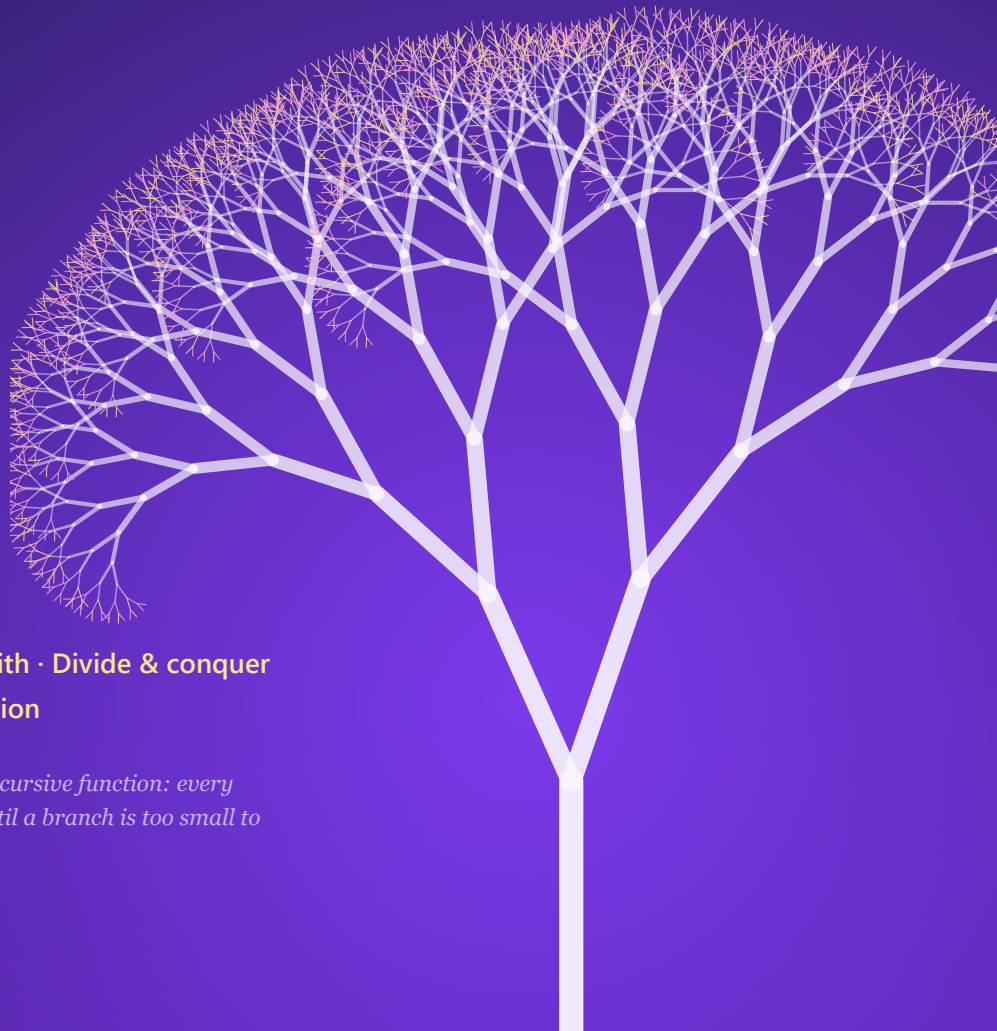
Recursion

From First Principles to
Thinking Recursively

27 chapters

21 runnable Java programs

Real, verified output



Base case · Call stack · Leap of faith · Divide & conquer
· Trees · Backtracking · Memoization

The tree on this page was drawn by a recursive function: every branch draws two smaller branches, until a branch is too small to split — the base case.

Contents

PART I	Foundations	6
01	The Idea Before the Code	7
02	Anatomy of a Recursive Method	12
03	The Call Stack: What the Machine Really Does	20
04	The Leap of Faith: Thinking Recursively	27
05	Tracing Recursion Like a Pro	32
PART II	Basic Recursion	38
06	Before or After the Call: Order Matters	39
07	Recursion with Numbers	43
08	Three Ways to Carry Results	49
09	Recursion on Arrays	53
10	Recursion on Strings	59
PART III	Deeper Recursion	64
11	Measuring Recursive Code: Time and Space	65
12	Tail Recursion and the Java Truth	69
13	Multiple Recursion: When One Call Is Not Enough	73
14	Finding the Recurrence: The Real Skill	79
15	Divide and Conquer	83
PART IV	Recursion on Data Structures	89
16	Linked Lists: A Node Plus a Smaller List	90
17	Binary Trees: Recursion's Natural Home	95
PART V	Generating Possibilities: Backtracking	105
18	Subsets and Subsequences: Take It or Leave It	106
19	Permutations: Every Possible Order	112
20	Backtracking: The Complete Framework	117
21	Backtracking on Boards and Grids	124

PART VI	Making Recursion Fast	132
22	Memoization: Recursion With a Memory	133
23	Recursion vs. Iteration	139
PART VII	Mastery	143
24	Bugs, Traps and How to Debug Them	144
25	The Recursive Problem-Solving Playbook	151
26	Practice Ladder	157
27	Cheat Sheet: Recursion at a Glance	161

How to Read This Book

Recursion has a strange reputation. Some people find it beautiful; many find it confusing; almost everyone remembers the moment it “clicked”. This book is built to create that moment for you deliberately, instead of leaving it to luck, and then to take you far beyond it, to the point where you **think** recursively when you meet a new problem.

The approach is simple: start from absolutely nothing, build one idea at a time, and never ask you to accept something you have not seen with your own eyes. Every idea is explained in words first, then shown in Java, then traced step by step, and finally summarized so it sticks.

What makes this book different

- **First principles, not tricks.** You will learn *why* recursion works (the call stack, and the logic of mathematical induction) before you learn *how* to write it. Once you understand the why, the how becomes obvious, and you stop memorizing solutions.
- **Every program is real.** All Java programs in this book were compiled and executed while the book was being built. The green *Output* boxes contain the **actual** output of those runs. Nothing is hand-typed or made up.
- **A slow, steady ramp.** Part I has no difficult problems at all; it is entirely about understanding. Part II builds muscle with simple problems. Only then do we climb into trees, backtracking and memoization.
- **Pictures of what the machine does.** Call-stack snapshots, recursion trees and trace logs appear throughout. Recursion is invisible when you only read code; these pictures make it visible.

How the book is organized

Part	What you will learn
I. Foundations	What recursion is, the three laws, the call stack, the leap of faith, and how to trace
II. Basic Recursion	Order of work, numbers, arrays, strings, and different ways to carry results
III. Deeper Recursion	Complexity, tail recursion, multiple recursion, finding recurrences, divide and conquer
IV. Data Structures	Linked lists and binary trees, the natural home of recursion
V. Backtracking	Subsets, permutations, the backtracking framework, grids and boards
VI. Making It Fast	Memoization, the bridge to dynamic programming, and converting recursion to iteration
VII. Mastery	A bug gallery, the problem-solving playbook, a practice ladder, and a cheat sheet

The colored boxes

Throughout the book you will see boxes with different colors. Each color means something:

★ KEY IDEA

The most important concept in a section. If you remember nothing else, remember these.

✦ PRO TRICK

A technique experienced programmers use that beginners usually don't know.

✖ COMMON MISTAKE

A trap that almost everyone falls into at least once, and how to avoid it.

≈ REAL-LIFE ANALOGY

A picture from everyday life that makes an abstract idea concrete.

? TRY IT YOURSELF

A short exercise. Actually do it, on paper, before reading the answer that follows.

How to study this book (please read)

1. **Read Part I slowly, even if you think you know recursion.** Everything later depends on it. Most people who “can't do recursion” have a gap in exactly one place: the call stack or the leap of faith.
2. **Type the programs yourself.** All the code is also available as `.java` files in the `recursion-java` folder next to this PDF, but typing it yourself is how your fingers learn the patterns.
3. **Trace on paper.** When a section shows a trace, cover it, trace the code yourself, then compare.
4. **Do the “Try it yourself” boxes.** Recursion is a skill, like swimming. Reading about swimming does not make you a swimmer.
5. **Come back to the cheat sheet** (the last chapter) regularly. It compresses the entire book into a few pages.

“

Recursion is not a trick to memorize. It is a way of seeing problems: as smaller copies of themselves.

Let's begin.

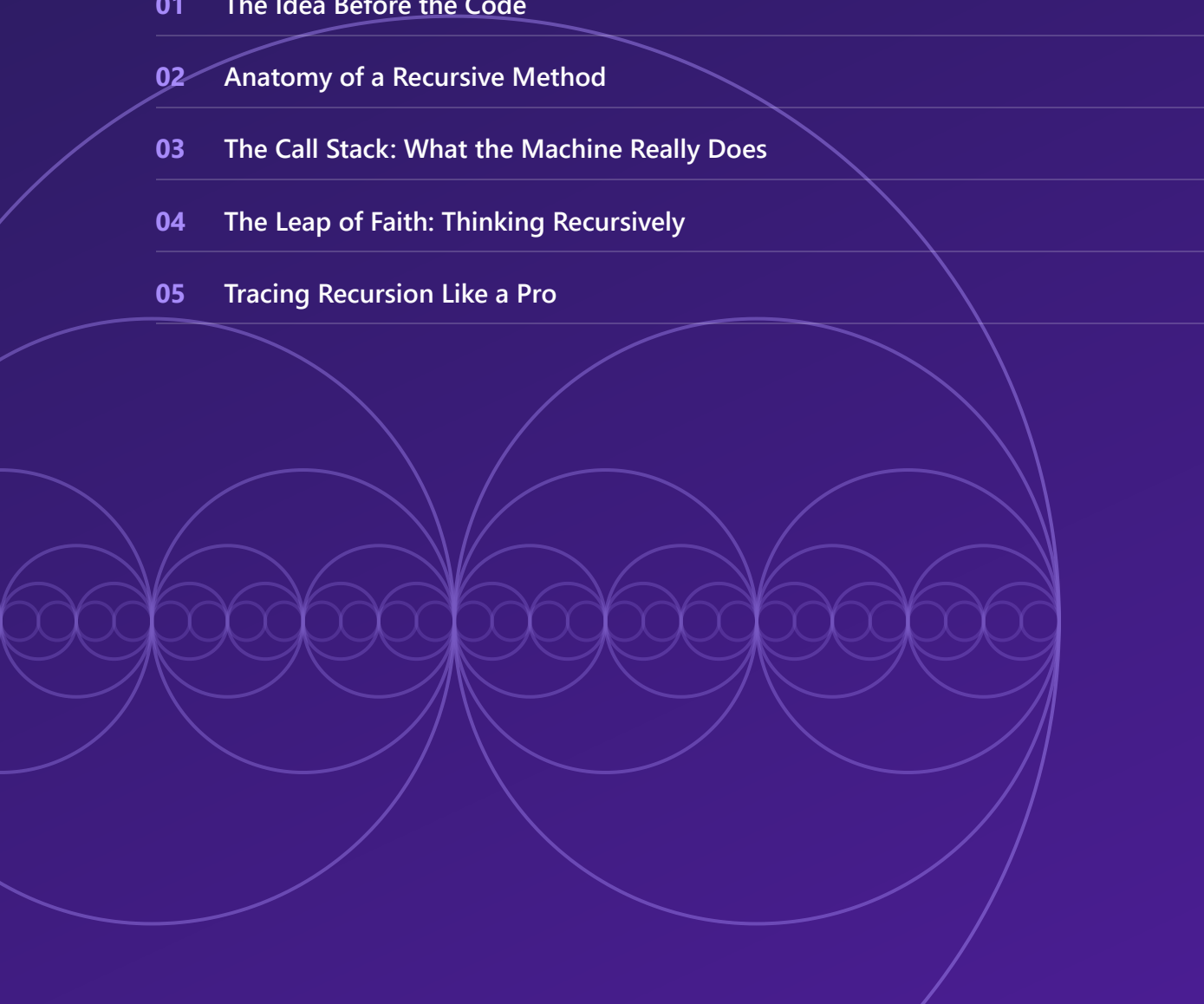
P A R T

I

Foundations

What recursion really is, why it works, and what the machine is doing when a method calls itself.

- 01 The Idea Before the Code
- 02 Anatomy of a Recursive Method
- 03 The Call Stack: What the Machine Really Does
- 04 The Leap of Faith: Thinking Recursively
- 05 Tracing Recursion Like a Pro



CHAPTER 1

The Idea Before the Code

01

IN THIS CHAPTER

- See recursion as an everyday idea, long before any code
- Understand what “a problem that contains a smaller copy of itself” means
- Learn the precise definition of recursion in programming
- Understand why recursion is worth mastering

1.1 A problem that contains itself

Most problems you solve in programming are solved by **repetition**: do something, then do it again, and again, until you are done. Loops (`for` , `while`) are the tool for that. Recursion is a different way of thinking about repetition. Instead of saying “repeat this step n times”, recursion says:

“

Solve a big problem by solving a smaller version of the same problem, and then doing a tiny bit of extra work.

That is the whole idea. Everything in this book is built on that one sentence. But the sentence only becomes meaningful when you see it in many forms, so let’s look at a few, without any code at all.

1.2 The line at the ticket counter

Imagine you are standing somewhere in a long queue at a ticket counter. You want to know your position in the line, but the line is long, it curves around a corner, and you cannot see the front. How do you find your position?

You could leave the line and count everyone, but you would lose your place. Here is a cleverer way. You tap the person in front of you and ask: “What’s your position in the line?” That person doesn’t know either, so they do the exact same thing: they tap the person in front of *them* and ask the same question. This continues, person by person, towards the front.

Eventually the question reaches the person at the very front. That person doesn’t need to ask anyone. They can **see** the counter, so they simply answer: “I’m number 1.”

Now the answers travel **back**. The second person hears “1”, adds one, and says “I’m number 2” to the person behind them. The third person hears “2” and says “I’m number 3”. And so on, until the answer reaches you. You hear, say, “I’m number 36”, and you know you are number 37.

Look carefully at what each person did:

1. **Checked whether they could answer immediately** (only the front person can).
2. If not, **asked the same question of a slightly smaller line**: the line in front of them.
3. **Waited** for the answer.
4. **Did one small piece of work** with that answer (added 1) and passed it back.

Nobody counted the whole line. Every person did a tiny amount of work, and every person followed *exactly the same rule*. That is recursion.

★ KEY IDEA

A recursive solution has every “person” follow the same simple rule: *if the answer is obvious, give it; otherwise ask the same question about a smaller problem, then use that answer to build yours.*

Let’s write the queue rule as if it were a method:

THE QUEUE RULE, WRITTEN AS A RECIPE

```
position(me):
  if nobody is in front of me:      <- the obvious case (the base case)
    return 1
  else:
    return position(person in front of me) + 1    <- smaller problem + tiny work
```

Keep this picture in your mind. We will come back to it many times.

1.3 Russian dolls

A matryoshka is a set of wooden dolls nested inside each other. Open the big doll, and inside is a smaller doll. Open that one, and there is an even smaller doll. Eventually you reach a tiny doll that does not open: it is solid.

Suppose someone asks: “*How many dolls are in this set?*” A recursive way to answer is:

- If the doll is solid (does not open), the answer is **1**.
- Otherwise, open it, count the dolls in the smaller set inside, and add **1** for the doll you just opened.

Notice two phases. First you go **inward**, opening doll after doll, without being able to answer yet. Then, once you reach the solid doll, you come back **outward**, closing each doll and adding 1 at each step. The answer is only assembled on the way back out. This “in, then out” shape is one of the most important things to understand about recursion, and Chapter 3 shows exactly how the computer does it.

≈ REAL-LIFE ANALOGY: TWO DIRECTIONS

Going **in** (opening dolls) is the *calling phase*: problems get smaller, but nothing is answered yet. Coming **out** (closing dolls) is the *returning phase*: answers flow back and get combined. Beginners usually only think about the first phase. Experts think about both.

1.4 Recursive definitions in mathematics

Mathematicians have used recursion for centuries, under the name **recursive definition** (or *inductive definition*). Here are a few you may know.

Factorial. The factorial of n , written $n!$, is the product of all whole numbers from 1 to n . So $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. But look:

FACTORIAL CONTAINS A SMALLER FACTORIAL

$$\begin{aligned} 5! &= 5 \times 4 \times 3 \times 2 \times 1 \\ &= 5 \times (4 \times 3 \times 2 \times 1) \\ &= 5 \times 4! \end{aligned}$$

So factorial can be defined **in terms of itself**:

RECURSIVE DEFINITION OF FACTORIAL

$$\begin{array}{ll} 0! = 1 & \text{(the starting point: defined directly)} \\ n! = n \times (n - 1)! & \text{for every } n \geq 1 \end{array}$$

This definition is complete. With it you can compute any factorial: $3! = 3 \times 2! = 3 \times 2 \times 1! = 3 \times 2 \times 1 \times 0! = 3 \times 2 \times 1 \times 1 = 6$.

Sum of the first n numbers. Let $S(n) = 1 + 2 + \dots + n$. Then $S(n) = n + S(n - 1)$, and $S(0) = 0$.

Powers. $x^0 = 1$, and $x^n = x \times x^{n-1}$.

Fibonacci numbers. $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n - 1) + F(n - 2)$. Here each number depends on *two* smaller ones. This will be important later.

Natural numbers themselves. Even the counting numbers can be defined recursively: *0 is a natural number; if n is a natural number, then $n + 1$ is a natural number.*

Every one of these definitions has the same two pieces:

1. A **starting case** that is defined directly, with no reference to itself ($0! = 1$, $S(0) = 0$, $x^0 = 1$).
2. A **rule** that defines a bigger case using a smaller case ($n! = n \times (n-1)!$).

That's not a coincidence. These two pieces are exactly what every recursive program needs, and they have names: the **base case** and the **recursive case**.

1.5 Recursive structures all around us

Recursion is not only about numbers. Many *structures* are recursive: they contain smaller versions of themselves.

- **Folders on your computer.** A folder contains files and *other folders*, which contain files and other folders, and so on. To compute the total size of a folder, you add the sizes of its files plus the total sizes of its sub-folders. The same question, asked of smaller folders.
- **A family tree.** Your ancestors are your parents, plus the ancestors of each of your parents.

- **A sentence in a language.** “The cat that chased the mouse that ate the cheese...” — a phrase can contain another phrase of the same kind.
- **Trees in nature.** A branch splits into smaller branches, which split into smaller branches. The cover of this book was drawn exactly that way.
- **A linked list** (in Java): a list is either empty, or a node followed by a *smaller list*.
- **A binary tree** (in Java): a tree is either empty, or a root node with a *left tree* and a *right tree*.

★ KEY IDEA

When the **data** is recursive (lists, trees, folders), the **code** that processes it is naturally recursive too. This is why recursion is not optional for a serious programmer.

1.6 So what is recursion in programming?

Now we can state it precisely.

§ DEFINITION

Recursion is a technique in which a method solves a problem by **calling itself** on a **smaller or simpler input**, continuing until it reaches an input so simple that the answer is known directly (the **base case**). The answers from the smaller calls are then **combined** to produce the answer to the original problem.

A method that calls itself is called a **recursive method**. Each time the method calls itself, we say it makes a **recursive call**.

There are three phrases in that definition that matter enormously:

1. **“calling itself”** — the method uses its own name inside its own body.
2. **“a smaller or simpler input”** — every call must move closer to the simple case. If it doesn’t, the method will call itself forever.
3. **“combined”** — the work isn’t finished when the recursive call returns; often you still have to do something with the answer.

1.7 Why learn recursion at all?

A reasonable question: “If loops can repeat things, why do I need recursion?” Here are honest reasons.

- **Some problems are naturally recursive**, and a recursive solution is dramatically shorter and clearer. Printing every file inside nested folders, processing a tree, or generating all arrangements of letters are painful with loops and elegant with recursion.
- **Trees and graphs** are everywhere: file systems, HTML documents, compilers, databases, game AI, maps. Almost every algorithm on them is recursive.
- **Backtracking** (solving Sudoku, N-Queens, mazes, puzzles, generating combinations) is built on recursion.

- **Divide and conquer** algorithms (merge sort, quick sort, binary search, fast exponentiation) are recursive by nature.
- **Dynamic programming**, one of the most powerful algorithm techniques, is usually discovered by first writing a recursive solution and then optimizing it.
- **Interviews.** Technical interviews test recursion heavily, because it reveals whether a candidate can think abstractly.
- **It changes how you think.** Once recursion clicks, you start seeing problems as compositions of smaller problems. That skill transfers to all of programming.

Σ CHAPTER SUMMARY

- Recursion solves a problem by solving a **smaller version of the same problem** and doing a little extra work.
- Every recursive definition has a **base case** (answered directly) and a **recursive case** (defined using a smaller case).
- Recursion has two phases: going **in** (calls get smaller) and coming **out** (answers are combined).
- Recursive **data** (lists, trees, folders) leads naturally to recursive **code**.

CHAPTER 2

Anatomy of a Recursive Method

02

IN THIS CHAPTER

- Write and run your first recursive Java method
- Learn the three laws every recursive method must obey
- Understand base cases deeply: what they are, how to find them, and how many you need
- Learn the universal template for recursive methods
- See exactly what goes wrong when a law is broken

2.1 Your first recursive program

Let's write a countdown: print the numbers from `n` down to 1, and then print "Liftoff!".

With a loop you would write `for (int i = n; i >= 1; i--) System.out.println(i);`. Let's think recursively instead, using the queue idea. What is the *smallest* countdown? A countdown from 0: there is nothing to count, so we just print "Liftoff!". And a countdown from `n`? Print `n`, and then... do a countdown from `n - 1`. The same problem, one step smaller.

```

Countdown.java  complete program

public class Countdown {
    static void countdown(int n) {
        if (n == 0) {                // BASE CASE: nothing left to count
            System.out.println("Liftoff!");
            return;                  // stop -- do not call again
        }
        System.out.println(n);       // this call's own small job
        countdown(n - 1);            // hand a SMALLER problem to a new call
    }

    public static void main(String[] args) {
        countdown(5);
    }
}

```

>_ OUTPUT OF COUNTDOWN.JAVA (ACTUAL RUN)

```
5
4
3
2
1
Liftoff!
```

Read the method slowly, line by line, as if you were the computer:

- `countdown(5)` is called. Is `n == 0` ? No. So it prints `5` and calls `countdown(4)` .
- `countdown(4)` : not 0, prints `4` , calls `countdown(3)` .
- ... and so on until ...
- `countdown(0)` : `n == 0` is true, so it prints "Liftoff!" and **returns**, without calling itself again.

Then something subtle happens: `countdown(0)` returns to `countdown(1)` , which had nothing left to do after its call, so it also returns, to `countdown(2)` , and so on back to `main` . We'll look at that return journey in detail in the next chapter.

I NOTE: A METHOD CALLING ITSELF IS LEGAL AND NORMAL

Java does not treat recursive calls specially. Calling `countdown` from inside `countdown` works exactly like calling any other method. There is no special keyword, no special syntax. The only "magic" is the call stack, which Java already uses for *every* method call.

2.2 The three laws of recursion

Every correct recursive method obeys three laws. If you remember these, you will never write an infinite recursion again.

§ THE THREE LAWS OF RECURSION

- 1. A recursive method must have a base case:** an input for which it returns (or stops) *without* calling itself.
- 2. A recursive method must make progress towards the base case:** every recursive call must be on an input that is *closer* to the base case.
- 3. A recursive method must call itself** on that smaller input, and use (or combine) the result.

Let's check `countdown` against the laws:

Law	In <code>countdown</code>
Base case exists	<code>if (n == 0)</code> prints "Liftoff!" and returns without recursing
Progress towards base	each call passes <code>n - 1</code> , so <code>n</code> gets closer to 0 every time
Calls itself	<code>countdown(n - 1)</code>

A fourth thing is usually present too, though it is not a "law": **combination** — the work you do with the result of the recursive call. In `countdown` the work is done *before* the call (printing `n`), and there is nothing to combine because the method returns `void`. In `factorial`, as we'll see, the combination step is the multiplication `n * factorial(n - 1)`.

2.3 The base case, in depth

The base case is the most important line in any recursive method, and the most common source of bugs. Let's understand it thoroughly.

WHAT IS A BASE CASE?

It is the **smallest, simplest version** of the problem, one whose answer you know *without doing any recursion*. It is the front person in the queue, the solid Russian doll, the `0! = 1` in the math definition.

HOW DO YOU FIND THE BASE CASE?

Ask yourself: **"For which input is the answer so obvious that I don't need to think?"** Typical answers:

Kind of input	Typical base case	Why
A number <code>n</code>	<code>n == 0</code> or <code>n == 1</code>	$0! = 1$, sum of nothing = 0, $x^0 = 1$
An array with index <code>i</code>	<code>i == arr.length</code>	no elements left to process
A string	<code>s.isEmpty()</code> or <code>s.length() == 1</code>	nothing to reverse, trivially a palindrome
A range <code>[low, high]</code>	<code>low > high</code> or <code>low == high</code>	empty range, or a single element
A linked list node	<code>node == null</code>	empty list
A tree node	<code>node == null</code>	empty tree
A target sum	<code>remaining == 0</code>	found a valid combination

✦ PRO TRICK: PREFER THE *EMPTIEST* BASE CASE

When possible, pick the base case for the **empty** input (0, empty string, `null`, `i == length`). It usually makes the code shorter and handles edge cases for free. For example, `sum(arr, i)` with base case `i == arr.length → return 0` works even for an empty array, while a base case of “one element left” would crash on an empty array.

CAN THERE BE MORE THAN ONE BASE CASE?

Yes, and often there must be. Fibonacci needs **two**: `fib(0) = 0` and `fib(1) = 1`, because the recursive case `fib(n-1) + fib(n-2)` reaches back two steps. If you only had `fib(0)`, then `fib(1)` would compute `fib(0) + fib(-1)`, and `fib(-1)` would recurse forever towards negative infinity.

A good rule: **look at how far back your recursive case reaches**. If it uses `n-1` only, one base case is usually enough. If it uses `n-2`, you typically need two.

There are also “**early exit**” base cases that stop the recursion not because the input is small, but because the answer has been found or has become impossible:

isSorted: two different ways to stop

```
static boolean isSorted(int[] a, int i) {
    if (i >= a.length - 1) return true;    // base case: nothing left to compare
    if (a[i] > a[i + 1]) return false;     // early exit: found a violation, stop now
    return isSorted(a, i + 1);
}
```

THE BASE CASE MUST BE REACHABLE

It isn't enough for a base case to *exist*; every chain of recursive calls must eventually *reach* it. Consider:

Base case that is never reached for odd n

X BUGGY

```
static int halfSum(int n) {
    if (n == 0) return 0;
    return n + halfSum(n - 2);    // 5 -> 3 -> 1 -> -1 -> -3 -> ... never hits 0!
}
```

For even `n` this works. For odd `n`, the values skip over 0 and go negative forever. The fix is a base case that catches every way of arriving at the bottom:

Base case that catches both parities

✓ FIXED

```
static int halfSum(int n) {
    if (n <= 0) return 0;        // catches 0 AND the negative overshoot
    return n + halfSum(n - 2);
}
```

✦ **PRO TRICK: USE `<=` OR `>=` IN BASE CASES WHEN OVERTHOOTING IS POSSIBLE**

Writing `n <= 0` instead of `n == 0`, or `low >= high` instead of `low == high`, makes a method robust against inputs that jump past the exact boundary. It costs nothing and prevents a whole class of infinite recursions.

2.4 The recursive case: making the problem smaller

The recursive case is where the method calls itself. Its job is to **reduce** the problem. There are many ways an input can “get smaller”; recognizing them is a big part of recursive thinking:

How the input shrinks	Example call	Used in
Subtract one	<code>f(n - 1)</code>	factorial, countdown, sum
Divide	<code>f(n / 2)</code> , <code>f(n / 10)</code>	fast power, digit problems, binary search
Move an index forward	<code>f(arr, i + 1)</code>	array problems
Move two pointers inward	<code>f(s, left + 1, right - 1)</code>	palindrome, reverse
Shorten a string	<code>f(s.substring(1))</code>	string problems
Go to the next node	<code>f(node.next)</code>	linked lists
Go to a child	<code>f(node.left)</code> , <code>f(node.right)</code>	trees
Reduce a remaining target	<code>f(remaining - x)</code>	subset sum, coin change
Make one decision	<code>f(i + 1, withChoice)</code>	subsets, permutations, backtracking

In every case, there is some measure (a number, a length, a distance, the size of a subtree) that gets strictly smaller with each call, and the base case sits at the bottom of that measure.

2.5 The universal template

Almost every recursive method you will ever write fits this shape:

●●● The universal shape of a recursive method

```
ReturnType solve(Input input) {
    // 1. BASE CASE(S): the answer is obvious, return it directly
    if (isSimple(input)) {
        return directAnswer(input);
    }

    // 2. REDUCE: build one (or more) smaller inputs
    Input smaller = shrink(input);

    // 3. RECURSE: trust the method to solve the smaller input correctly
    ReturnType smallerAnswer = solve(smaller);

    // 4. COMBINE: build this answer from the smaller answer
    return combine(input, smallerAnswer);
}
```

And here is `factorial` written in exactly that shape, so you can see the mapping:

●●● factorial, in template form

```
static int factorial(int n) {
    if (n == 0) { // 1. base case
        return 1;
    }
    int smaller = n - 1; // 2. reduce
    int smallerAnswer = factorial(smaller); // 3. recurse
    return n * smallerAnswer; // 4. combine
}
```

Normally we compress steps 2 to 4 into one line: `return n * factorial(n - 1);`. But when you are learning, or when you are stuck, **writing the four steps out separately** is extremely helpful.

2.6 What happens when you break the laws

Understanding failure is as important as understanding success. Here is what each broken law looks like.

●●● Law 1 broken: no base case

✗ BUGGY

```
static int factorial(int n) {
    return n * factorial(n - 1); // calls forever: 3, 2, 1, 0, -1, -2, ...
}
```

The method never stops calling itself. Each call uses a little memory on the call stack, and eventually Java runs out and throws `StackOverflowError`.

● ● ● Law 2 broken: no progress

✗ BUGGY

```
static int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n);    // same n every time: never gets closer to 0
}
```

The base case exists, but the calls never approach it. Result: `StackOverflowError` again.

● ● ● Law 2 broken: progress in the wrong direction

✗ BUGGY

```
static int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n + 1); // moves AWAY from the base case
}
```

● ● ● Combination forgotten: the result is thrown away

✗ BUGGY

```
static int factorial(int n) {
    if (n == 0) return 1;
    factorial(n - 1);           // the answer is computed... and ignored
    return n;                   // returns just n, not n!
}
```

This last one doesn't crash, which makes it more dangerous: it silently returns the wrong answer. `factorial(5)` returns 5.

Mistake	Symptom
No base case	<code>StackOverflowError</code>
Base case never reached	<code>StackOverflowError</code> (sometimes only for some inputs)
No progress / wrong direction	<code>StackOverflowError</code>
Result of recursive call ignored	Wrong answer, no error
Wrong base case value (e.g. <code>0! = 0</code>)	Every answer wrong (often everything becomes 0)

✗ COMMON MISTAKE: THE WRONG BASE VALUE

If you write `if (n == 0) return 0;` in factorial, then every factorial becomes 0, because everything is eventually multiplied by that 0. The base value must be the correct answer for the base input. For products the neutral value is 1; for sums it is 0; for “is there any” questions it is `false`; for “are all” questions it is `true`.

Σ CHAPTER SUMMARY

- The **three laws**: have a base case, make progress towards it, call yourself on the smaller input.
- Choose the **simplest possible base case**, preferably the empty input, and use `<=` / `>=` to guard against overshooting.
- There are many ways to shrink an input: `n-1`, `n/2`, `i+1`, two pointers, `node.next`, children, remaining target.
- Template: **base case** → **reduce** → **recurse** → **combine**.
- Breaking a law gives either `StackOverflowError` or a silently wrong answer.

CHAPTER 3

The Call Stack: What the Machine Really Does

03

IN THIS CHAPTER

- Understand what happens in memory when *any* method is called
- See that every recursive call gets its own separate copy of variables
- Watch `factorial(3)` execute, frame by frame
- Understand the winding and unwinding phases
- Understand `StackOverflowError` and the memory cost of recursion

This chapter is the one that makes recursion stop feeling like magic. Once you see the call stack, you will understand *exactly* why recursion works, and you'll be able to predict what any recursive method does.

3.1 How any method call works

Forget recursion for a moment. What happens when method `a` calls method `b`?

Java needs to remember several things:

- **Where to come back to.** When `b` finishes, execution must continue in `a`, at the exact line right after the call.
- **`a`'s local variables.** While `b` runs, `a`'s variables must be preserved, untouched.
- **`b`'s parameters and local variables.** `b` needs its own space for its own variables.

Java keeps all of this in a region of memory called the **call stack** (or just "the stack"). Each method call gets a block of memory on the stack called a **stack frame** (or *activation record*). A frame contains:

WHAT IS INSIDE ONE STACK FRAME

```
+-----+
| STACK FRAME for one call of a method |
| - the parameter values for THIS call |
| - the local variables for THIS call  |
| - the return address (where to resume)|
| - space for the value being returned |
+-----+
```

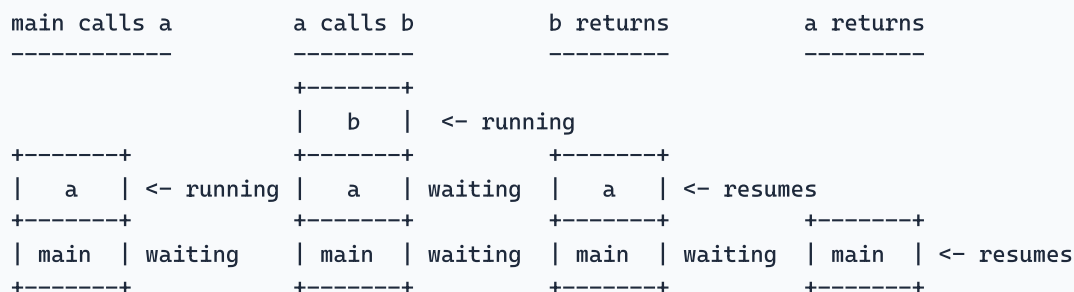
The stack works exactly like a stack of plates:

- When a method is **called**, a new frame is **pushed** on top.

- The frame on **top** is always the method that is currently running.
- When a method **returns**, its frame is **popped** off, and the method underneath continues from where it left off.

This is called **LIFO**: Last In, First Out. The last method called is the first one to finish.

A NORMAL CHAIN OF CALLS: MAIN -> A -> B



3.2 Recursion is just calls to the same method

Here is the key realization: **Java does not care that the method being called is the same method.** When `factorial(3)` calls `factorial(2)`, Java does exactly what it does when `a` calls `b`: it pushes a *new*, *separate* frame. The new frame has its **own** `n` (equal to 2), completely independent of the `n` in the previous frame (which stays 3).

So “a method calling itself” is really “many separate copies of a method’s variables, stacked on top of each other, each waiting for the one above it”. The program below proves this. Each call has a local variable `local`, and we print it both when the call starts and when it ends:

```
CallStackDemo.java frames

static void showFrames(int n) {
    int local = n * 10;           // every call gets its OWN copy of 'local'
    System.out.println(" ".repeat(3 - n) + "enter n=" + n + " local=" + local);
    if (n > 0) {
        showFrames(n - 1);
    }
    System.out.println(" ".repeat(3 - n) + "exit n=" + n + " local=" + local);
}
```

>_ OUTPUT OF CALLSTACKDEMO.JAVA (ACTUAL RUN)

```
factorial(5) = 120

enter n=3  local=30
  enter n=2  local=20
    enter n=1  local=10
      enter n=0  local=0
        exit n=0  local=0
      exit n=1  local=10
    exit n=2  local=20
  exit n=3  local=30

StackOverflowError! Depth reached before crash: about 11000 calls
```

Look at the middle block of output (ignore the first and last lines for now). Two crucial facts:

1. **Every call keeps its own `local`.** When the `n=3` call resumes at the very end, its `local` is still 30, even though three other calls created their own `local` variables with different values in between. They never interfered.
2. **The exits happen in reverse order of the entries.** Entered 3, 2, 1, 0; exited 0, 1, 2, 3. That's the stack: last in, first out.

★ **KEY IDEA**

Each recursive call is an **independent copy** of the method, with its own parameters and local variables, living in its own stack frame. Calls don't share variables (unless you use fields/static variables or pass the same object around, which we'll discuss later).

3.3 Watching factorial(3), frame by frame

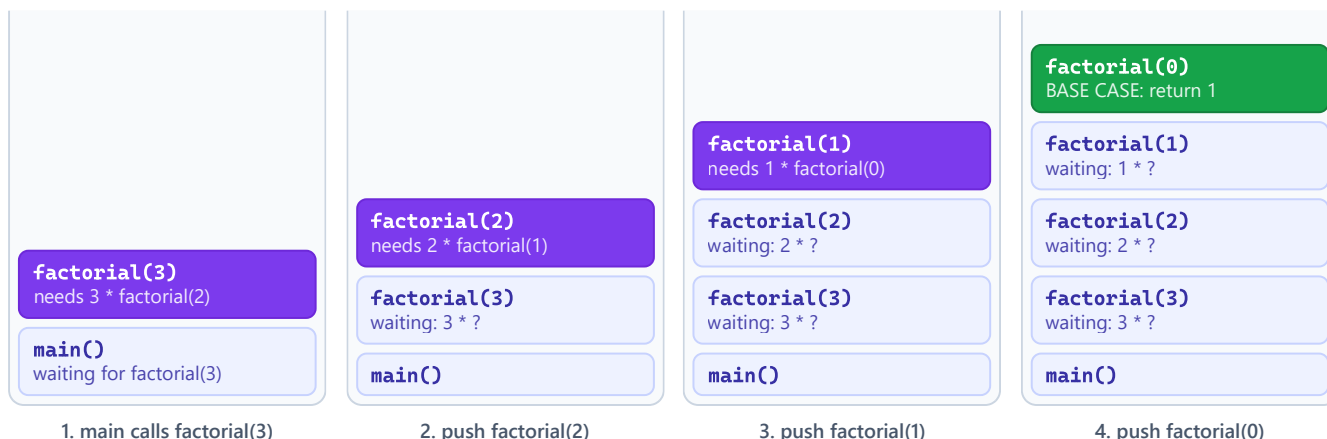
Here is the method again:

```
CallStackDemo.java  factorial

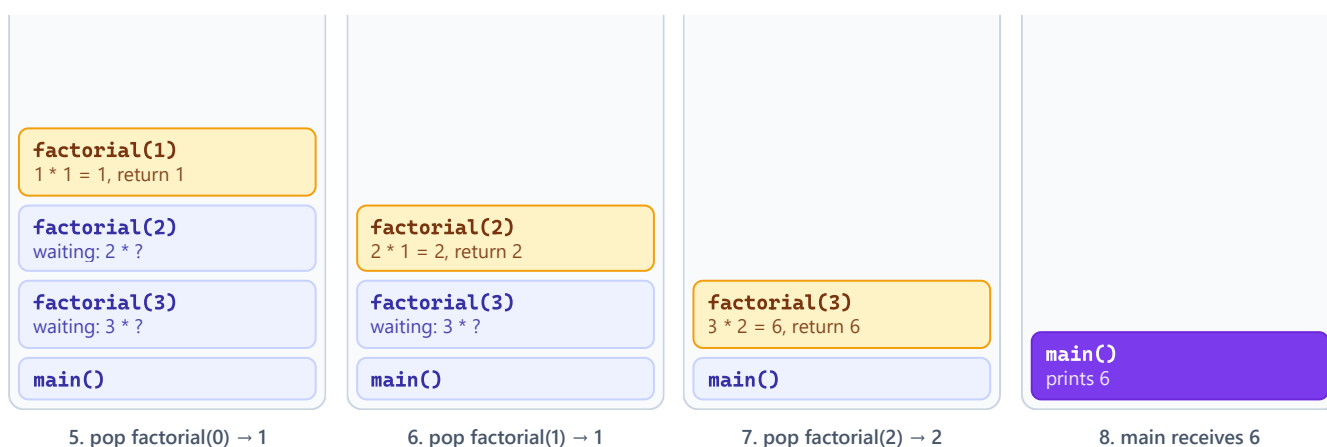
static int factorial(int n) {
    if (n == 0) return 1;           // base case: 0! = 1
    return n * factorial(n - 1);    // n! = n * (n-1)!
}
```

Let's watch `factorial(3)` run. In the picture below, each column is a snapshot of the stack. The top box is the frame currently running.

WINDING — calls are pushed, every frame is waiting



UNWINDING — base case answers, frames pop and finish their pending work



The call stack while computing factorial(3). The top box is the frame currently running.

Step by step, in words:

1. `main` calls `factorial(3)`. A frame with `n = 3` is pushed. It checks `n == 0` — false — and reaches `return 3 * factorial(2)`. It cannot multiply yet, because it doesn't know `factorial(2)`. So it **pauses** at that exact spot and calls `factorial(2)`.
2. A new frame with `n = 2` is pushed. Same story: it pauses at `2 * factorial(1)`.
3. A frame with `n = 1` is pushed. It pauses at `1 * factorial(0)`.
4. A frame with `n = 0` is pushed. Now `n == 0` is **true**. This is the base case. It returns `1` immediately, without calling anything. Its frame is popped.
5. Control returns to the `n = 1` frame, exactly where it paused. It now knows `factorial(0) = 1`, computes `1 * 1 = 1`, returns 1, and is popped.
6. The `n = 2` frame resumes, computes `2 * 1 = 2`, returns 2, is popped.
7. The `n = 3` frame resumes, computes `3 * 2 = 6`, returns 6, is popped.
8. `main` receives 6.

★ KEY IDEA

A recursive call **pauses** the current method at the exact point of the call. The paused method remembers everything (its `n`, and the fact that it still has to multiply). When the call returns, the method **resumes** from that point. The work written *after* a recursive call is the “unfinished business” that waits on the stack.

3.4 The two phases: winding and unwinding

Every recursion has two phases, which you saw in the picture:

- **Winding** (the calling phase): calls are made, one after another, frames pile up. The problem gets smaller but *nothing is completed yet*. Every frame is waiting.
- **Unwinding** (the returning phase): starting from the base case, calls return one after another, frames are removed, and each frame finishes the work it had postponed.

This explains a lot:

- Code written **before** the recursive call runs during **winding**, in the order the calls are made (biggest to smallest).
- Code written **after** the recursive call runs during **unwinding**, in the **reverse** order (smallest to biggest).

That is why recursion can “reverse” things so naturally, as you’ll see in Chapter 6 when printing 1 to n versus n to 1.

SUBSTITUTION VIEW OF THE SAME COMPUTATION

<code>factorial(3)</code>	
<code>= 3 * factorial(2)</code>	winding: nothing can be multiplied yet
<code>= 3 * (2 * factorial(1))</code>	
<code>= 3 * (2 * (1 * factorial(0)))</code>	
<code>= 3 * (2 * (1 * 1))</code>	base case reached
<code>= 3 * (2 * 1)</code>	unwinding: multiplications happen now,
<code>= 3 * 2</code>	innermost first
<code>= 6</code>	

3.5 StackOverflowError

The call stack is not infinite. The JVM gives each thread a fixed amount of stack memory (commonly around 512 KB to 1 MB by default, depending on platform and settings). Every frame uses some of it. If recursion goes too deep, the stack fills up and Java throws `java.lang.StackOverflowError`.

This program deliberately recurses without a base case and counts how deep it gets before crashing:

```

CallStackDemo.java  overflow

static int depth = 0;

static void noBaseCase() {
    depth++;
    noBaseCase();           // never stops: no base case, no progress
}

```

The last line of the output shown earlier is the result: roughly twenty thousand calls before the stack ran out, on the machine this book was built on. The exact number varies with the JVM, the operating system, how big each frame is, and JIT compilation.

! WHAT THIS MEANS IN PRACTICE

- Recursion depth of a few thousand is usually fine in Java.
- Recursion depth in the **hundreds of thousands or millions** will crash, even if the code is logically correct.
- So: `factorial(20)` recursively is fine; summing a 1,000,000-element array with one recursive call per element is **not**. Use a loop, or a recursion whose depth grows slowly (like `log n`).
- You *can* increase the stack size with the JVM option `-Xss` (e.g. `java -Xss64m Main`) or by running code in a `Thread` created with a larger stack size, but that is a workaround, not a design.

✗ COMMON MISTAKE: BLAMING THE LOGIC FOR A DEPTH PROBLEM

If your recursion works for small inputs but throws `StackOverflowError` for large inputs, the logic may be perfectly fine; the input is simply too deep for the stack. If it throws for **every** input, even tiny ones, your base case is missing or unreachable.

3.6 The memory cost of recursion

Because each active call occupies a frame, recursion uses extra memory proportional to the **maximum depth** of the recursion: the largest number of frames that are on the stack *at the same time*.

- `factorial(n)`: depth n , so $O(n)$ extra space.
- Binary search on n elements: depth about $\log_2 n$, so $O(\log n)$ space.
- A loop version of factorial: $O(1)$ extra space.

Note the word “at the same time”. A method that makes a million calls in total, but never more than 20 at once, only needs 20 frames. This distinction becomes important with tree recursion (Chapter 13), where the total number of calls can be enormous while the depth stays small.

Σ CHAPTER SUMMARY

- Every method call pushes a **stack frame** holding its parameters, local variables and return address. Returning pops it.
- Recursive calls are ordinary calls: each gets a **separate frame** with its **own variables**.
- A recursive call **pauses** the caller; the caller **resumes** when the call returns.
- **Winding**: frames pile up, nothing finishes. **Unwinding**: frames pop, postponed work runs in reverse order.
- Too much depth causes `StackOverflowError`. Recursion costs **$O(\text{depth})$** memory.

CHAPTER 4

The Leap of Faith: Thinking Recursively

04

IN THIS CHAPTER

- Understand why tracing every call in your head is the wrong way to *design* recursion
- Learn mathematical induction in five minutes, and see that it is recursion
- Learn the “leap of faith”: assume the smaller call works
- Master a five-step recipe for designing any recursive method
- Apply the recipe to new problems from scratch

Chapter 3 showed you what the computer does. This chapter shows you how *you* should think. They are different, and confusing the two is the number one reason people struggle with recursion.

4.1 The biggest beginner mistake

When beginners try to write a recursive method, they usually try to imagine the whole chain of calls: “OK so *this calls with 4, which calls with 3, which calls with 2, and then that returns, and then this multiplies...*”. For factorial, you can just about manage that. For anything with two recursive calls, or with a tree, or with backtracking, it is completely impossible. Your brain overflows long before the computer’s stack does.

Experienced programmers **do not do this** when designing recursion. Instead they use a way of thinking borrowed from mathematics, called **induction**. Tracing is still useful, but it is a tool for *checking* and *debugging*, not for *designing*.

4.2 Mathematical induction in five minutes

Imagine an infinite line of dominoes. How can you be sure that *every* domino will fall? You don’t need to watch them all. You only need two facts:

1. **The first domino falls.** (Someone pushes it.)
2. **Whenever a domino falls, it knocks over the next one.** (They’re spaced correctly.)

If both are true, *every* domino falls: the first falls by fact 1, so the second falls by fact 2, so the third falls by fact 2, and so on forever.

This is **mathematical induction**. To prove a statement is true for all $n \geq 0$, you prove:

1. **Base case:** it’s true for $n = 0$.
2. **Inductive step:** *if it’s true for $n - 1$, then it’s true for n .*

The magic is in step 2: you get to **assume** the statement for $n - 1$ (this assumption is called the *inductive hypothesis*) and you only have to show how to get from $n - 1$ to n . You never have to think about all the cases at once.

4.3 Recursion is induction, run backwards

Now compare induction with a recursive method:

Induction (proving)	Recursion (computing)
Base case: prove it for $n = 0$	Base case: return the answer for $n = 0$
Assume it's true for $n - 1$	Assume <code>solve(n - 1)</code> returns the right answer
Show it's true for n , using the assumption	Compute the answer for n , using <code>solve(n - 1)</code>
Conclusion: true for all n	Conclusion: <code>solve</code> works for all n

They are the same structure. Induction climbs *up* from the base case; recursion reaches *down* to the base case and then climbs back up. And this gives us the most powerful tool in recursive thinking:

★ THE LEAP OF FAITH

When writing the recursive case, **assume the recursive call already works correctly** for the smaller input. Don't trace it. Don't wonder how it works. Just trust it, exactly like the inductive hypothesis, and ask only: "Given the correct answer for the smaller problem, how do I build the answer for my problem?"

It's called a leap of *faith* because it feels like cheating: you are using the method before you've finished writing it. But it isn't cheating. As long as the base case is correct and every call makes progress, induction *guarantees* the whole thing works.

4.4 Proving factorial correct

Let's do it once formally, so you can see that the leap of faith is not hand-waving.

● ● ● The method we want to prove correct

```
static int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

Claim: for every $n \geq 0$, `factorial(n)` returns $n!$.

Base case ($n = 0$): the method returns 1, and $0! = 1$. Correct.

Inductive step: take any $n \geq 1$, and assume `factorial(n - 1)` correctly returns $(n - 1)!$. Then `factorial(n)` returns $n \times (n - 1)!$, which by definition equals $n!$. Correct.

Progress: each call decreases n by 1, so starting from any $n \geq 0$ we reach 0 after n calls.

So `factorial` is correct for all $n \geq 0$. Notice that at no point did we trace `factorial(5)` or think about the stack. We checked **one base case** and **one step**. That is all you ever need to check.

4.5 The recipe: designing any recursive method

Here is a step-by-step recipe that turns the leap of faith into a practical method. Use it every time you face a new problem.

§ THE FIVE-STEP RECURSIVE DESIGN RECIPE

1. **Define the method in words.** Write one sentence saying exactly what the method returns (or does) for its parameters. “`sum(arr, i)` returns the sum of `arr[i]` through the end of the array.” This sentence is your contract.
2. **Find the base case(s).** For which input is the answer obvious? What is the answer?
3. **Assume the smaller problem is solved.** Pick a smaller input and pretend a genie hands you the correct answer for it, according to your sentence from step 1.
4. **Combine.** Using that answer, plus a little work, produce the answer for the current input.
5. **Check progress.** Make sure every recursive call moves strictly towards a base case.

The single most underrated step is **step 1**. If you cannot state precisely what your method returns, you cannot trust it in step 3, and everything falls apart. Many “recursion bugs” are actually “I never decided what my method means” bugs.

4.6 Worked example: sum of digits

Problem: given a non-negative integer, return the sum of its digits. For 907215 the answer is $9+0+7+2+1+5 = 24$.

Step 1 — define in words. `sumDigits(n)` returns the sum of the decimal digits of n .

Step 2 — base case. What’s the simplest number? 0 has no nonzero digits, so `sumDigits(0) = 0`. (A single-digit n would also work as a base case: `if (n < 10) return n;`.)

Step 3 — smaller problem. How do we make a number “smaller” in a way that relates to digits? Remove its last digit: `n / 10`. For 907215 that gives 90721. By the leap of faith, `sumDigits(90721)` correctly returns $9+0+7+2+1 = 19$. We don’t care how.

Step 4 — combine. The digits of 907215 are the digits of 90721 plus the last digit, 5. The last digit is `n % 10`. So the answer is `n % 10 + sumDigits(n / 10)` = $5 + 19 = 24$.

Step 5 — progress. `n / 10` is strictly smaller than n for every $n \geq 1$, so we always reach 0.

```

NumberRecursion.java  sumDigits

static int sumDigits(int n) {
    if (n == 0) return 0;
    return n % 10 + sumDigits(n / 10);    // last digit + sum of the rest
}

```

Four lines, and we never traced it. Let's do one more.

4.7 Worked example: counting a character in a string

Problem: count how many times the character `ch` appears in string `s`.

Step 1. `countChar(s, ch, i)` returns the number of times `ch` appears in `s` from index `i` to the end. (Using an index instead of cutting the string is cheaper; we'll discuss why in Chapter 10.)

Step 2. If `i == s.length()`, there are no characters left, so the count is 0.

Step 3. Trust that `countChar(s, ch, i + 1)` correctly counts occurrences in the rest of the string.

Step 4. The count from `i` onward is: 1 if `s.charAt(i) == ch` (otherwise 0), plus the count in the rest.

Step 5. `i` increases by 1 each call and stops at `s.length()`.

```

StringRecursion.java  countChar

static int countChar(String s, char ch, int i) {
    if (i == s.length()) return 0;
    return (s.charAt(i) == ch ? 1 : 0) + countChar(s, ch, i + 1);
}

```

✦ PRO TRICK: SAY THE RECURSIVE LINE OUT LOUD

Read the recursive line as an English sentence using your contract. *"The count from `i` onward equals (one if this character matches) plus the count from `i+1` onward."* If the sentence is obviously true, the code is correct. If you stumble while saying it, you've found your bug.

4.8 The delegation model

Here is one more mental model that many people find helpful. Think of each recursive call as a **person** (like the queue). Each person:

- receives a task,
- checks if it's small enough to do alone (base case),
- if not, does **one small piece** of it and **delegates** the rest to a new person who follows the same instructions,
- waits for that person's report,

- and combines the report with their own piece.

No person ever needs to know how the others do their job. The person computing `factorial(5)` does not know or care that there's a chain of four more people behind the delegation. They just need `factorial(4)` from someone and trust it to be correct.

≈ REAL-LIFE ANALOGY: A MANAGER DELEGATING

A manager is asked to count all employees in a company division. She counts herself, then asks each of her direct reports: *"How many people are in your team, including you?"* Each report asks the same question of their own reports. People with no reports just say "1". The manager adds up the answers plus 1 for herself. Nobody ever sees the whole org chart, yet the total is exact. (This is exactly how we'll count the nodes of a tree in Chapter 17.)

? TRY IT YOURSELF

Using the five-step recipe, design (on paper, before reading on) a method `power(x, n)` that returns x^n for $n \geq 0$. Write your step 1 sentence, your base case, and your combination.

→ ANSWER

1. `power(x, n)` returns x multiplied by itself n times.
2. Base case: $n = 0$ gives 1.
3. Trust `power(x, n - 1)`.
4. Combine: `x * power(x, n - 1)`.
5. n decreases by 1 each call and reaches 0. The full program is in Chapter 7, and a much faster version is in Chapter 15.

Σ CHAPTER SUMMARY

- Don't **design** recursion by tracing every call. **Trace** only to check or debug.
- Recursion mirrors **mathematical induction**: base case + "if it works for smaller, it works for this".
- **The leap of faith**: assume the recursive call returns the correct answer for the smaller input.
- **Recipe**: define in words → base case → assume smaller solved → combine → check progress.
- The most important step is defining **exactly what your method returns**.

CHAPTER 5

Tracing Recursion Like a Pro

IN THIS CHAPTER

- Learn four ways to trace a recursive method: indented logs, stack tables, recursion trees and substitution
- Build a reusable tracing technique you can apply to any recursive code
- Practice predicting the output of “mystery” recursive methods

You design recursion with the leap of faith, but you **verify** and **debug** it by tracing. Tracing is also how you read other people’s recursive code. Here are four techniques, from most practical to most visual.

5.1 Method 1: indented trace logs

The most practical debugging trick in all of recursion: add a `depth` parameter, and print indentation proportional to the depth whenever a call starts and whenever it returns.

```

RecursionTracer.java  factorialTrace

static int factorial(int n, int depth) {
    String pad = "|".repeat(depth);
    System.out.println(pad + "factorial(" + n + ") called");
    if (n == 0) {
        System.out.println(pad + "base case -> return 1");
        return 1;
    }
    int smaller = factorial(n - 1, depth + 1);
    int result = n * smaller;
    System.out.println(pad + "return " + n + " * " + smaller + " = " + result);
    return result;
}
  
```

Here is the actual output for `factorial(4, 0)`, followed by the output of a Fibonacci version we’ll look at shortly:

>_ OUTPUT OF RECURSIONTRACER.JAVA (ACTUAL RUN)

```
factorial(4) called
| factorial(3) called
| | factorial(2) called
| | | factorial(1) called
| | | | factorial(0) called
| | | | base case -> return 1
| | | | return 1 * 1 = 1
| | | return 2 * 1 = 2
| | return 3 * 2 = 6
| return 4 * 6 = 24

fib(4)
| fib(3)
| | fib(2)
| | | fib(1)
| | | => fib(1) = 1
| | | | fib(0)
| | | | => fib(0) = 0
| | | => fib(2) = 1
| | | fib(1)
| | | => fib(1) = 1
| | => fib(3) = 2
| | fib(2)
| | | fib(1)
| | | => fib(1) = 1
| | | fib(0)
| | | => fib(0) = 0
| | => fib(2) = 1
| => fib(4) = 3
```

Look at how readable the factorial part is. You can see the winding phase (the calls, moving right) and the unwinding phase (the returns, moving back left). Each vertical line of `|` connects a call to its return.

✦ PRO TRICK: THE DEPTH PARAMETER

Whenever recursive code misbehaves, add `int depth` as an extra parameter, pass `depth + 1` in every recursive call, and print `" ".repeat(depth)` before your messages. In two minutes you'll see exactly which call does something wrong. Remove it once fixed.

5.2 Method 2: the stack table

On paper, a very reliable method is to keep a table where each row is a call. Fill it **top-down** during the winding phase (writing what each call is waiting for), then fill in the return values **bottom-up** during unwinding.

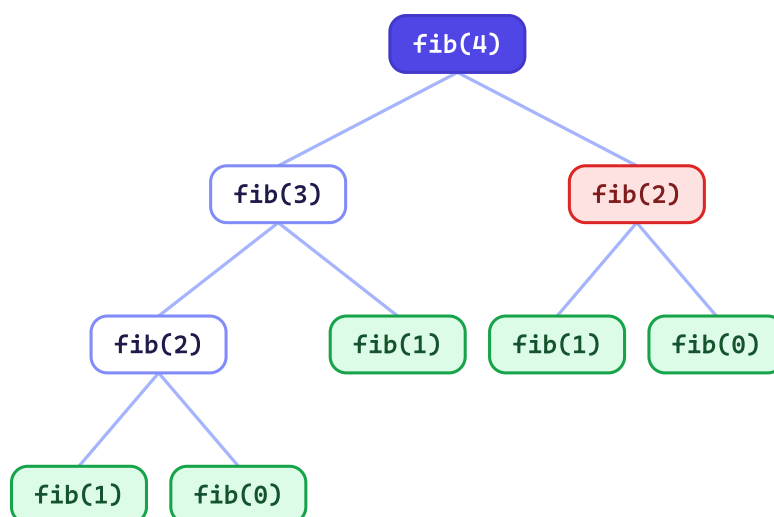
For `sumDigits(4071)`, where `sumDigits(n) = n % 10 + sumDigits(n / 10)`:

Call	<code>n % 10</code>	Waiting for	Returns (filled in bottom-up)
<code>sumDigits(4071)</code>	1	<code>sumDigits(407)</code>	$1 + 11 = 12$
<code>sumDigits(407)</code>	7	<code>sumDigits(40)</code>	$7 + 4 = 11$
<code>sumDigits(40)</code>	0	<code>sumDigits(4)</code>	$0 + 4 = 4$
<code>sumDigits(4)</code>	4	<code>sumDigits(0)</code>	$4 + 0 = 4$
<code>sumDigits(0)</code>	—	nothing (base case)	0

This table is the call stack, written sideways. The top row is the bottom of the stack.

5.3 Method 3: the recursion tree

When a method calls itself **more than once**, the calls no longer form a simple chain; they form a **tree**. Each node is a call, and its children are the calls it makes. Here is the tree for the Fibonacci method `fib(n) = fib(n-1) + fib(n-2)`:



The recursion tree of `fib(4)`: 9 calls to compute one number

■ first call ■ base case ■ repeated work

Compare this tree with the Fibonacci part of the trace output above. The indented log is simply this tree **walked depth-first**, left child before right child: `fib(4)` calls `fib(3)`, which calls `fib(2)`, which calls `fib(1)` (returns 1) and `fib(0)` (returns 0), then `fib(2)` returns 1, then `fib(3)` calls `fib(1)` ... and so on.

Two important observations:

- At any moment, the **call stack** holds just **one path** from the root down to the current node. The tree shows *all* calls over time; the stack shows the calls that are *currently* active.
- The tree reveals **wasted work**: `fib(2)` (in red) is computed twice. For larger n , the repetition explodes. Chapter 22 fixes this with memoization.

★ KEY IDEA

Stack depth = height of the recursion tree = memory used. **Number of nodes** in the tree = total number of calls = time used. Drawing the tree lets you see both at once.

5.4 Method 4: substitution (expansion)

For methods that return values through simple expressions, you can literally substitute each call with its definition until you reach base cases, like simplifying algebra:

EXPANDING POWER(2, 4) WHERE POWER(X, N) = X * POWER(X, N - 1)

```
power(2, 4)
= 2 * power(2, 3)
= 2 * (2 * power(2, 2))
= 2 * (2 * (2 * power(2, 1)))
= 2 * (2 * (2 * (2 * power(2, 0))))
= 2 * (2 * (2 * (2 * 1)))          <- base case: power(x, 0) = 1
= 16
```

This is great for building intuition about what a method *computes*, and it's how we'll derive time complexities in Chapter 11.

5.5 Tracing practice: mystery methods

The best way to become fluent is to predict outputs. For each mystery below, **work out the answer on paper first**, then check.

? MYSTERY 1

What does `mystery1(4)` print?

```
Java

static void mystery1(int n) {
    if (n == 0) return;
    mystery1(n - 1);
    System.out.print(n + " ");
    mystery1(n - 1);
}
```

→ ANSWER TO MYSTERY 1

It prints `1 2 1 3 1 2 1 4 1 2 1 3 1 2 1`. Think of it as: “print `mystery1(n-1)`, then `n`, then `mystery1(n-1)` again”. So `mystery1(1)` = “1”, `mystery1(2)` = “1 2 1”, `mystery1(3)` = “1 2 1 3 1 2 1”, and `mystery1(4)` wraps that around a 4. It makes $2^n - 1$ prints, the same count as the moves in the Tower of Hanoi (Chapter 13).

? MYSTERY 2

What does `mystery2(5)` return?

```

Java

static int mystery2(int n) {
    if (n <= 1) return n;
    return n + mystery2(n - 2);
}

```

→ ANSWER TO MYSTERY 2

$5 + \text{mystery2}(3) = 5 + (3 + \text{mystery2}(1)) = 5 + 3 + 1 = 9$. It adds every other number going down: the sum of the odd numbers up to 5. Notice the base case `n <= 1` catches both 1 and 0, so it works for even inputs too.

? MYSTERY 3

What does `mystery3(13)` print?

```

Java

static void mystery3(int n) {
    if (n == 0) return;
    mystery3(n / 2);
    System.out.print(n % 2);
}

```

→ ANSWER TO MYSTERY 3

It prints `1101`, which is 13 in binary. The recursive call comes first, so the remainders are printed during unwinding, in reverse order of computation: the highest bit first. Order of work relative to the call is the topic of the very next chapter.

Σ CHAPTER SUMMARY

- **Indented logs** with a `depth` parameter are the best practical debugging tool.
- A **stack table** on paper: fill “waiting for” top-down, fill “returns” bottom-up.
- A **recursion tree** shows all calls; the stack is one root-to-node path of it.
- **Substitution** expands calls like algebra, useful for understanding results and complexity.

P A R T

II

Basic Recursion

Building muscle: numbers, arrays and strings, one small pattern at a time.

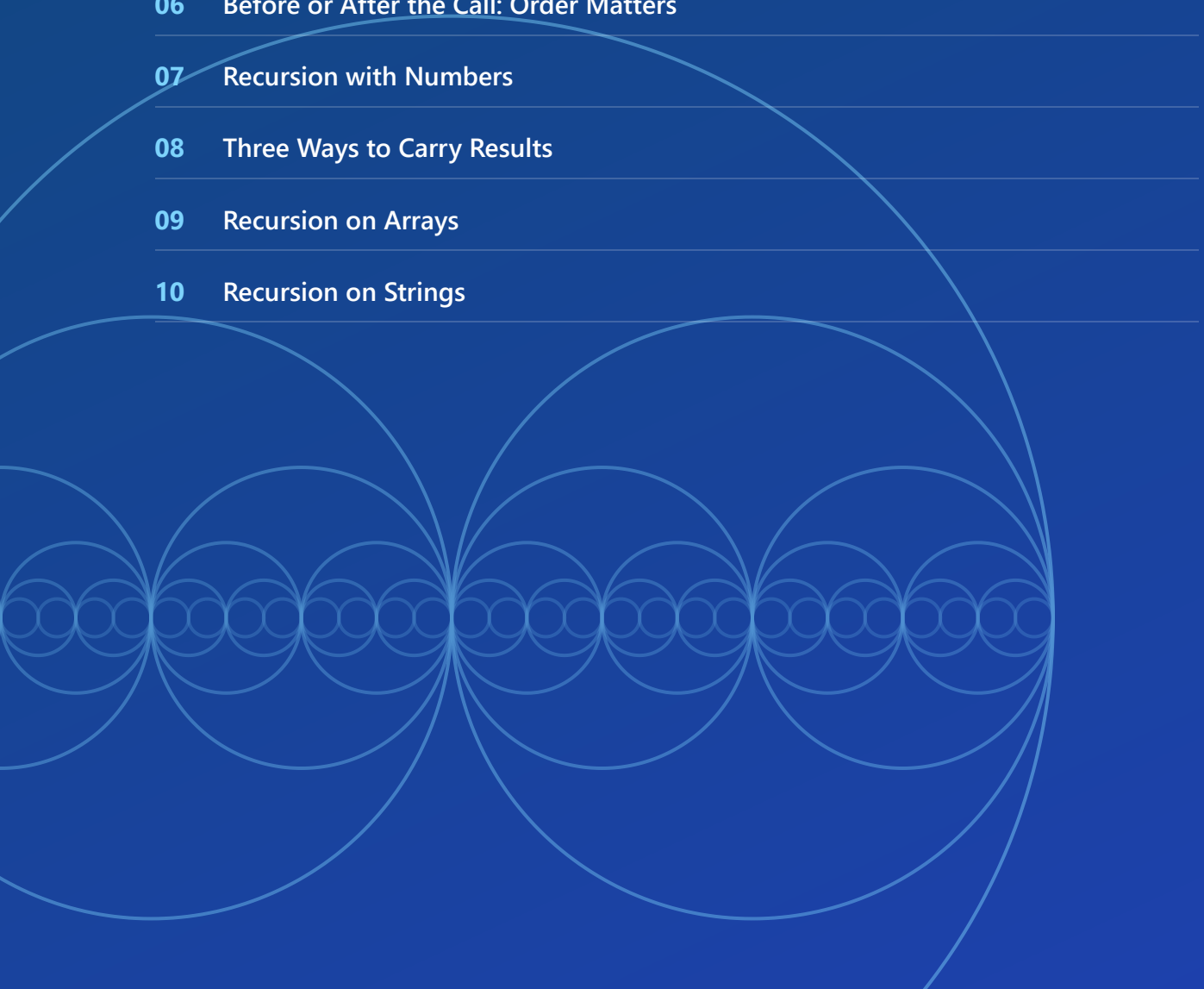
06 Before or After the Call: Order Matters

07 Recursion with Numbers

08 Three Ways to Carry Results

09 Recursion on Arrays

10 Recursion on Strings



CHAPTER 6

Before or After the Call: Order Matters

IN THIS CHAPTER

- See how moving one line (before vs. after the recursive call) completely changes the behavior
- Connect this to the winding and unwinding phases
- Learn two ways to count upward recursively
- Understand why recursion naturally reverses things

6.1 Printing n down to 1, and 1 up to n

Here are two methods. The *only* difference is whether the `print` comes before or after the recursive call.

```
BeforeAfterCall.java  printDown · printUp

static void printDown(int n) {
    if (n == 0) return;
    System.out.print(n + " ");    // work BEFORE the call (on the way down)
    printDown(n - 1);
}

static void printUp(int n) {
    if (n == 0) return;
    printUp(n - 1);
    System.out.print(n + " ");    // work AFTER the call (on the way back up)
}
```

>_ OUTPUT OF BEFOREAFTERCALL.JAVA (ACTUAL RUN)

```
printDown(5):    5 4 3 2 1
printUp(5):      1 2 3 4 5
printBoth(3):    3 2 1 1 2 3
printOneToN(1,5): 1 2 3 4 5
```

`printDown(5)` prints `5 4 3 2 1`, but `printUp(5)` prints `1 2 3 4 5`. Same calls, same base case, opposite output. Why?

6.2 Why the order flips

Remember the two phases from Chapter 3:

- Code **before** the call runs during **winding**, as the calls are being made: first in `printX(5)`, then `printX(4)`, ..., then `printX(1)`.
- Code **after** the call runs during **unwinding**, as the calls return: first in `printX(1)` (the last one called is the first one to finish), then `printX(2)`, ..., and finally `printX(5)`.

WHERE EACH PRINT HAPPENS

	<code>printDown(n)</code> (print BEFORE call)	<code>printUp(n)</code> (print AFTER call)
call <code>printX(3)</code>	prints 3, then calls	calls first, 3 waits
call <code>printX(2)</code>	prints 2, then calls	calls first, 2 waits
call <code>printX(1)</code>	prints 1, then calls	calls first, 1 waits
<code>printX(0)</code>	base case: return	base case: return
back in <code>printX(1)</code>	nothing left	prints 1
back in <code>printX(2)</code>	nothing left	prints 2
back in <code>printX(3)</code>	nothing left	prints 3
output:	3 2 1	1 2 3

Think about it with the leap of faith too. For `printUp(n)`: “First print all the numbers from 1 to $n-1$ in order (trust the recursive call), then print n .” That obviously produces 1 to n . For `printDown(n)`: “Print n , then print $n-1$ down to 1.” That obviously produces n down to 1. Both explanations are correct, and they agree with the stack explanation. That’s a good sign you understand it.

★ KEY IDEA

Work before the recursive call happens in calling order (top-down). Work after the recursive call happens in reverse order (bottom-up). This single fact explains a huge number of recursive algorithms.

6.3 Doing work on both sides

What if we print both before *and* after?

```

BeforeAfterCall.java  printBoth

static void printBoth(int n) {
    if (n == 0) return;
    System.out.print(n + " ");    // descending phase
    printBoth(n - 1);
    System.out.print(n + " ");    // ascending phase
}
```

`printBoth(3)` prints `3 2 1 1 2 3` (see the output above). The first half is printed during winding, the second half during unwinding. The base case is the “turning point” in the middle. Each call prints its own `n` twice: once on the way down, once on the way back up. This shape (do something, recurse, then do something again or *undo* something) is exactly the shape of **backtracking**, which you’ll master in Part V.

6.4 Counting up with an extra parameter

There's another way to print 1 to n that uses work *before* the call. Instead of counting n down to 0, add a second parameter i that counts *up* towards n :

```
BeforeAfterCall.java upWithIndex

static void printOneToN(int i, int n) {
    if (i > n) return;
    System.out.print(i + " ");
    printOneToN(i + 1, n);           // count UP by growing i toward n
}
```

Here the “problem size” is $n - i + 1$, the number of values still to print, and it shrinks as i grows. The base case is $i > n$: nothing left to print. This style (carry an index that moves towards a limit) is how we'll process arrays and strings in the next chapters.

Approach	Base case	Recursive call	Work position
<code>printUp(n)</code>	<code>n == 0</code>	<code>printUp(n - 1)</code>	after the call
<code>printOneToN(i, n)</code>	<code>i > n</code>	<code>printOneToN(i + 1, n)</code>	before the call

Both are correct. Knowing both gives you flexibility: sometimes it's easier to shrink from the top, sometimes to grow from the bottom.

6.5 Recursion reverses things for free

Because post-call work runs in reverse, recursion gives you **reversal without any extra data structure**. Printing a linked list backwards, printing a number's binary digits in the right order (Mystery 3 in the last chapter), reversing a string: all of these use the call stack as a free stack.

✦ PRO TRICK: NEED SOMETHING REVERSED? DO THE WORK AFTER THE CALL.

Whenever a problem says “print in reverse”, “process from the end”, or you notice you'd need to push things onto a stack and pop them, consider a recursion that does its work *after* the recursive call. The call stack will reverse the order for you.

? TRY IT YOURSELF

Write `printEvensDown(n)` that prints even numbers from n down to 2, and `printEvensUp(n)` that prints them from 2 up to n . (Assume n is even.) Then write a single method that prints `n ... 4 2 2 4 ... n`.

Σ CHAPTER SUMMARY

- Moving work from **before** to **after** the recursive call reverses the order of that work.
- Before = winding = top-down. After = unwinding = bottom-up.
- You can count upward either with post-call work or with an extra index parameter.
- The call stack gives you reversal for free.

CHAPTER 7

Recursion with Numbers

07

IN THIS CHAPTER

- Solve ten classic number problems recursively, each with the five-step recipe
- Learn the digit-peeling trick with `% 10` and `/ 10`
- Meet the helper-method pattern
- Analyze the time and space of each solution

Numbers are the perfect training ground: the problems are small, the base cases are obvious, and you can check every answer with a calculator. For each problem we'll state the **contract** (what the method returns), the **base case**, and the **recursive relation**, then show the code. All methods are in one program; its real output is at the end of the chapter.

7.1 Factorial

Contract: `factorial(n)` returns $n! = 1 \times 2 \times \dots \times n$. **Base case:** $0! = 1$ and $1! = 1$. **Relation:** $n! = n \times (n - 1)!$.

```
NumberRecursion.java factorial

static long factorial(int n) {
    if (n < 0) throw new IllegalArgumentException("n must be >= 0");
    if (n <= 1) return 1; // 0! = 1! = 1
    return n * factorial(n - 1);
}
```

Notes worth knowing:

- We return `long`, because factorials grow extremely fast. `13!` already overflows an `int`, and `21!` overflows a `long`. For bigger values, use `java.math.BigInteger`.
- Negative input has no factorial, so we throw an exception instead of recursing forever (a negative `n` would never reach the base case).
- **Time:** n calls, each doing constant work: $O(n)$. **Space:** depth n : $O(n)$.

COMMON MISTAKE: SILENT OVERFLOW

`factorial(20)` in `long` is correct (2432902008176640000), but `factorial(21)` silently wraps around to a negative number. Java does not warn you about integer overflow. If results suddenly become negative or tiny, suspect overflow, not your recursion.

7.2 Sum of 1 to n

Contract: `sumToN(n)` returns $1 + 2 + \dots + n$. **Base case:** `sumToN(0) = 0` (the sum of nothing). **Relation:** `sumToN(n) = n + sumToN(n - 1)`.

NumberRecursion.java `sumToN`

```
static int sumToN(int n) {
    if (n == 0) return 0;
    return n + sumToN(n - 1);
}
```

Of course, the formula $n(n+1)/2$ is faster. We're not using recursion here because it's the best tool for this problem, but because it's the clearest way to practice the pattern. Time $O(n)$, space $O(n)$.

7.3 Power: x to the n

Contract: `power(x, n)` returns x^n for $n \geq 0$. **Base case:** $x^0 = 1$. **Relation:** $x^n = x \times x^{n-1}$.

NumberRecursion.java `power`

```
static long power(long x, int n) {
    if (n == 0) return 1;           // x^0 = 1
    return x * power(x, n - 1);    // x^n = x * x^(n-1)
}
```

This makes n calls: $O(n)$. In Chapter 15 we'll see a trick (square the half) that brings this down to $O(\log n)$; computing 2^{62} will take 6 calls instead of 62.

7.4 Counting digits: the peeling trick

Many number problems use two operations to “peel” a number apart:

- `n % 10` gives the **last digit** ($907215 \% 10 = 5$).
- `n / 10` **removes** the last digit ($907215 / 10 = 90721$, integer division).

`n / 10` is the “smaller problem”. It's the number with one digit fewer.

Contract: `countDigits(n)` returns how many digits n has ($n \geq 0$). **Base case:** any $n < 10$ has exactly one digit. **Relation:** `digits(n) = 1 + digits(n / 10)`.

NumberRecursion.java `countDigits`

```
static int countDigits(int n) {
    if (n < 10) return 1;           // single digit
    return 1 + countDigits(n / 10); // peel off the last digit
}
```

Notice the base case is `n < 10`, not `n == 0`. Why? If we used `n == 0 → return 0`, then `countDigits(0)` would return 0, but 0 has one digit! Choosing the base case carefully handles that edge case. **Time and space: $O(d)$** where d is the number of digits, which is about $\log_{10} n$.

7.5 Sum of digits

We designed this one in Chapter 4 using the recipe:

```
NumberRecursion.java  sumDigits

static int sumDigits(int n) {
    if (n == 0) return 0;
    return n % 10 + sumDigits(n / 10);    // last digit + sum of the rest
}
```

7.6 Reversing a number: the helper-method pattern

Problem: reverse the digits of a number: 12345 → 54321.

Try to write this with the recipe and you'll hit a problem. Suppose you trust `reverse(1234) = 4321`. How do you combine it with the last digit 5 to get 54321? You'd need $5 \times 10^4 + 4321$, which requires knowing how many digits 4321 has. Possible, but clumsy.

A cleaner idea: carry the **partially built answer** along as an extra parameter. Peel digits off the end of `n` and append them to the end of `built`:

CARRYING THE ANSWER BEING BUILT

```
n = 12345, built = 0
n = 1234,  built = 0*10 + 5    = 5
n = 123,   built = 5*10 + 4    = 54
n = 12,    built = 54*10 + 3   = 543
n = 1,     built = 543*10 + 2  = 5432
n = 0,     built = 5432*10 + 1 = 54321  <- base case: return built
```

But users shouldn't have to pass that extra `0`. So we write a **public method with the clean signature** that calls a **private helper with the extra parameters**:

```
NumberRecursion.java  reverseNumber

static int reverseNumber(int n) {
    return reverseHelper(n, 0);
}

static int reverseHelper(int n, int built) {
    if (n == 0) return built;
    return reverseHelper(n / 10, built * 10 + n % 10);
}
```

✦ PRO TRICK: THE HELPER-METHOD PATTERN

When a recursion needs extra parameters (an index, an accumulator, a lower/upper bound, a list to fill), keep the public method clean and let it call a helper that carries the extra state. You'll see this pattern in almost every chapter from here on: `sum(arr)` calls `sum(arr, 0)`, `isPalindrome(s)` calls `isPalindrome(s, 0, s.length() - 1)`, and so on.

7.7 Palindrome number

A number is a palindrome if it reads the same backwards, like 12321. With `reverseNumber` already written, this is one line. Reusing a recursive method inside another is perfectly normal:

```
NumberRecursion.java  palindromeNumber

static boolean isPalindromeNumber(int n) {
    return n == reverseNumber(n);
}
```

7.8 Greatest common divisor (Euclid's algorithm)

This is one of the oldest algorithms in the world (around 300 BC), and it's naturally recursive.

Contract: `gcd(a, b)` returns the largest number that divides both `a` and `b`. **Base case:** `gcd(a, 0) = a` (every number divides 0, so the answer is `a` itself). **Relation:** `gcd(a, b) = gcd(b, a mod b)`.

Why is the relation true? Any number that divides both `a` and `b` also divides `a - b`, `a - 2b`, and so on, down to `a mod b`. So the common divisors of `(a, b)` are exactly the common divisors of `(b, a mod b)`. The numbers shrink quickly, which makes this extremely fast.

```
NumberRecursion.java  gcd

static int gcd(int a, int b) {
    if (b == 0) return a;           // gcd(a, 0) = a
    return gcd(b, a % b);          // Euclid: gcd(a, b) = gcd(b, a mod b)
}
```

TRACE OF GCD(84, 36)

```
gcd(84, 36) -> 84 % 36 = 12 -> gcd(36, 12)
gcd(36, 12) -> 36 % 12 = 0  -> gcd(12, 0)
gcd(12, 0)  -> base case   -> 12
```

Time: $O(\log(\min(a, b)))$. Euclid's algorithm is so fast that it's used inside cryptography libraries today.

7.9 Decimal to binary

Contract: `toBinary(n)` returns the binary representation of `n` as a `String`. **Base case:** 0 and 1 are their own binary representation. **Relation:** `binary(n) = binary(n / 2)` followed by `(n % 2)`.

The last binary digit of `n` is `n % 2`, and the remaining digits are the binary form of `n / 2`. Because we *append* the last bit *after* the recursive result, the bits come out in the correct order: most significant first.

NumberRecursion.java toBinary

```
static String toBinary(int n) {
    if (n < 2) return String.valueOf(n);    // "0" or "1"
    return toBinary(n / 2) + (n % 2);      // higher bits first, then last bit
}
```

TRACE OF TOBINARY(37)

```
toBinary(37) = toBinary(18) + "1"
toBinary(18) = toBinary(9) + "0"
toBinary(9)  = toBinary(4) + "1"
toBinary(4)  = toBinary(2) + "0"
toBinary(2)  = toBinary(1) + "0"
toBinary(1)  = "1"                                <- base case
unwinding:   "1" -> "10" -> "100" -> "1001" -> "10010" -> "100101"
```

7.10 Multiplication using only addition

Just to show that recursion can build complex operations from simple ones: $a \times b = a + a \times (b - 1)$, with $a \times 0 = 0$.

NumberRecursion.java multiply

```
static int multiply(int a, int b) {
    if (b == 0) return 0;                // a * 0 = 0
    return a + multiply(a, b - 1);       // a * b = a + a * (b-1)
}
```

7.11 The results

>_ OUTPUT OF NUMBERRECURSION.JAVA (ACTUAL RUN)

```
factorial(10)      = 3628800
sumToN(100)       = 5050
power(2, 10)      = 1024
countDigits(907215) = 6
sumDigits(907215) = 24
reverseNumber(12345) = 54321
isPalindrome(12321) = true
isPalindrome(12345) = false
gcd(84, 36)       = 12
toBinary(37)      = 100101
multiply(7, 6)    = 42
```

Problem	Base case	Relation	Time	Space
factorial	$n \leq 1 \rightarrow 1$	$n \times f(n-1)$	$O(n)$	$O(n)$
sumToN	$n = 0 \rightarrow 0$	$n + f(n-1)$	$O(n)$	$O(n)$
power	$n = 0 \rightarrow 1$	$x \times f(n-1)$	$O(n)$	$O(n)$
countDigits	$n < 10 \rightarrow 1$	$1 + f(n/10)$	$O(\log n)$	$O(\log n)$
sumDigits	$n = 0 \rightarrow 0$	$n \% 10 + f(n/10)$	$O(\log n)$	$O(\log n)$
reverseNumber	$n = 0 \rightarrow \text{built}$	$f(n/10, \text{built} \cdot 10 + n \% 10)$	$O(\log n)$	$O(\log n)$
gcd	$b = 0 \rightarrow a$	$f(b, a \bmod b)$	$O(\log n)$	$O(\log n)$
toBinary	$n < 2 \rightarrow "n"$	$f(n/2) + n \% 2$	$O(\log n)$	$O(\log n)^*$

(String concatenation adds hidden costs; see Chapter 11.)

? TRY IT YOURSELF

Write recursive methods for: (a) the product of the digits of n ; (b) the number of zeros in n ; (c) whether n is a power of 2; (d) the sum $1 + 1/2 + 1/4 + \dots + 1/2^n$ as a `double`.

Σ CHAPTER SUMMARY

- `% 10` peels the last digit, `/ 10` removes it: the standard way to make a number “smaller” digit by digit.
- Choose base cases carefully: `countDigits` needs `n < 10`, not `n == 0`.
- When combining is awkward, **carry the partial answer** in a parameter and use a **helper method**.
- Recursion depth for digit problems is logarithmic, so it’s always safe.

CHAPTER 8

Three Ways to Carry Results

IN THIS CHAPTER

- Compare the three ways a recursive method can produce its answer
- Understand the “functional” and “parameterized” styles and when to use each
- Understand why global/static state is dangerous in recursion

In the last chapter, `sumDigits` returned its answer through return values, but `reverseNumber` carried its answer in a parameter. These are two different *styles* of recursion, and there is a third. Knowing all three, and their trade-offs, makes you far more flexible.

8.1 Style 1: functional (return values)

The answer is built **on the way back up**. Each call returns its result, and the caller combines it.

```

RecursionStyles.java  functional

// Style 1 -- FUNCTIONAL: the answer comes back through return values.
static int sumFunctional(int n) {
    if (n == 0) return 0;
    return n + sumFunctional(n - 1);
}

```

- The combination (`n + ...`) happens **after** the recursive call returns, during unwinding.
- The method is **pure**: same input, same output, no side effects. Easy to test, easy to reason about with the leap of faith.
- This is the style of most mathematical definitions.

8.2 Style 2: parameterized (carry the answer down)

The partial answer travels **on the way down**, as an argument. When the base case is reached, the answer is complete, and it's printed (or stored).

```

RecursionStyles.java parameterized

// Style 2 -- PARAMETERIZED: the partial answer travels DOWN as an argument.
static void sumParameterized(int n, int soFar) {
    if (n == 0) {
        System.out.println("sumParameterized result = " + soFar);
        return;
    }
    sumParameterized(n - 1, soFar + n);
}

```

- The work (`soFar + n`) happens **before** the recursive call, during winding.
- Nothing needs to happen after the call returns.
- Very natural when the goal is to **print** or **collect** results at the base case (all subsets, all paths, all permutations). You'll use this constantly in backtracking.

A common hybrid is an **accumulator** that is also *returned* from the base case. This combines the “work on the way down” of style 2 with the clean return value of style 1:

```

RecursionStyles.java accumulatorReturn

// Style 2b -- ACCUMULATOR that also RETURNS the final value.
static int sumAccumulator(int n, int soFar) {
    if (n == 0) return soFar;
    return sumAccumulator(n - 1, soFar + n);
}

```

This is exactly the shape of `reverseHelper` from the last chapter, and it's the shape of **tail recursion**, which Chapter 12 explores.

8.3 Style 3: global or static state

The answer is accumulated into a field (a `static` variable) that lives outside the method.

```

RecursionStyles.java global

// Style 3 -- GLOBAL / FIELD state (works, but avoid it: hidden, not reusable).
static int total = 0;

static void sumGlobal(int n) {
    if (n == 0) return;
    total += n;
    sumGlobal(n - 1);
}

```

It works, as the output below shows, but it has real problems:

- **You must remember to reset it** before every use (`total = 0`). Forget once, and the second call returns a wrong answer that includes the first call's result.
- **It is invisible.** Reading `sumGlobal(10)`, you can't tell where the answer goes.
- **It isn't reusable or thread-safe.** Two uses at the same time corrupt each other.
- **It breaks the leap of faith**, because a call's behavior depends on hidden state, not just its parameters.

>_ OUTPUT OF RECURSIONSTYLES.JAVA (ACTUAL RUN)

```
sumFunctional(10)      = 55
sumParameterized result = 55
sumAccumulator(10, 0)  = 55
sumGlobal(10) -> total = 55
```

! AVOID STATIC STATE IN RECURSION

Use a static field only for things like counting calls while debugging (we do that in Chapter 13 to count Fibonacci calls). For real answers, use return values or parameters. If you need a mutable "box" that many calls update (for example, the best answer seen so far), pass an object or a one-element array as a parameter, as we'll do with the tree diameter in Chapter 17.

8.4 When to use which

Style	Answer built	Best for	Example
Functional	on the way up , via return	computing one value; math-style problems; trees	factorial, height of a tree
Parameterized	on the way down , via arguments	printing/collecting many results; paths; choices	all subsets, all paths
Accumulator + return	down via argument, returned at base	tail recursion; building a value left-to-right	reverse a number
Global state	in a field	quick debugging counters only	counting calls

★ KEY IDEA

Functional style asks: "What does the smaller problem give back to me?" **Parameterized style asks: "What have I built so far, and what do I pass on?"** Both are valid ways to think; some problems are much easier in one than the other.

Σ CHAPTER SUMMARY

- **Functional:** answers flow up through return values. Pure, clean, ideal for single results.
- **Parameterized:** partial answers flow down through arguments. Ideal for printing or collecting results.
- **Accumulator:** flows down, returned at the base case; it's the shape of tail recursion.
- **Global state:** works but is fragile. Avoid it for real answers.

CHAPTER 9

Recursion on Arrays

IN THIS CHAPTER

- Learn the index trick: recursion on an array without copying it
- Understand what “the smaller problem” means for an array
- Solve ten classic array problems recursively
- Learn the difference between deciding *before* and *after* the recursive call

9.1 The index trick

How do you make an array “smaller”? You could create a new array without the first element, but that means copying $n - 1$ elements on every call, which is slow and wasteful.

Instead, we keep the **same array** and pass an **index** `i` that marks where the “remaining” part starts. The smaller problem is the **suffix** of the array starting at `i + 1`.

THE SAME ARRAY, VIEWED AS SMALLER AND SMALLER SUFFIXES

```
index:      0   1   2   3   4   5   6
array:    [ 4, 7, 1, 7, 9, 2, 7 ]

f(a, 0) works on [ 4, 7, 1, 7, 9, 2, 7 ]
f(a, 1) works on  [ 7, 1, 7, 9, 2, 7 ]
f(a, 2) works on  [ 1, 7, 9, 2, 7 ]
...
f(a, 6) works on  [ 7 ]
f(a, 7) works on  [ ]      <- i == a.length: empty suffix (base case)
```

★ KEY IDEA

For array recursion, **define your method in terms of the suffix** `a[i..n-1]`. For example: “`sum(a, i)` returns the sum of `a[i]` through `a[n-1]`.” Then the base case is `i == a.length` (empty suffix), and the smaller problem is always `f(a, i + 1)`.

All the examples in this chapter use the array `{4, 7, 1, 7, 9, 2, 7}`, and the real output is at the end.

9.2 Printing and summing

```

● ● ● ArrayRecursion.java printArray · sum

static void printArray(int[] a, int i) {
    if (i == a.length) return;           // walked past the last index
    System.out.print(a[i] + " ");
    printArray(a, i + 1);
}

// sum(a, i) = a[i] + a[i+1] + ... + a[n-1]
static int sum(int[] a, int i) {
    if (i == a.length) return 0;         // empty suffix sums to 0
    return a[i] + sum(a, i + 1);
}

```

`sum` reads as: “the sum of the suffix from i is $a[i]$ plus the sum of the suffix from $i+1$; the sum of an empty suffix is 0”. Every array method in this chapter follows the same pattern: **handle one element, trust the recursion for the rest**.

9.3 Maximum element

```

● ● ● ArrayRecursion.java max

// max(a, i) = largest value among a[i..n-1]
static int max(int[] a, int i) {
    if (i == a.length - 1) return a[i]; // one element: it is the max
    return Math.max(a[i], max(a, i + 1));
}

```

Here the base case is a suffix with **one** element (`i == a.length - 1`), because “the maximum of an empty array” has no sensible value. That means this method assumes the array is non-empty. (Alternatively, you could return `Integer.MIN_VALUE` for the empty suffix, the neutral element for `max`, like 0 for sums and 1 for products.)

9.4 Linear search: first occurrence

```

● ● ● ArrayRecursion.java firstIndex

static int firstIndex(int[] a, int target, int i) {
    if (i == a.length) return -1;         // searched everything: not found
    if (a[i] == target) return i;         // found -- stop early
    return firstIndex(a, target, i + 1);
}

```

Two ways to stop: we ran out of elements (return `-1`), or we found the target (return `i`). The found case is an **early exit**: once found, no more calls are made.

9.5 Last occurrence: deciding after the call

Now a more interesting one. To find the **last** index of the target, we could search from the end. But here is a more instructive solution that still walks left to right:

```

ArrayRecursion.java  lastIndex

static int lastIndex(int[] a, int target, int i) {
    if (i == a.length) return -1;
    int inRest = lastIndex(a, target, i + 1); // ask the REST of the array first
    if (inRest != -1) return inRest;          // a later match wins
    return a[i] == target ? i : -1;          // otherwise, is it me?
}

```

The idea: “First ask the rest of the array (everything to my right) whether it contains the target. If it does, that index is further right than me, so it wins. Only if the rest has no match do I check myself.” The decision is made **after** the recursive call returns, during unwinding, which means the rightmost match is found first on the way back. Compare it with `firstIndex`, which decides **before** the call.

★ KEY IDEA

Decide before the call when the current element takes priority (first occurrence, early exit). **Decide after the call** when the rest of the input takes priority (last occurrence), or when you need the answer from the rest to decide.

9.6 All occurrences: collecting into a list

To collect every index, we pass a list down and add to it (the parameterized style):

```

ArrayRecursion.java  allIndices

static void allIndices(int[] a, int target, int i, List<Integer> out) {
    if (i == a.length) return;
    if (a[i] == target) out.add(i);
    allIndices(a, target, i + 1, out);
}

```

All calls share **the same list object** (Java passes the reference), so every `add` goes into the one list the caller created.

9.7 Counting occurrences

● ● ● ArrayRecursion.java countOccurrences

```
static int count(int[] a, int target, int i) {
    if (i == a.length) return 0;
    return (a[i] == target ? 1 : 0) + count(a, target, i + 1);
}
```

This is the functional twin of `allIndices`: instead of collecting indices, each call returns a count and the counts are added on the way up.

9.8 Is the array sorted?

● ● ● ArrayRecursion.java isSorted

```
static boolean isSorted(int[] a, int i) {
    if (i >= a.length - 1) return true;      // 0 or 1 element left: sorted
    if (a[i] > a[i + 1]) return false;       // found a violation
    return isSorted(a, i + 1);
}
```

An array is sorted if the first pair is in order **and** the rest is sorted. We check the current pair; if it's bad, return `false` immediately; otherwise, trust the recursion for the rest.

9.9 Reversing in place: two pointers

Some problems shrink from **both ends**. To reverse an array in place, swap the first and last elements, then reverse the part in between:

● ● ● ArrayRecursion.java reverse

```
static void reverse(int[] a, int left, int right) {
    if (left >= right) return;                // pointers met: done
    int tmp = a[left];
    a[left] = a[right];
    a[right] = tmp;
    reverse(a, left + 1, right - 1);          // shrink from both ends
}
```

REVERSING WITH TWO POINTERS

```
[4, 7, 1, 7, 9, 2, 7]  left=0, right=6: swap -> [7, 7, 1, 7, 9, 2, 4]
      ^               ^
[7, 7, 1, 7, 9, 2, 4]  left=1, right=5: swap -> [7, 2, 1, 7, 9, 7, 4]
      ^               ^
[7, 2, 1, 7, 9, 7, 4]  left=2, right=4: swap -> [7, 2, 9, 7, 1, 7, 4]
      ^               ^
left=3, right=3: left >= right -> base case, done
```

The problem size is `right - left + 1`, and it shrinks by 2 each call. The base case `left >= right` covers both the odd case (pointers meet) and the even case (pointers cross).

9.10 The results

>_ OUTPUT OF ARRAYRECURSION.JAVA (ACTUAL RUN)

```
printArray      : 4 7 1 7 9 2 7
sum             : 37
max             : 9
firstIndex(7)   : 1
lastIndex(7)    : 6
allIndices(7)   : [1, 3, 6]
count(7)        : 3
isSorted(a)     : false
isSorted(1..5)  : true
reversed        : [7, 2, 9, 7, 1, 7, 4]
```

✦ PRO TRICK: THE ARRAY RECURSION CHECKLIST

1. Write the contract in terms of the suffix `a[i..]` (or a range `a[low..high]`).
2. Base case: `i == a.length` (empty) unless an empty answer makes no sense.
3. Recursive call: `f(a, i + 1)` (or shrink `low` / `high`).
4. Decide whether your work goes before the call (current element first) or after (rest first).
5. Never copy the array; pass indices.

? TRY IT YOURSELF

Write recursive methods to: (a) find the minimum; (b) check whether all elements are even; (c) return the second-largest element; (d) check whether an array is a palindrome using two pointers; (e) count how many elements are greater than their left neighbor.

Σ CHAPTER SUMMARY

- Recursion on arrays uses an **index** (or two) instead of copying; the smaller problem is the suffix `a[i+1..]`.
- The pattern: **handle one element, trust recursion for the rest**.
- Deciding **before** vs **after** the call changes which element wins (first vs last occurrence).
- Two-pointer recursion shrinks from both ends; base case `left >= right`.

CHAPTER 10

Recursion on Strings

10

IN THIS CHAPTER

- Apply recursion to strings in two styles: `substring` and index-based
- Understand the hidden cost of `substring` and string concatenation
- Solve nine classic string problems
- Master the “first character + solve the rest” pattern

Strings are a lot like arrays of characters, so everything from the last chapter applies. There is one new consideration: Java strings are **immutable**, so “making a string smaller” either means creating a new, shorter string (with `substring`) or keeping an index into the original.

10.1 `substring` vs. index: a cost warning

`s.substring(1)` creates a **brand new string** with a copy of all characters except the first. That’s $O(n)$ work and $O(n)$ memory on every call. For a string of length n , a recursion that calls `substring(1)` n times does $1 + 2 + \dots + n \approx n^2/2$ character copies in total: **$O(n^2)$** time.

For learning and for short strings, `substring` is fine: it makes the code read beautifully. For performance-sensitive code or interviews, prefer **indices** (and a `StringBuilder` for building output). We’ll show both styles.

! HIDDEN QUADRATIC COST

`substring` and `+` on strings each copy characters. A recursion that uses them on every call is usually $O(n^2)$ overall, even though it looks like $O(n)$. This rarely matters for strings of length 20, but it matters a lot for strings of length 100,000 (and those would overflow the stack anyway).

10.2 Length of a string

```
StringRecursion.java  length

static int length(String s) {
    if (s.isEmpty()) return 0;
    return 1 + length(s.substring(1));    // 1 for the first char + length of the rest
}
```

The simplest string recursion: the length is 1 (for the first character) plus the length of everything after it. The empty string has length 0.

10.3 Reversing a string

```
StringRecursion.java reverse

static String reverse(String s) {
    if (s.length() <= 1) return s;          // "" and "a" are their own reverse
    return reverse(s.substring(1)) + s.charAt(0);
}
```

With the leap of faith: “The reverse of `s` is the reverse of everything after the first character, followed by the first character.” For “stack”: `reverse("tack") + "s" = "kcat" + "s" = "kcats"`.

UNWINDING BUILDS THE REVERSED STRING

```
reverse("stack") = reverse("tack") + 's'
reverse("tack")  = reverse("ack")  + 't'
reverse("ack")   = reverse("ck")   + 'a'
reverse("ck")    = reverse("k")    + 'c'
reverse("k")     = "k"              <- base case
unwinding: "k" -> "kc" -> "kca" -> "kcat" -> "kcats"
```

The efficient, index-based version walks from the **end** of the string and appends into a shared `StringBuilder`:

```
StringRecursion.java reverseFast

static void reverseInto(String s, int i, StringBuilder out) {
    if (i < 0) return;
    out.append(s.charAt(i));          // take chars from the END
    reverseInto(s, i - 1, out);
}
```

10.4 Palindrome check

```
StringRecursion.java palindrome

static boolean isPalindrome(String s, int left, int right) {
    if (left >= right) return true;      // 0 or 1 char left
    if (s.charAt(left) != s.charAt(right)) return false; // mismatch
    return isPalindrome(s, left + 1, right - 1); // check the inside
}
```

Two pointers again, exactly like reversing an array. “racecar” is checked as `r = r?` `a = a?` `c = c?` then the pointers meet at “e”, and the base case returns true.

✦ PRO TRICK: STOP AT THE FIRST MISMATCH

Put the “failure” check (`charAt(left) != charAt(right) → false`) **before** the recursive call. Then the recursion stops as soon as it finds a mismatch, instead of checking the whole string. Early exits like this are a free speedup in any “check whether all ...” recursion.

10.5 Removing a character

StringRecursion.java removeChar

```
static String removeChar(String s, char ch) {
    if (s.isEmpty()) return "";
    String rest = removeChar(s.substring(1), ch); // clean the rest first
    char first = s.charAt(0);
    return first == ch ? rest : first + rest;      // keep or drop the first char
}
```

This is the “first character + solve the rest” pattern in its purest form: *“Clean the rest of the string (trust it). Then either keep my first character in front of it, or drop it.”* For “banana” with ‘a’, the result is “bnn”.

10.6 Replacing “pi” with “3.14”

A classic that needs to look at **two** characters:

StringRecursion.java replacePi

```
static String replacePi(String s) {
    if (s.length() < 2) return s;
    if (s.startsWith("pi")) return "3.14" + replacePi(s.substring(2));
    return s.charAt(0) + replacePi(s.substring(1));
}
```

If the string starts with “pi”, output “3.14” and skip **two** characters; otherwise, keep one character and skip one. Different branches can shrink the input by different amounts; that’s perfectly fine as long as every branch shrinks it.

10.7 Removing consecutive duplicates

```
StringRecursion.java dedupe

static String removeConsecutiveDuplicates(String s) {
    if (s.length() <= 1) return s;
    if (s.charAt(0) == s.charAt(1)) {
        return removeConsecutiveDuplicates(s.substring(1)); // drop the first copy
    }
    return s.charAt(0) + removeConsecutiveDuplicates(s.substring(1));
}
```

If the first two characters are equal, drop the first and recurse (the second copy will be handled in the next call). Otherwise, keep the first character. “aaabccddda” becomes “abcda”.

10.8 Moving all ‘x’ characters to the end

```
StringRecursion.java moveX

static String moveXToEnd(String s) {
    if (s.isEmpty()) return "";
    String rest = moveXToEnd(s.substring(1));
    return s.charAt(0) == 'x' ? rest + 'x' : s.charAt(0) + rest;
}
```

Solve the rest first; then, if my character is an ‘x’, put it at the **end** of the solved rest, otherwise at the **front**. Since the rest is already arranged correctly (leap of faith), the result is correct.

10.9 The results

>_ OUTPUT OF STRINGRECURSION.JAVA (ACTUAL RUN)

```
length("recursion")      = 9
reverse("stack")         = kcats
reverseInto("frames")    = semarf
isPalindrome("racecar")  = true
isPalindrome("rocket")   = false
removeChar("banana", 'a') = bnn
countChar("mississippi", 's') = 4
replacePi("pippxxppiixipi") = 3.14ppxxp3.14ixi3.14
dedupe("aaabccddda")     = abcda
moveXToEnd("axbxcxd")    = abcdxxx
```

★ KEY IDEA: THE STRING RECURSION PATTERN

Almost every string recursion is: “Handle the first character (or the first few), trust the recursion for the rest, then combine.” The only question is how you combine: put the character in front, at the end, drop it, replace it, or compare it.

? TRY IT YOURSELF

Write recursive methods to: (a) convert a string to uppercase; (b) check whether a string contains only digits; (c) insert a `*` between identical adjacent characters ("hello" → "hel*lo"); (d) count the vowels; (e) check whether one string is a subsequence of another.

Σ CHAPTER SUMMARY

- Strings recurse like arrays: by `substring` (clear but $O(n^2)$ overall) or by index (efficient).
- Pattern: **first character + solve the rest**, then combine (keep, drop, move, replace, compare).
- Put failure checks **before** the recursive call to exit early.
- Different branches may shrink the input by different amounts.

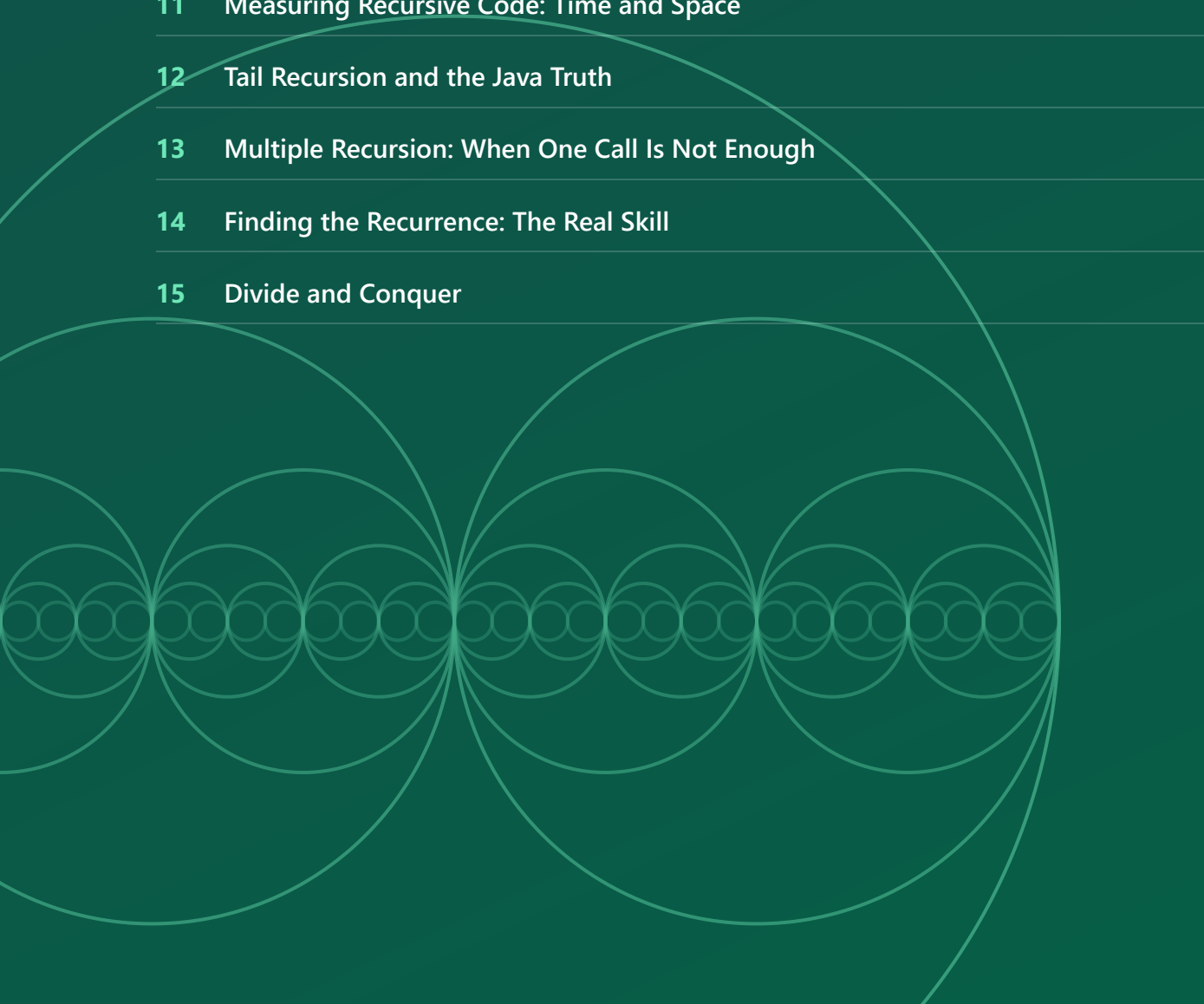
P A R T

III

Deeper Recursion

Measuring recursion, tail calls, many-branched recursion, the art of finding recurrences, and divide and conquer.

- 11 Measuring Recursive Code: Time and Space
- 12 Tail Recursion and the Java Truth
- 13 Multiple Recursion: When One Call Is Not Enough
- 14 Finding the Recurrence: The Real Skill
- 15 Divide and Conquer



CHAPTER 11

Measuring Recursive Code: Time and Space

11

IN THIS CHAPTER

- Learn to count the work done by any recursive method
- Write and solve recurrence relations
- Use the recursion-tree method for many-branched recursion
- Memorize the table of common recurrences
- Measure space correctly (depth, not total calls)

11.1 Two questions that give you the answer

For a loop, you estimate running time by asking how many times the loop body runs. For recursion, ask two questions:

1. **How many calls are made in total?** (the number of nodes in the recursion tree)
2. **How much work does each call do**, *not counting* the work inside its recursive calls?

Total time = sum of the work over all calls. When every call does the same amount of work, that's simply (number of calls) $\times$ (work per call).

For `factorial(n)`: $n + 1$ calls, each doing a comparison and a multiplication (constant work), so **$O(n)$** .

For `binarySearch` on n elements: each call halves the range, so there are about $\log_2 n$ calls, each constant work: **$O(\log n)$** .

11.2 Recurrence relations

A more systematic tool: write a formula for the running time **in terms of itself**. Yes, we analyze recursive code with a recursive formula!

Let $T(n)$ be the time for input size n . Look at the code and translate:

TRANSLATING CODE INTO A RECURRENCE

```
static int factorial(int n) {
    if (n == 0) return 1;           -> constant work, call it c
    return n * factorial(n - 1);    -> c + T(n - 1)
}
```

$$T(0) = c$$

$$T(n) = T(n - 1) + c$$

Each piece of the method becomes a piece of the formula: the recursive calls become $T(\text{smaller sizes})$, and everything else is the “work per call”.

11.3 Solving recurrences by expansion

The simplest way to solve a recurrence is to substitute it into itself until you see the pattern (the same substitution technique from Chapter 5).

Pattern 1: $T(n) = T(n - 1) + c$ (factorial, sum, linear search)

$$\begin{aligned} T(n) &= T(n-1) + c \\ &= T(n-2) + c + c \\ &= T(n-3) + 3c \\ &= \dots \\ &= T(0) + n \cdot c \quad \rightarrow O(n) \end{aligned}$$

Pattern 2: $T(n) = T(n / 2) + c$ (binary search, fast power)

$$\begin{aligned} T(n) &= T(n/2) + c \\ &= T(n/4) + 2c \\ &= T(n/8) + 3c \\ &= \dots \quad \text{after } k \text{ steps: } T(n/2^k) + k \cdot c \\ n/2^k &= 1 \text{ when } k = \log_2(n) \\ &= T(1) + c \cdot \log_2(n) \quad \rightarrow O(\log n) \end{aligned}$$

Pattern 3: $T(n) = T(n - 1) + n$ (a recursion that does a loop of n work per call, or **substring** on every call)

$$T(n) = n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2 \quad \rightarrow O(n^2)$$

Pattern 4: $T(n) = 2 \cdot T(n - 1) + c$ (Tower of Hanoi, naive subsets)

$$\begin{aligned} T(n) &= 2T(n-1) + c \\ &= 4T(n-2) + 2c + c \\ &= 8T(n-3) + 4c + 2c + c \\ &= \dots \\ &= 2^n \cdot T(0) + (2^n - 1) \cdot c \quad \rightarrow O(2^n) \end{aligned}$$

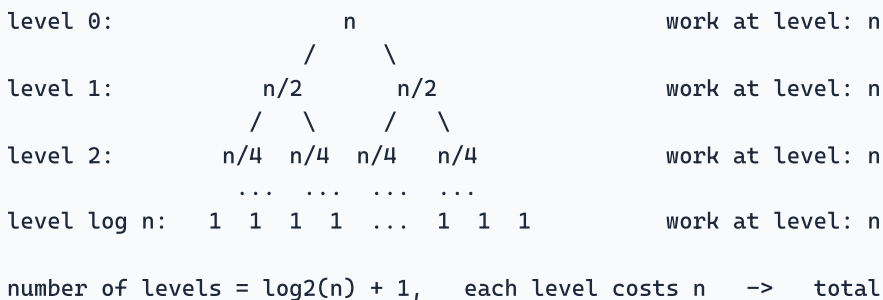
Pattern 5: $T(n) = 2 \cdot T(n/2) + n$ (merge sort)

This one is best seen with a tree.

11.4 The recursion-tree method

Draw the recursion tree, write the work done **at each node**, add up each **level**, then add up the levels.

RECURSION TREE FOR $T(N) = 2T(N/2) + N$ (MERGE SORT)



The same method explains Fibonacci: each call makes 2 calls, the tree has depth about n , so there are up to about 2^n nodes. (More precisely, about 1.618^n , because the right branch is shorter. Either way, it's exponential.)

11.5 The table to memorize

Recurrence	Example	Solution
$T(n) = T(n-1) + O(1)$	factorial, linear search, list length	$O(n)$
$T(n) = T(n/2) + O(1)$	binary search, fast power	$O(\log n)$
$T(n) = T(n-1) + O(n)$	selection sort recursively, substring recursion	$O(n^2)$
$T(n) = 2T(n/2) + O(1)$	max by divide and conquer, tree traversal	$O(n)$
$T(n) = 2T(n/2) + O(n)$	merge sort, quick sort (average)	$O(n \log n)$
$T(n) = 2T(n-1) + O(1)$	Tower of Hanoi, all subsets	$O(2^n)$
$T(n) = T(n-1) + T(n-2) + O(1)$	naive Fibonacci	$O(1.618^n)$
$T(n) = n \cdot T(n-1) + O(1)$	all permutations	$O(n!)$

✦ PRO TRICK: A QUICK RULE OF THUMB

Branches ^ depth. If each call makes b recursive calls and the depth is d , there are up to about b^d calls. One branch with depth n : n calls. Two branches with depth n : 2^n . Two branches with depth $\log n$: $2^{(\log n)} = n$. This rough estimate gets you the right complexity class most of the time.

11.6 Space complexity: depth, not total calls

Memory used by the call stack is determined by the **maximum number of frames alive at the same time**, which is the **depth** of the recursion tree, not its size.

Method	Total calls (time)	Max depth (stack space)
factorial(n)	n	n
binarySearch	log n	log n
fib(n) naive	$\sim 1.618^n$	n
mergeSort	$\sim 2n$	log n (plus $O(n)$ for the temporary arrays)
all subsets of n items	2^n	n

So naive `fib(40)` makes over 300 million calls but never has more than 40 frames on the stack at once. It's slow, but it won't overflow.

! DON'T FORGET NON-STACK SPACE

Recursion that creates new strings (`substring` , `+`), new arrays, or copies of lists uses heap memory too. A method whose stack depth is $O(n)$ but that creates a new string of length n in each frame can hold $O(n^2)$ memory at once.

Σ CHAPTER SUMMARY

- **Time** = total work over all calls = (number of calls) $\times$ (work per call), when work is uniform.
- Write a **recurrence** by translating the code; solve it by **expansion** or with a **recursion tree**.
- Memorize the common recurrences: $O(n)$, $O(\log n)$, $O(n^2)$, $O(n \log n)$, $O(2^n)$, $O(n!)$.
- **Space** = maximum depth of the recursion (plus any heap memory you allocate).

CHAPTER 12

Tail Recursion and the Java Truth

12

IN THIS CHAPTER

- Define tail recursion precisely, and spot it
- Convert non-tail recursion into tail recursion with an accumulator
- Convert tail recursion into a loop, mechanically
- Learn the important truth: Java does *not* optimize tail calls

12.1 Tail vs. non-tail

A recursive call is a **tail call** if it is the **very last thing** the method does: after the call returns, the method immediately returns that same value, with no further work. A method whose recursive calls are all tail calls is **tail recursive**.

```

TailRecursion.java nonTail

static long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);    // NOT tail: the multiply waits for the call
}
```

This is **not** tail recursive. After `factorial(n - 1)` returns, there's still a multiplication by `n` to do. That pending multiplication is exactly why every frame has to stay on the stack.

```

TailRecursion.java tail

static long factorialTail(int n, long acc) {
    if (n <= 1) return acc;
    return factorialTail(n - 1, acc * n);    // TAIL: the call is the very last action
}
```

This **is** tail recursive. The multiplication happens *before* the call (in the argument `acc * n`), and the result of the recursive call is returned directly, untouched. When the call returns, the current frame has nothing left to do.

✗ **COMMON MISTAKE: 'THE CALL IS ON THE LAST LINE, SO IT'S TAIL RECURSIVE'**

`return n * factorial(n - 1);` is on the last line, but it is **not** a tail call, because the multiplication happens after the call returns. What matters is whether any work remains *after* the call, not which line the call is on.

Code	Tail call?	Why
<code>return f(n - 1);</code>	Yes	result returned directly
<code>return f(n - 1, acc * n);</code>	Yes	work done in the argument, before the call
<code>return n * f(n - 1);</code>	No	multiply after the call
<code>return 1 + f(n - 1);</code>	No	add after the call
<code>f(n - 1); print(n);</code>	No	print after the call
<code>print(n); f(n - 1);</code>	Yes	nothing after the call
<code>return f(n - 1) + f(n - 2);</code>	No	addition after both calls

12.2 Converting to tail recursion with an accumulator

The recipe: add a parameter (the **accumulator**) that carries the partial result **down**, and do the combining **before** the call instead of after.

1. Look at the pending work after the call: `n * (...)`.
2. Add an accumulator that holds the product computed so far. Start it at the neutral value (1 for products, 0 for sums, "" for strings).
3. Do the work into the accumulator before the call: `acc * n`.
4. At the base case, return the accumulator.

This is the parameterized style from Chapter 8. Tail recursion *is* the accumulator style.

12.3 Converting tail recursion into a loop

Because a tail-recursive method has nothing to do after the call, its frame is useless once the call is made. So instead of making a new frame, we can just **overwrite the parameters and jump back to the top**. That is exactly a loop:

```

TailRecursion.java  loop

static long factorialLoop(int n) {
    long acc = 1;           // parameter 'acc' becomes a local variable
    while (n > 1) {         // base case becomes the loop condition
        acc = acc * n;      // the argument update becomes an assignment
        n = n - 1;
    }
    return acc;
}

```

The mechanical recipe:

Tail-recursive method	Equivalent loop
parameters <code>n</code> , <code>acc</code>	local variables <code>n</code> , <code>acc</code>
initial call <code>f(n, 1)</code>	initialize <code>acc = 1</code>
base case <code>if (n <= 1) return acc;</code>	loop condition <code>while (n > 1)</code> , then <code>return acc;</code>
recursive call <code>f(n - 1, acc * n)</code>	<code>acc = acc * n; n = n - 1;</code> (compute new values from old ones)

✦ PRO TRICK: SIMULTANEOUS ASSIGNMENT

When converting, compute **all** new parameter values from the **old** values. If the call is `f(b, a % b)` (like `gcd`), writing `a = b; b = a % b;` is wrong because `a` has already changed. Use a temporary: `int t = a % b; a = b; b = t;`.

12.4 The Java truth: no tail-call optimization

In many languages (Scheme, Haskell, Scala with `@tailrec`, Kotlin with `tailrec`), the compiler automatically performs the conversion above, so tail recursion runs in constant stack space. This is called **tail-call optimization (TCO)**.

Java does not do this. The Java compiler and the standard HotSpot JVM keep a frame for every call, tail call or not. The program below proves it: it sums the numbers from 1 to 1,000,000, once with a tail-recursive method and once with a loop.

```

TailRecursion.java  deep

static long sumTail(long n, long acc) {
    if (n == 0) return acc;
    return sumTail(n - 1, acc + n);
}

static long sumLoop(long n) {
    long acc = 0;
    while (n > 0) {
        acc += n;
        n--;
    }
    return acc;
}

```

>_ OUTPUT OF TAILRECURSION.JAVA (ACTUAL RUN)

```

factorial(15)           = 1307674368000
factorialTail(15, 1)    = 1307674368000
factorialLoop(15)       = 1307674368000
sumTail(1_000_000, 0)   -> StackOverflowError (Java has no tail-call optimization)
sumLoop(1_000_000)      = 500000500000

```

The tail-recursive version overflows the stack; the loop computes the answer instantly.

! WHAT THIS MEANS FOR YOU AS A JAVA PROGRAMMER

- Tail recursion is still valuable as a **thinking tool**: it is the bridge between recursion and loops.
- But in Java, it gives **no memory benefit**. A tail-recursive method is exactly as deep as a non-tail one.
- If the recursion depth can be large (roughly more than a few thousand) and the method is tail recursive, **convert it to a loop**; the conversion is mechanical.
- If the recursion is not tail recursive and deep, use an explicit stack (Chapter 23) or redesign it to be shallower (divide and conquer).

Σ CHAPTER SUMMARY

- A **tail call** is a call whose result is returned directly with no work afterwards.
- Convert to tail form with an **accumulator** that carries the partial result down.
- Tail recursion converts mechanically into a **loop**: parameters become variables, the call becomes reassignment.
- **Java has no TCO**: deep tail recursion still overflows. Use loops for deep linear recursion.

CHAPTER 13

Multiple Recursion: When One Call Is Not Enough

13

IN THIS CHAPTER

- Understand recursion with two (or more) recursive calls
- See why naive Fibonacci is exponential, with real call counts
- Solve the climbing-stairs problem and see how it relates to Fibonacci
- Master the Tower of Hanoi, the classic example of trusting recursion

So far, every method made at most one recursive call per invocation, so the calls formed a chain. Now each call can make **several** recursive calls, and the calls form a **tree**. This is called **multiple recursion** or **tree recursion**, and it is where recursion becomes truly powerful (and where the leap of faith becomes essential).

13.1 Fibonacci

The Fibonacci sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, ...: each number is the sum of the two before it. The definition translates directly into code:

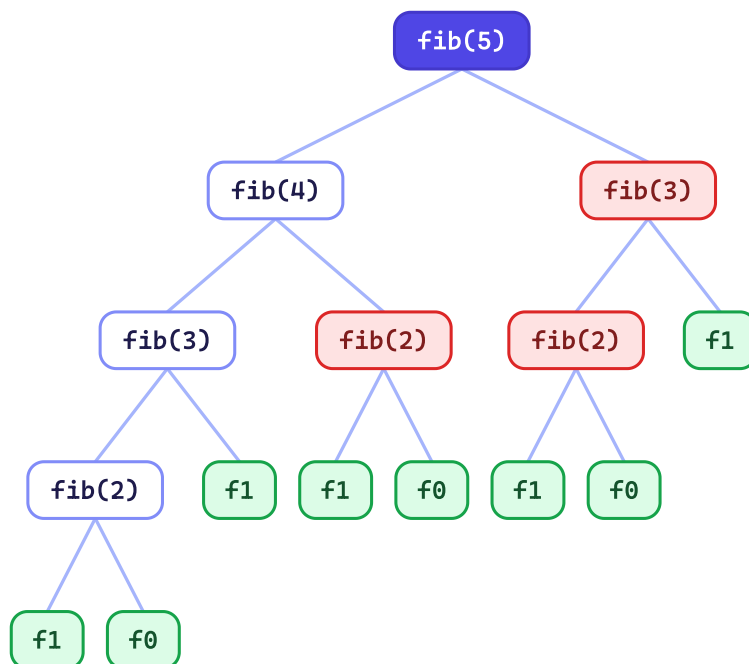
```
MultipleRecursion.java  fib

static long calls = 0;

static long fib(int n) {
    calls++;
    if (n <= 1) return n;           // fib(0) = 0, fib(1) = 1
    return fib(n - 1) + fib(n - 2); // TWO recursive calls
}
```

(The `calls` counter is only there so we can measure how much work is done.)

The recursion tree for `fib(5)` shows what's happening. Every node makes two calls, and the red nodes are calls that repeat work already done elsewhere:



fib(5): red nodes are repeated work -- the same question asked again

■ first call ■ base case ■ repeated work

`fib(3)` is computed twice, `fib(2)` three times, `fib(1)` five times. As n grows, the duplication explodes. The program measures it:

>_ OUTPUT OF MULTIPLERECURSION.JAVA (ACTUAL RUN)

```

fib( 5) =      5   computed with      15 calls
fib(10) =     55   computed with     177 calls
fib(20) =   6,765   computed with   21,891 calls
fib(30) = 832,040   computed with 2,692,537 calls
    
```

```

climbStairs(1) = 1
climbStairs(2) = 2
climbStairs(3) = 3
climbStairs(4) = 5
climbStairs(5) = 8
climbStairs(6) = 13
    
```

```

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves for 3 disks: 7
Moves for 10 disks: 1023
Moves for 64 disks: 18446744073709551615
    
```

Look at the first four lines of that output. Computing `fib(30)` takes **2.7 million calls**. Each time n goes up by 1, the number of calls is multiplied by about 1.618 (the golden ratio). `fib(50)` would take about 40

billion calls. This is **exponential time**, $O(1.618^n)$, and it's the reason the naive version is unusable for large n . In Chapter 22 we'll add one array and bring it down to $O(n)$.

★ KEY IDEA

Multiple recursion is **correct** by the same leap of faith: *"fib(n) is fib(n-1) plus fib(n-2); trust both calls."* But its **cost** is the number of nodes in the recursion tree, which can grow exponentially. Whenever you see a method making two or more calls on inputs that are only slightly smaller ($n-1$, $n-2$), check whether the calls overlap.

13.2 Climbing stairs

Problem: you are climbing a staircase with n steps. Each move, you can climb either 1 step or 2 steps. In how many distinct ways can you reach the top?

Let's use the recipe.

Step 1. `climbStairs(n)` returns the number of ways to climb exactly n steps.

Step 2 — base cases. If $n = 0$, you're already at the top: there is exactly **one** way (do nothing). If $n < 0$, you overshot: **zero** ways.

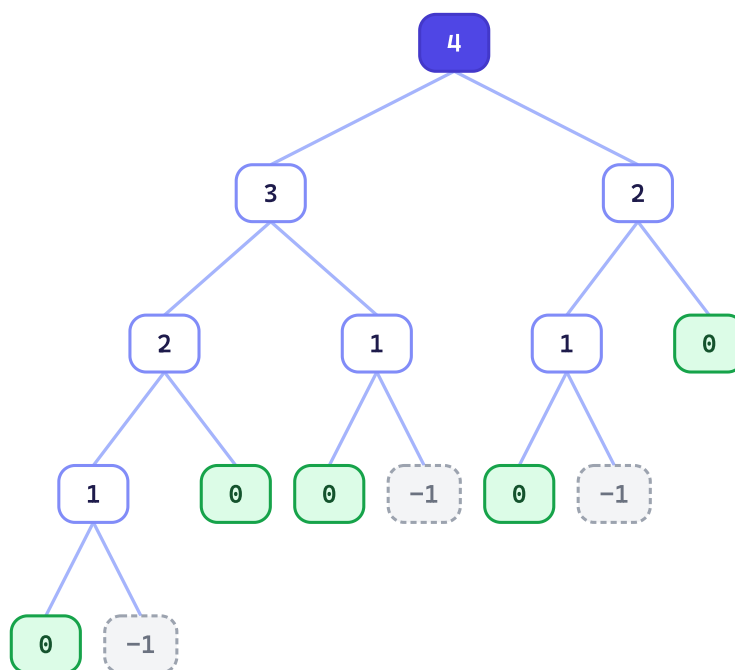
Step 3 and 4 — the key insight. Think about your **first move**. It is either a 1-step or a 2-step. If it's a 1-step, you have $n - 1$ steps left, which you can climb in `climbStairs(n - 1)` ways. If it's a 2-step, you have `climbStairs(n - 2)` ways. Those two groups never overlap (they differ in the first move) and together they cover every possibility. So:

$$\text{climbStairs}(n) = \text{climbStairs}(n - 1) + \text{climbStairs}(n - 2)$$

```

MultipleRecursion.java  stairs

// Number of ways to climb n stairs taking 1 or 2 steps at a time.
static int climbStairs(int n) {
    if (n == 0) return 1;           // at the top: one way (do nothing)
    if (n < 0) return 0;           // overshoot: not a valid way
    return climbStairs(n - 1) + climbStairs(n - 2);
}
```



climbStairs(4): 1-step on the left, 2-step on the right

■ first call ■ base case ▨ invalid / dead end

Every green leaf is one complete way to climb, and there are 5 of them for $n = 4$: (1,1,1,1), (1,1,2), (1,2,1), (2,1,1), (2,2). The gray leaves are overshoots, which contribute 0.

It's the Fibonacci recurrence in disguise! The outputs 1, 2, 3, 5, 8, 13 are Fibonacci numbers. This happens a lot: many counting problems turn out to have the same recurrence, even when the stories are completely different. Chapter 14 is about finding these recurrences.

✦ PRO TRICK: SPLIT BY THE FIRST CHOICE

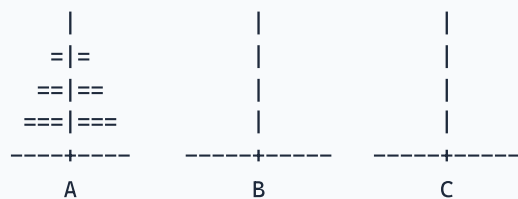
To count "the number of ways" to do something, look at the **first decision**. List every possible first decision; for each one, the rest is a smaller version of the same problem. Add the counts together. This one technique solves a remarkable number of counting problems.

13.3 The Tower of Hanoi

This is *the* classic recursion puzzle, and understanding it is a rite of passage. There are three pegs, A, B and C. Peg A holds n disks of decreasing size, the largest at the bottom. The goal is to move the whole tower to peg C, obeying two rules:

1. You may move only **one disk at a time** (the top disk of some peg).
2. You may **never place a larger disk on top of a smaller one**.

THE STARTING POSITION FOR $N = 3$ (GOAL: MOVE THE TOWER TO C)



Trying to solve this by thinking about individual moves is hard. Thinking recursively, it's almost trivial. Focus only on the **largest disk**:

- The largest disk must eventually move from A to C. For that move to be legal, **all the other $n - 1$ disks must be out of the way**, stacked on B (they can't be on A, since they'd be on top of the largest disk, and they can't be on C, since that's where the largest disk is going).
- So: **first** move the top $n - 1$ disks from A to B (using C as the spare). **Then** move the largest disk from A to C. **Then** move the $n - 1$ disks from B to C (using A as the spare).

"Move $n - 1$ disks from one peg to another" is the same problem, smaller. Leap of faith: trust that it can be done. The base case: moving 0 disks requires no moves.

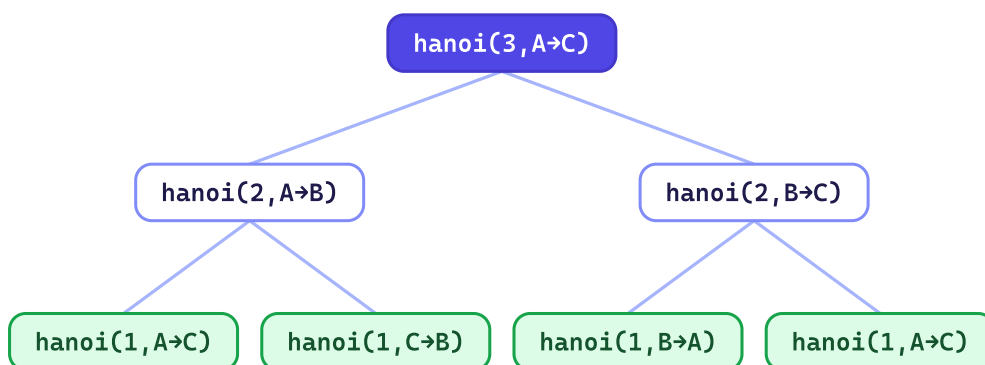
```

MultipleRecursion.java hanoi

static int moves = 0;

static void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;           // no disks: nothing to do
    hanoi(n - 1, from, via, to);  // 1. park the n-1 smaller disks on 'via'
    moves++;                      // 2. move the biggest disk
    System.out.println("Move disk " + n + " from " + from + " to " + to);
    hanoi(n - 1, via, to, from);  // 3. bring the n-1 disks onto it
}
    
```

The complete sequence of moves for 3 disks is in the output above: 7 moves. Let's trace it with the tree of calls:



hanoi(3): each call spawns two calls of size $n-1$

■ first call ■ base case

Reading the output with the tree in mind: moves 1 to 3 are `hanoi(2, A→B)` (park the top two disks on B), move 4 is the big disk 3 going from A to C, and moves 5 to 7 are `hanoi(2, B→C)` (bring the two disks on top of it).

★ **KEY IDEA: THIS IS THE LEAP OF FAITH AT ITS PUREST**

You don't need to know *how* the $n - 1$ disks get moved. You only need to know *that* the smaller problem is solvable (by the same method) and to see how the big disk fits in. If you try to trace Hanoi for 5 disks in your head, you'll get lost. If you trust the recursion, the solution is three lines.

HOW MANY MOVES?

Let $M(n)$ be the number of moves for n disks. From the algorithm: $M(n) = M(n - 1) + 1 + M(n - 1) = 2 \cdot M(n - 1) + 1$, with $M(0) = 0$.

```
MultipleRecursion.java hanoiCount

static long hanoiMoves(int n) {
    if (n == 0) return 0;
    return 2 * hanoiMoves(n - 1) + 1;      // T(n) = 2T(n-1) + 1 => 2^n - 1
}
```

Solving it (pattern 4 from Chapter 11): $M(n) = 2^n - 1$. For 3 disks, 7 moves; for 10 disks, 1023. There's a legend of monks moving 64 golden disks, one per second: they'd need $2^{64} - 1 = 18,446,744,073,709,551,615$ moves (it's in the output), about 585 billion years. And it can be proven that $2^n - 1$ is the **minimum**: no algorithm can do better.

Σ **CHAPTER SUMMARY**

- **Multiple recursion:** each call makes several calls, forming a recursion **tree**.
- It's designed with the same leap of faith: trust **every** recursive call.
- Cost = number of tree nodes, which can be **exponential** (fib: 2.7 million calls for $n = 30$).
- **Split by the first choice** to count ways (climbing stairs).
- **Tower of Hanoi:** move $n-1$ away, move the largest, move $n-1$ back: $2^n - 1$ moves.

CHAPTER 14

Finding the Recurrence: The Real Skill

14

IN THIS CHAPTER

- Learn the skill that separates beginners from experts: turning a problem statement into a recurrence
- Practice on four classic problems: tiling, pairing, restricted strings, and Josephus
- Learn the questions to ask when you're stuck

Writing the code is the easy part of recursion. The hard part, the real skill, is finding the **recurrence**: the relationship between the answer for a problem and the answers for smaller versions of it. Once you have it, the code writes itself. This chapter trains that skill.

14.1 The questions to ask

When facing a new problem, ask these questions in order:

1. **What is the “size” of the problem?** (n, length, number of items, target value...)
2. **What happens at the first (or last) step?** What choices exist for the first element, first move, first person?
3. **After making that choice, what's left?** Is it a smaller version of the same problem? Exactly how much smaller?
4. **How do the choices combine?** If you're counting ways: **add** the counts of different choices. If you're finding the best: take the **max/min**. If you're checking existence: **OR**. If everything must hold: **AND**.
5. **What are the smallest cases?** Work them out by hand.

★ KEY IDEA

Different choices → add (for counting), max/min (for optimizing), OR (for existence). **Sequential steps** → multiply (for counting), add costs (for optimizing).

Let's apply these questions to four problems. All the code is in one program; the output is at the end.

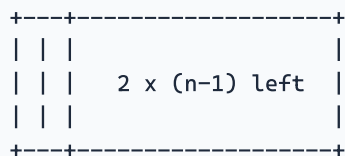
14.2 Tiling a $2 \times n$ board with dominoes

Problem: how many ways can you tile a board 2 units tall and n units wide with 2×1 dominoes (each covers two adjacent squares, placed vertically or horizontally)?

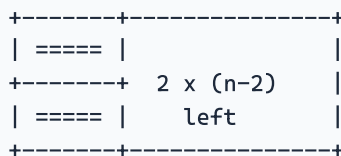
Size: n , the width. **First step:** look at the **leftmost column**. There are only two ways to cover it:

TWO WAYS TO COVER THE LEFT EDGE OF THE BOARD

Option 1: one VERTICAL domino



Option 2: two HORIZONTAL dominoes



- A **vertical** domino covers the first column, leaving a $2 \times (n - 1)$ board.
- Otherwise, the top-left square must be covered by a **horizontal** domino, which forces another horizontal domino beneath it. They cover two columns, leaving a $2 \times (n - 2)$ board.

The two options are different and cover every tiling, so we **add**: $T(n) = T(n - 1) + T(n - 2)$. Smallest cases: $T(0) = 1$ (the empty board has exactly one tiling: place nothing), $T(1) = 1$.

RecurrenceThinking.java tiling

```

// Ways to tile a 2 x n board with 2 x 1 dominoes.
static int tilings(int n) {
    if (n <= 1) return 1;           // 2x0 and 2x1 boards: exactly one way
    return tilings(n - 1)           // first domino placed vertically
        + tilings(n - 2);          // first two dominoes placed horizontally
}
  
```

Fibonacci again! The output shows 1, 2, 3, 5, 8, 13.

14.3 Friends pairing

Problem: n friends are going to a party. Each friend can go alone or pair up with exactly one other friend. How many different ways can they go?

First step: focus on friend number n (the last one). There are two kinds of choices:

- Friend n goes **alone**: the other $n - 1$ friends can arrange themselves in $f(n - 1)$ ways.
- Friend n **pairs** with someone: there are $n - 1$ possible partners. For each partner, the remaining $n - 2$ friends arrange themselves in $f(n - 2)$ ways. That's $(n - 1) \times f(n - 2)$ ways in total (choices in sequence multiply).

So $f(n) = f(n - 1) + (n - 1) \cdot f(n - 2)$, with $f(1) = 1$ and $f(2) = 2$.

```

RecurrenceThinking.java friends

// Each of n friends stays single or pairs up with exactly one other friend.
static long friendsPairing(int n) {
    if (n <= 2) return n; // f(1)=1, f(2)=2
    return friendsPairing(n - 1) // friend n stays single
        + (n - 1) * friendsPairing(n - 2); // friend n pairs with one of n-1 others
}

```

For 4 friends: $f(3) = f(2) + 2 \cdot f(1) = 4$, so $f(4) = f(3) + 3 \cdot f(2) = 4 + 6 = 10$, which matches the output.

14.4 Binary strings without consecutive 1s

Problem: generate all binary strings of length n that never contain “11”.

This time we generate the strings rather than count them, so we build each string character by character, and at each position we decide which characters are allowed. A ‘0’ is always allowed; a ‘1’ is allowed only if the previous character wasn’t ‘1’. We pass the last character down as a parameter so each call knows what’s allowed:

```

RecurrenceThinking.java noConsecutiveOnes

static void binaryNoConsecutiveOnes(int n, String built, char last, List<String> out) {
    if (built.length() == n) {
        out.add(built);
        return;
    }
    binaryNoConsecutiveOnes(n, built + '0', '0', out); // '0' is always allowed
    if (last != '1') {
        binaryNoConsecutiveOnes(n, built + '1', '1', out); // '1' only after a '0'
    }
}

```

For $n = 4$ there are 8 such strings (see the output). If you only wanted to *count* them, the “first step” analysis gives: a valid string either starts with “0” followed by any valid string of length $n - 1$, or starts with “10” followed by any valid string of length $n - 2$. Fibonacci, yet again!

14.5 The Josephus problem

Problem: n people stand in a circle, numbered 0 to $n - 1$. Starting from person 0, count k people around the circle; the k -th person is eliminated. Continue counting from the next person, eliminating every k -th person, until one remains. Which position survives?

This looks like it needs a simulation, but there’s a beautiful recurrence. After the first elimination (the person at position $k - 1$), $n - 1$ people remain, and counting restarts from position k . That’s the Josephus problem for $n - 1$ people, except that everyone’s position is **shifted by k** . So if $J(n - 1)$ is the survivor’s position in the smaller circle, the survivor’s position in the original circle is $(J(n - 1) + k) \bmod n$.

```

RecurrenceThinking.java josephus

// n people in a circle (positions 0..n-1), every k-th person is removed.
// Returns the 0-based position of the survivor.
static int josephus(int n, int k) {
    if (n == 1) return 0;                // only one person: they survive
    return (josephus(n - 1, k) + k) % n; // shift the smaller answer by k
}

```

With 7 people and $k = 3$, the survivor is at position 3 (the 4th person). In the historical version with 41 people and $k = 3$, the survivor is at position 30 (the 31st person).

>_ OUTPUT OF RECURRENCETHINKING.JAVA (ACTUAL RUN)

```

josephus(7, 3) -> survivor at position 3
josephus(41, 3) -> survivor at position 30
tilings(1)=1 tilings(2)=2 tilings(3)=3 tilings(4)=5 tilings(5)=8 tilings(6)=13
length-4 binary strings without "11": [0000, 0001, 0010, 0100, 0101, 1000, 1001, 1010]
friendsPairing(4) = 10

```

✦ PRO TRICK: WORK OUT SMALL CASES BY HAND, THEN LOOK FOR THE PATTERN

When you can't see the recurrence, compute the answers for $n = 0, 1, 2, 3, 4, 5$ by hand (draw pictures!). Then look at the sequence. If it goes 1, 1, 2, 3, 5, 8, it's Fibonacci-like. If each term is double the previous, look for a $2 \cdot f(n - 1)$. The numbers often reveal the recurrence, and then you can go back and understand *why*.

? TRY IT YOURSELF

Find the recurrence (then write the code) for: (a) the number of ways to climb n stairs taking 1, 2 or 3 steps at a time; (b) the number of ways to tile a $3 \times n$ board with 3×1 tiles; (c) the number of binary strings of length n with no two consecutive 0s; (d) the number of ways to make change for amount n with coins {1, 2, 5} (hint: the order of coins shouldn't matter, so pass an index of which coin types are still allowed).

Σ CHAPTER SUMMARY

- The core skill is finding the **recurrence**; code follows directly.
- Ask: size? **first (or last) choice**? what's left? how do choices **combine**? smallest cases?
- Different choices → **add** / max / OR. Sequential steps → **multiply**.
- Many unrelated problems share the same recurrence (Fibonacci shows up everywhere).

CHAPTER 15

Divide and Conquer

15

IN THIS CHAPTER

- Learn the divide-and-conquer strategy: divide, conquer, combine
- Implement recursive binary search, fast exponentiation, merge sort and quick sort
- Understand why halving the problem gives such dramatic speedups

15.1 Divide, conquer, combine

Divide and conquer is a recursion strategy with three steps:

1. **Divide** the problem into smaller subproblems of the same kind (usually halves).
2. **Conquer** each subproblem recursively (the leap of faith). Tiny subproblems are solved directly (base case).
3. **Combine** the sub-answers into the answer for the whole problem.

The difference from the recursion we've done so far: instead of shrinking the problem by **one** element ($n \rightarrow n - 1$), we shrink it by a **fraction** ($n \rightarrow n/2$). That changes the depth from n to $\log n$, which is why divide and conquer is so fast, and why it never overflows the stack in practice ($\log_2$ of a billion is only 30).

15.2 Binary search

Problem: find a target in a **sorted** array.

Divide: look at the middle element. **Conquer:** if the target is smaller, it can only be in the left half; if larger, only in the right half; search that half recursively. **Combine:** nothing to combine; the answer from the half is the answer.

```


DivideAndConquer.java



binarySearch



```

// Searches the sorted range a[low..high] (both inclusive).
static int binarySearch(int[] a, int target, int low, int high) {
 if (low > high) return -1; // empty range: not found
 int mid = low + (high - low) / 2; // avoids int overflow of (low+high)
 if (a[mid] == target) return mid;
 if (target < a[mid]) {
 return binarySearch(a, target, low, mid - 1); // discard right half
 }
 return binarySearch(a, target, mid + 1, high); // discard left half
}

```


```

SEARCHING FOR 23 IN [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

```
low=0, high=9 -> mid=4, a[4]=16   23 > 16 -> search right half [5..9]
low=5, high=9 -> mid=7, a[7]=56   23 < 56 -> search left half [5..6]
low=5, high=6 -> mid=5, a[5]=23   found! return 5
```

Each call halves the range, so a search among a million elements takes at most 20 calls. **Time: $O(\log n)$.**
Space: $O(\log n)$ for the stack.

✗ COMMON MISTAKE: $(LOW + HIGH) / 2$ CAN OVERFLOW

If `low` and `high` are both large (above about 1.07 billion), `low + high` overflows `int` and becomes negative. Writing `low + (high - low) / 2` computes the same midpoint without overflow. This bug sat unnoticed in Java's own `Arrays.binarySearch` for about nine years.

15.3 Fast exponentiation

In Chapter 7, `power(x, n)` made n calls. We can do much better with one observation:

```
x^n = (x^(n/2))^2      when n is even
x^n = (x^(n/2))^2 * x  when n is odd   (n/2 rounds down)
```

So to compute x^{13} we only need x^6 (and then square it and multiply by x). To get x^6 we need x^3 . To get x^3 we need x^1 . To get x^1 we need x^0 . That's 5 calls instead of 13.

● ● ● DivideAndConquer.java fastPower

```
static long fastPower(long x, int n) {
    if (n == 0) return 1;
    long half = fastPower(x, n / 2);           // solve ONE half-size problem...
    if (n % 2 == 0) return half * half;        // ...and reuse it: x^n = (x^(n/2))^2
    return half * half * x;                    // odd n needs one extra x
}
```

✗ COMMON MISTAKE: CALLING THE HALF TWICE

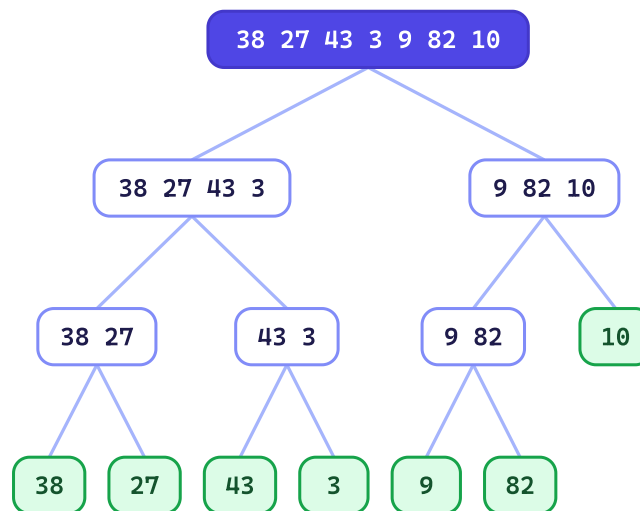
Writing `return fastPower(x, n / 2) * fastPower(x, n / 2);` is correct, but it makes **two** recursive calls of size $n/2$, and $T(n) = 2T(n/2) + c$ is $O(n)$: all the speedup is lost! Compute the half **once**, store it in a variable, and reuse it. Solving a subproblem once and reusing it is the seed of memoization (Chapter 22).

Time: $O(\log n)$. `fastPower(2, 62)` takes 7 calls. This algorithm, often called **binary exponentiation**, is used everywhere from cryptography (RSA) to competitive programming.

15.4 Merge sort

Problem: sort an array.

- **Divide:** split the array into two halves.
- **Conquer:** sort each half recursively. (Leap of faith: after these two calls, both halves are sorted.) An array of size 0 or 1 is already sorted: that's the base case.
- **Combine:** **merge** the two sorted halves into one sorted array.



Merge sort: split until single elements (base case), then merge back up

■ first call ■ base case

THE MERGE (COMBINE) PHASE, BOTTOM-UP

```
[38] [27] -> [27, 38]           [43] [3] -> [3, 43]           [9] [82] -> [9, 82]   [10]
[27, 38] + [3, 43] -> [3, 27, 38, 43]       [9, 82] + [10] -> [9, 10, 82]
[3, 27, 38, 43] + [9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82]
```

Merging two sorted lists is easy: repeatedly compare the front elements of both lists and take the smaller one.

```

DivideAndConquer.java mergeSort

static void mergeSort(int[] a, int low, int high) {
    if (low >= high) return;           // 0 or 1 element: already sorted
    int mid = low + (high - low) / 2;
    mergeSort(a, low, mid);           // trust: left half becomes sorted
    mergeSort(a, mid + 1, high);      // trust: right half becomes sorted
    merge(a, low, mid, high);         // combine two sorted halves
}

static void merge(int[] a, int low, int mid, int high) {
    int[] temp = new int[high - low + 1];
    int i = low, j = mid + 1, k = 0;
    while (i <= mid && j <= high) {
        temp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    }
    while (i <= mid) temp[k++] = a[i++]; // leftovers from the left half
    while (j <= high) temp[k++] = a[j++]; // leftovers from the right half
    System.arraycopy(temp, 0, a, low, temp.length);
}

```

Notice how little of this code is “sorting logic”. The `mergeSort` method itself contains no comparisons at all! It just splits, trusts, and merges. All the real work is in `merge`, which is a simple loop.

Time: $O(n \log n)$ (pattern 5 from Chapter 11: $\log n$ levels, n work per level). **Space:** $O(n)$ for the temporary arrays, plus $O(\log n)$ stack. Merge sort is **stable** (equal elements keep their order) and always $O(n \log n)$, which is why Java uses a merge-sort variant (TimSort) to sort objects.

15.5 Quick sort

Quick sort flips the work around: merge sort does the hard work **after** the recursive calls (merging), while quick sort does it **before** (partitioning).

- **Divide:** pick a **pivot** element and **partition** the array so that everything smaller than the pivot is on its left and everything larger is on its right. The pivot is now in its final sorted position.
- **Conquer:** sort the left part and the right part recursively.
- **Combine:** nothing! Once both parts are sorted, the whole array is sorted.

```

● ● ● DivideAndConquer.java quickSort

static void quickSort(int[] a, int low, int high) {
    if (low >= high) return;
    int p = partition(a, low, high);           // pivot lands at its final index p
    quickSort(a, low, p - 1);                 // sort everything smaller
    quickSort(a, p + 1, high);                // sort everything larger
}

static int partition(int[] a, int low, int high) {
    int pivot = a[high];                      // Lomuto scheme: last element is pivot
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            swap(a, i, j);
        }
    }
    swap(a, i + 1, high);
    return i + 1;
}

static void swap(int[] a, int i, int j) {
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
}

```

The partition here uses the **Lomuto scheme**: the last element is the pivot; `i` marks the end of the “smaller than pivot” region; every element smaller than the pivot is swapped into that region; finally, the pivot is swapped into place just after it.

Time: $O(n \log n)$ on average, but $O(n^2)$ in the worst case (for example, an already-sorted array with the last element as pivot, since every partition is maximally lopsided). Choosing a random pivot makes the worst case extremely unlikely. Quick sort sorts **in place** (no temporary arrays), which is why Java uses a quick-sort variant (dual-pivot quicksort) for arrays of primitives.

15.6 Maximum by divide and conquer

Even simple problems can be solved this way, and it’s a good exercise in the pattern:

```

● ● ● DivideAndConquer.java maxDC

static int maxDivideConquer(int[] a, int low, int high) {
    if (low == high) return a[low];
    int mid = low + (high - low) / 2;
    return Math.max(maxDivideConquer(a, low, mid), maxDivideConquer(a, mid + 1, high));
}

```

$T(n) = 2T(n/2) + O(1) = \mathbf{O(n)}$, the same as a loop. But its recursion depth is only $\log n$, so unlike the “one element at a time” version from Chapter 9, it works on huge arrays without overflowing the stack.

>_ OUTPUT OF DIVIDEANDCONQUER.JAVA (ACTUAL RUN)

```
binarySearch(23) -> index 5
binarySearch(40) -> index -1
fastPower(3, 13) = 1594323
fastPower(2, 62) = 4611686018427387904
mergeSort -> [3, 9, 10, 27, 38, 43, 82]
quickSort -> [10, 30, 40, 50, 70, 80, 90]
maxDivideConquer -> 42
```

Algorithm	Divide	Combine	Time	Depth
Binary search	pick one half	none	$O(\log n)$	$O(\log n)$
Fast power	halve exponent	square ($\times x$ if odd)	$O(\log n)$	$O(\log n)$
Merge sort	split in the middle	merge ($O(n)$)	$O(n \log n)$	$O(\log n)$
Quick sort	partition around pivot ($O(n)$)	none	$O(n \log n)$ avg	$O(\log n)$ avg
Max (D&C)	split in the middle	<code>Math.max</code>	$O(n)$	$O(\log n)$

Σ CHAPTER SUMMARY

- **Divide and conquer**: divide into fractions, conquer recursively, combine.
- Halving the problem makes the depth $\mathbf{O(\log n)}$: fast, and safe for the stack.
- Compute a shared subproblem **once** (fast power), never twice.
- **Merge sort** works after the calls (merge); **quick sort** works before them (partition).

P A R T

IV

Recursion on Data Structures

Linked lists and binary trees: structures that are defined recursively, and therefore processed recursively.

16 Linked Lists: A Node Plus a Smaller List

17 Binary Trees: Recursion's Natural Home



CHAPTER 16

Linked Lists: A Node Plus a Smaller List

16

IN THIS CHAPTER

- See the recursive definition of a linked list
- Build, print (both directions), measure and search lists recursively
- Understand recursive list reversal completely, pointer by pointer
- Merge two sorted lists with a short recursive method

16.1 The recursive definition

A singly linked list is a chain of nodes, each holding a value and a reference to the next node:

```

LinkedListRecursion.java  node

static class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
    }
}
```

Here is the recursive way to see it:

§ RECURSIVE DEFINITION OF A LINKED LIST

A linked list is either **empty** (`null`), or a **node** whose `next` field is a **linked list**.

The “smaller problem” for a list is simply **the rest of the list**: `head.next`. The base case is the **empty list**: `head == null`. With that definition, list recursion is very natural.

A LIST IS A NODE FOLLOWED BY A SMALLER LIST

```

head
|
v
[1] -> [2] -> [3] -> [4] -> [5] -> null
^         \-----/
|           the rest: head.next
the first node      (a smaller list)

```

16.2 Building a list recursively
   **LinkedListRecursion.java** build

```

static Node build(int[] values, int i) {
    if (i == values.length) return null;    // no values left: empty list
    Node head = new Node(values[i]);
    head.next = build(values, i + 1);        // the rest of the list is a smaller list
    return head;
}

```

"A list made from `values[i..]` is a new node holding `values[i]`, whose `next` is the list made from `values[i+1..]`." The base case (no values left) is the empty list.

16.3 Printing forwards and backwards
   **LinkedListRecursion.java** print

```

static void printForward(Node head) {
    if (head == null) {
        System.out.println("null");
        return;
    }
    System.out.print(head.data + " -> ");
    printForward(head.next);
}

static void printBackward(Node head) {
    if (head == null) return;
    printBackward(head.next);           // print the rest first...
    System.out.print(head.data + " ");  // ...then me
}

```

`printForward` prints before recursing; `printBackward` prints after. This is the "before or after the call" lesson from Chapter 6, applied to a list. Printing a singly linked list backwards is awkward with a loop (you can only move forward!), but with recursion it's four lines: the call stack remembers the nodes for you.

16.4 Length and search

LinkedListRecursion.java length · contains

```
static int length(Node head) {
    if (head == null) return 0;
    return 1 + length(head.next);
}

static boolean contains(Node head, int key) {
    if (head == null) return false;
    if (head.data == key) return true;
    return contains(head.next, key);
}
```

length: the empty list has length 0; any other list has length 1 + the length of the rest. **contains**: false for the empty list; true if this node matches; otherwise, search the rest. These are word-for-word the same as the array versions from Chapter 9, with **head.next** instead of **i + 1**.

16.5 Reversing a linked list recursively

This is one of the most frequently asked interview questions, and a wonderful exercise in the leap of faith. Here's the code, and then we'll understand every line.

LinkedListRecursion.java reverse

```
static Node reverse(Node head) {
    if (head == null || head.next == null) return head; // 0 or 1 node
    Node newHead = reverse(head.next); // trust: the rest is now reversed
    head.next.next = head; // my old successor now points back to me
    head.next = null; // I become the new tail
    return newHead; // the head of the reversed list never changes
}
```

The leap of faith. Let the list be **1 → 2 → 3 → 4 → 5**. **head** is node 1. Trust that **reverse(head.next)** reverses the rest of the list (**2 → 3 → 4 → 5**) and returns its new head, node 5. After that call, the situation is:

AFTER THE RECURSIVE CALL RETURNS (TRUST IT!)

```
newHead
|
v
[5] -> [4] -> [3] -> [2] -> null
                        ^
                        |
head -> [1] -----+ (1 still points to 2, its old next)
```

Crucially, node 1's **next** still points to node 2, which is now the **last** node of the reversed part. So we only need to fix two pointers:

1. `head.next.next = head;` — `head.next` is node 2; set node 2's `next` to node 1. Now `5 → 4 → 3 → 2 → 1`.
2. `head.next = null;` — node 1 is the new tail, so it must point to null (otherwise we'd have a cycle `1 → 2 → 1`).

Then return `newHead` (node 5), which was the head of the reversed list all along: it's passed up unchanged through every level.

FIXING THE TWO POINTERS

```
before:  [5] -> [4] -> [3] -> [2] -> null      [1] -> [2]
step 1:  [5] -> [4] -> [3] -> [2] -> [1]      (and 1 -> 2 still: a cycle!)
step 2:  [5] -> [4] -> [3] -> [2] -> [1] -> null  (cycle broken)
```

Base case. An empty list or a single node is already its own reverse, so return it as is. The single-node case is also where `newHead` comes from: the last node of the original list.

★ KEY IDEA

Don't trace list reversal call by call; that's confusing. Instead, draw the picture **right after the recursive call returns**, assume the rest is reversed correctly, and figure out the few pointer changes needed to attach the current node. This "picture after the call" technique works for every tricky linked-list recursion.

16.6 Merging two sorted lists

Problem: given two sorted lists, merge them into one sorted list (reusing the existing nodes).

Leap of faith: compare the two heads. The smaller one must be the first node of the result. Its `next` should be the merge of **everything that remains**: the rest of its own list, and the whole other list. That's the same problem, smaller. Base cases: if either list is empty, the result is the other list.

```
LinkedListRecursion.java merge

static Node mergeSorted(Node a, Node b) {
    if (a == null) return b;
    if (b == null) return a;
    if (a.data <= b.data) {
        a.next = mergeSorted(a.next, b);    // a wins; merge the rest behind it
        return a;
    }
    b.next = mergeSorted(a, b.next);        // b wins; merge the rest behind it
    return b;
}
```

>_ OUTPUT OF LINKEDLISTRECURSION.JAVA (ACTUAL RUN)

```
forward : 1 -> 2 -> 3 -> 4 -> 5 -> null
backward: 5 4 3 2 1
length   : 5
contains(4): true, contains(9): false
reversed: 5 -> 4 -> 3 -> 2 -> 1 -> null
merged   : 1 -> 2 -> 3 -> 4 -> 7 -> 8 -> 10 -> 12 -> 15 -> null
```

! DEPTH WARNING FOR LISTS

Every list method in this chapter has recursion depth equal to the length of the list. For lists with more than a few thousand nodes, Java will overflow the stack. In production code, linked-list operations are usually written with loops. The recursive versions are still worth mastering: they train the thinking needed for trees, where the depth is usually much smaller.

? TRY IT YOURSELF

Write recursive methods to: (a) return the sum of the list; (b) delete all nodes with a given value (return the new head); (c) return the k-th node from the end (hint: return a count on the way back); (d) check whether two lists are identical; (e) insert a value into a sorted list.

Σ CHAPTER SUMMARY

- A list is **empty**, or a **node plus a smaller list**. Base case: `head == null`. Smaller problem: `head.next`.
- Work before the call processes front-to-back; work after the call processes back-to-front.
- **Reverse**: trust the rest is reversed, then `head.next.next = head; head.next = null;`.
- **Merge**: the smaller head comes first; its `next` is the merge of what remains.

CHAPTER 17

Binary Trees: Recursion's Natural Home

17

IN THIS CHAPTER

- Understand the recursive definition of a binary tree
- Learn the “ask your children, then combine” template that solves most tree problems
- Implement the three traversals and understand their order
- Compute counts, sums, heights, maxima and leaves
- Pass information **down** (path sums, BST bounds) and **up** (diameter)
- Solve lowest common ancestor and mirror

If there's one data structure where recursion is not just convenient but essential, it's the tree. Almost every tree algorithm is recursive, and most of them are just a few lines long once you know the pattern.

17.1 The recursive definition

TreeRecursion.java node

```
static class Node {
    int val;
    Node left, right;

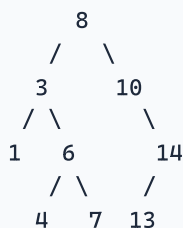
    Node(int val) {
        this.val = val;
    }
}
```

§ RECURSIVE DEFINITION OF A BINARY TREE

A binary tree is either **empty** (`null`), or a **node** with a value, a **left subtree** and a **right subtree**, each of which is a binary tree.

Every example in this chapter uses this binary search tree:

THE SAMPLE TREE



17.2 The “ask your children” template

Here is the template that solves the majority of tree problems:

●●● The tree recursion template

```

static Result solve(Node node) {
    if (node == null) {
        return answerForEmptyTree; // base case: the empty tree
    }
    Result left = solve(node.left); // trust: correct answer for the left subtree
    Result right = solve(node.right); // trust: correct answer for the right subtree
    return combine(node.val, left, right); // build my answer from my value + theirs
}
  
```

≈ REAL-LIFE ANALOGY: THE ORG CHART

This is the manager from Chapter 4. Each node is a manager with (at most) two direct reports. To answer a question about their whole team, a manager asks each report the same question about *their* team, then combines the two answers with information about themselves. The people at the bottom (the `null` children) answer immediately.

The only things that change from problem to problem are the **base case value** and the **combine** step. Keep that in mind as you read the rest of this chapter; you’ll see the same shape again and again.

17.3 The three traversals

A **traversal** visits every node once. For binary trees, there are three classic depth-first orders, and they differ **only** in when the node itself is processed relative to its two subtrees:

```

TreeRecursion.java traversals

static void preorder(Node node, StringBuilder out) {
    if (node == null) return;
    out.append(node.val).append(' '); // ROOT first
    preorder(node.left, out);         // then left subtree
    preorder(node.right, out);        // then right subtree
}

static void inorder(Node node, StringBuilder out) {
    if (node == null) return;
    inorder(node.left, out);          // left subtree first
    out.append(node.val).append(' '); // then ROOT
    inorder(node.right, out);         // then right subtree
}

static void postorder(Node node, StringBuilder out) {
    if (node == null) return;
    postorder(node.left, out);        // left subtree
    postorder(node.right, out);       // right subtree
    out.append(node.val).append(' '); // ROOT last
}

```

Traversal	Order	Sample tree result	Typical use
Preorder	root, left, right	8 3 1 6 4 7 10 14 13	copying a tree, printing structure, serializing
Inorder	left, root, right	1 3 4 6 7 8 10 13 14	BST: gives values in sorted order
Postorder	left, right, root	1 4 7 6 3 13 14 10 8	deleting a tree, computing sizes/heights (children before parent)

✦ PRO TRICK: REMEMBER BY THE ROOT'S POSITION

Pre-order: root comes **before** the children. **In**-order: root comes **in** between. **Post**-order: root comes **after** (post = after). It's the "before or after the call" lesson again, now with two calls instead of one.

Notice that inorder traversal of a binary search tree gives the values in sorted order (1 3 4 6 7 8 10 13 14). That's because in a BST, everything in the left subtree is smaller than the root and everything in the right subtree is larger, and inorder prints left subtree, root, right subtree.

17.4 Counting and summing

```

TreeRecursion.java  countSum

static int count(Node node) {
    if (node == null) return 0;
    return 1 + count(node.left) + count(node.right);
}

static int sum(Node node) {
    if (node == null) return 0;
    return node.val + sum(node.left) + sum(node.right);
}

```

count: an empty tree has 0 nodes; otherwise it's 1 (me) + the count of the left subtree + the count of the right subtree. **sum**: the same with values. Both are exactly the template, with **+** as the combine step.

17.5 Height

```

TreeRecursion.java  height

// Height = number of nodes on the longest root-to-leaf path (empty tree = 0).
static int height(Node node) {
    if (node == null) return 0;
    return 1 + Math.max(height(node.left), height(node.right));
}

```

"The height of a tree is 1 (for the root) plus the height of its taller subtree." The empty tree has height 0. For the sample tree, the longest root-to-leaf path is $8 \rightarrow 3 \rightarrow 6 \rightarrow 4$ (or $8 \rightarrow 10 \rightarrow 14 \rightarrow 13$), which has 4 nodes, so the height is 4.

I TWO DEFINITIONS OF HEIGHT

Some books measure height in **nodes** (empty tree = 0, single node = 1), as we do here. Others measure it in **edges** (single node = 0, and then the empty tree is -1). Both are fine; just be consistent, and make sure your base case matches your definition.

17.6 Maximum value and leaf count

```

TreeRecursion.java  max · leaves

static int max(Node node) {
    if (node == null) return Integer.MIN_VALUE; // identity for max
    return Math.max(node.val, Math.max(max(node.left), max(node.right)));
}

static int countLeaves(Node node) {
    if (node == null) return 0;
    if (node.left == null && node.right == null) return 1; // I am a leaf
    return countLeaves(node.left) + countLeaves(node.right);
}

```

For `max`, the empty tree returns `Integer.MIN_VALUE`, the neutral element for maximum (anything is bigger than it). `countLeaves` has an **extra base case**: a node with no children is a leaf, and it counts as 1.

17.7 Passing information down: path sum

So far, information flowed **up** (children returned answers to parents). Some problems need information to flow **down**, from the root towards the leaves. For that, we use **parameters**.

Problem: is there a root-to-leaf path whose values add up to a target?

```

TreeRecursion.java  pathSum

static boolean hasPathSum(Node node, int remaining) {
    if (node == null) return false;
    remaining -= node.val; // "spend" my value
    if (node.left == null && node.right == null) return remaining == 0;
    return hasPathSum(node.left, remaining) || hasPathSum(node.right, remaining);
}

```

Each node **subtracts** its value from the remaining target and passes the reduced target down. At a leaf, we check whether exactly zero remains. For target 21, the path $8 \rightarrow 3 \rightarrow 6 \rightarrow 4$ works ($8 + 3 + 6 + 4 = 21$).

17.8 Collecting all root-to-leaf paths

To collect the actual paths, we carry the **current path** down as a list, and use the *choose, explore, un-choose* pattern, a preview of backtracking (Part V):

```

TreeRecursion.java  paths

static void allPaths(Node node, List<Integer> path, List<String> out) {
    if (node == null) return;
    path.add(node.val);                                // choose
    if (node.left == null && node.right == null) {
        out.add(path.toString());
    } else {
        allPaths(node.left, path, out);                // explore
        allPaths(node.right, path, out);
    }
    path.remove(path.size() - 1);                      // un-choose
}

```

The line `path.remove(path.size() - 1)` is essential. All calls share **one** list; when a call is finished with its node, it must remove it, so that the list is correct for the next branch. Without it, the path would keep growing and contain nodes from other branches.

17.9 Validating a binary search tree

Problem: check whether a tree is a valid binary search tree: for **every** node, all values in its left subtree are smaller, and all values in its right subtree are larger.

✗ COMMON MISTAKE: ONLY COMPARING WITH THE DIRECT CHILDREN

The tempting solution checks `node.left.val < node.val < node.right.val` at every node. That's **wrong**. Consider a root 8 with left child 3, and 3's right child is 9. Locally, every parent-child pair looks fine ($3 < 8$, $9 > 3$), but 9 is in the **left** subtree of 8, so it should be smaller than 8. The rule is about entire subtrees, not just children.

The correct solution passes down the **allowed range** for each node. The root can be anything. When we go **left**, every value must be less than the current node, so the upper bound becomes `node.val`. When we go **right**, the lower bound becomes `node.val`.

```

TreeRecursion.java  validBST

static boolean isBST(Node node, long low, long high) {
    if (node == null) return true;
    if (node.val <= low || node.val >= high) return false; // outside allowed window
    return isBST(node.left, low, node.val)                // left must be < me
        && isBST(node.right, node.val, high);             // right must be > me
}

```

We use `long` bounds so that a tree containing `Integer.MIN_VALUE` or `Integer.MAX_VALUE` is handled correctly.

17.10 Returning one thing, tracking another: diameter

Problem: find the **diameter** of the tree: the number of edges on the longest path between **any** two nodes (the path doesn't have to pass through the root).

The key observation: every path has a single **highest node** where it "bends". The longest path that bends at node X goes down into X's left subtree as deep as possible and down into its right subtree as deep as possible, so its length is (depth of left) + (depth of right).

So at every node we need two things: the **depth** of the node (to give to its parent) and the **best path bending here** (to compare against the best overall). The method returns the depth and updates the best in a "box":

```
TreeRecursion.java  diameter

// Diameter = number of EDGES on the longest path between any two nodes.
static int diameter(Node root) {
    int[] best = {0}; // a 1-element array acts as a mutable box
    depth(root, best);
    return best[0];
}

static int depth(Node node, int[] best) {
    if (node == null) return 0;
    int left = depth(node.left, best);
    int right = depth(node.right, best);
    best[0] = Math.max(best[0], left + right); // longest path that BENDS at this node
    return 1 + Math.max(left, right); // but only one side can go to my parent
}
```

The one-element array `best` acts as a mutable container shared by all calls, which avoids a static field (Chapter 8). For the sample tree, the longest path is $4 \rightarrow 6 \rightarrow 3 \rightarrow 8 \rightarrow 10 \rightarrow 14 \rightarrow 13$: 6 edges. It bends at the root: left depth 3, right depth 3.

✦ PRO TRICK: RETURN WHAT THE PARENT NEEDS, RECORD WHAT THE PROBLEM NEEDS

When a tree problem asks for something that doesn't naturally combine upward (like the diameter, or the maximum path sum), have the recursion **return** the quantity the parent needs (like the depth) and **record** the actual answer on the side. This pattern solves a whole family of "hard" tree problems.

17.11 Lowest common ancestor

Problem: given two values p and q in the tree, find their **lowest common ancestor** (LCA): the deepest node that has both p and q in its subtree (a node counts as its own ancestor).

TreeRecursion.java lca

```

static Node lca(Node node, int p, int q) {
    if (node == null || node.val == p || node.val == q) return node;
    Node left = lca(node.left, p, q);
    Node right = lca(node.right, p, q);
    if (left != null && right != null) return node;    // p and q on different sides
    return left != null ? left : right;              // pass up whatever was found
}

```

The contract: `lca(node, p, q)` returns the LCA if both are in this subtree, the node `p` or `q` itself if only one of them is in this subtree, and `null` if neither is. Then:

- If this node is `p` or `q`, return it.
- Ask both children. If **both** return something, `p` and `q` are on different sides, so **this node** is the lowest point where their paths meet.
- If only one side returns something, pass it up.

For 4 and 7, both are under 6, so the LCA is 6. For 4 and 14, they're on different sides of 8, so the LCA is 8.

17.12 Mirroring a tree

TreeRecursion.java mirror

```

static Node mirror(Node node) {
    if (node == null) return null;
    Node newLeft = mirror(node.right);
    Node newRight = mirror(node.left);
    node.left = newLeft;
    node.right = newRight;
    return node;
}

```

Mirror both subtrees, then swap them. After mirroring, inorder traversal gives the values in **descending** order, and the tree is no longer a valid BST, which the output confirms:

>_ OUTPUT OF TREERECURSION.JAVA (ACTUAL RUN)

```

preorder   : 8 3 1 6 4 7 10 14 13
inorder    : 1 3 4 6 7 8 10 13 14
postorder  : 1 4 7 6 3 13 14 10 8
count      : 9
sum        : 66
height     : 4
max        : 14
leaves     : 4
pathSum 21 : true    (8+3+6+4)
pathSum 20 : false
all paths  : [[8, 3, 1], [8, 3, 6, 4], [8, 3, 6, 7], [8, 10, 14, 13]]
isBST      : true
diameter   : 6 edges
lca(4, 7)  : 6
lca(4, 14) : 8
lca(1, 7)  : 3
mirrored inorder : 14 13 10 8 7 6 4 3 1
isBST after mirror: false
    
```

17.13 Information flow: top-down vs. bottom-up

Direction	Mechanism	Examples
Bottom-up (children → parent)	return values	count, sum, height, max, leaves, diameter, LCA
Top-down (parent → children)	parameters	path sum (remaining target), BST bounds, depth of a node, current path
Both	parameters down + returns up	many advanced problems (e.g., max path sum with constraints)

★ KEY IDEA

Ask: “Does a node need to know something about its ancestors?” If yes, pass it **down** as a parameter. “Does a node need to know something about its descendants?” If yes, get it **up** from return values.

? TRY IT YOURSELF

Write recursive methods to: (a) check whether two trees are identical; (b) check whether a tree is symmetric (a mirror of itself); (c) count the nodes at depth k ; (d) find the minimum depth (shortest root-to-leaf path); (e) check whether the tree is height-balanced (the heights of the two subtrees of every node differ by at most 1) in $O(n)$ (hint: return -1 for “unbalanced”).

Σ CHAPTER SUMMARY

- A tree is **empty** or a **node with two subtrees**. Base case: `node == null`.
- Template: **ask both children, combine with the node's own value**.
- Pre/in/post-order differ only in when the node is processed. Inorder of a BST is sorted.
- Pass **down** via parameters (path sums, BST bounds); pass **up** via return values (height, count).
- **Return what the parent needs, record what the problem needs** (diameter).

P A R T

V

Generating Possibilities: Backtracking

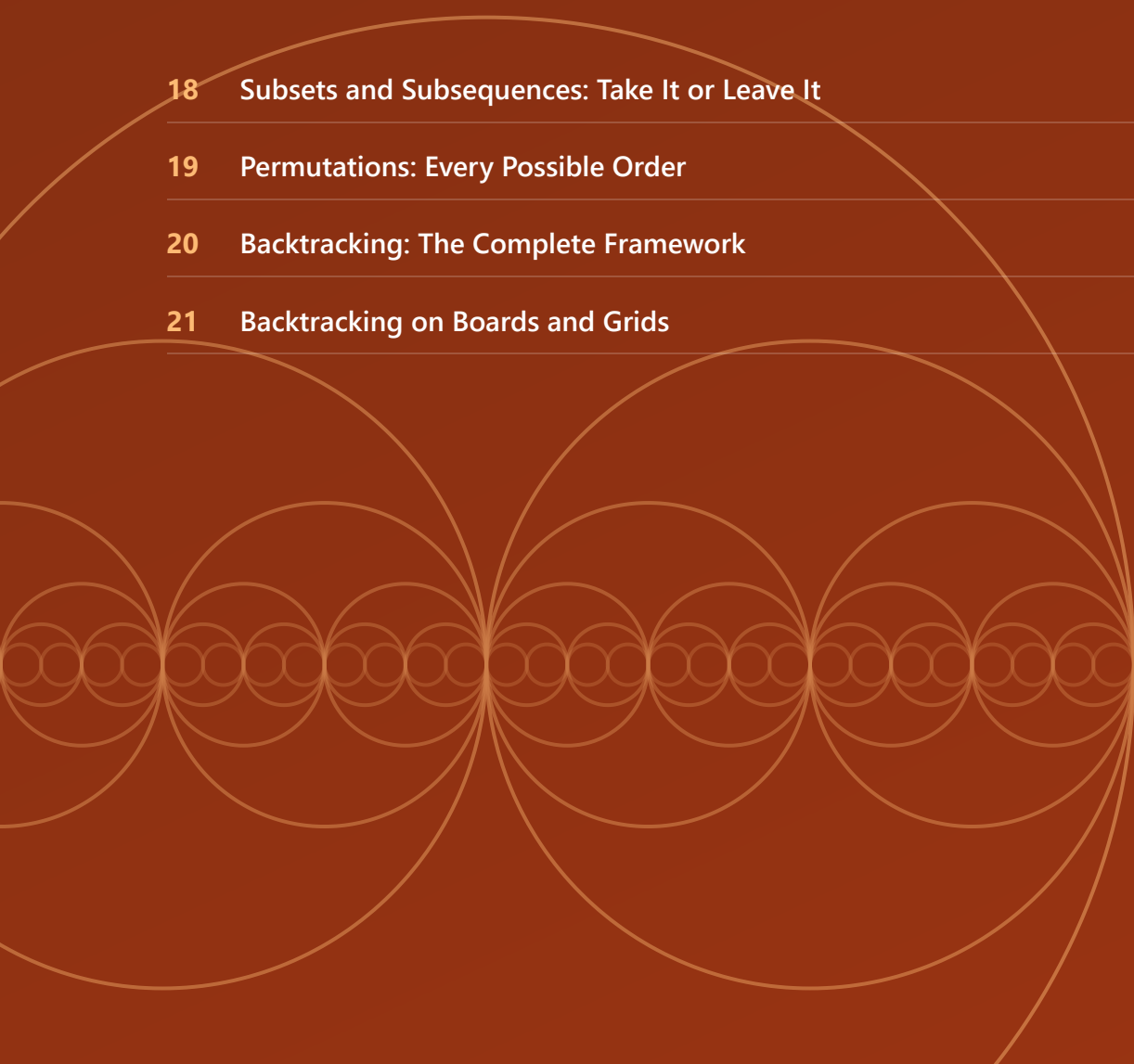
Subsets, permutations, combinations, puzzles and mazes: exploring every option with choose, explore, un-choose.

18 Subsets and Subsequences: Take It or Leave It

19 Permutations: Every Possible Order

20 Backtracking: The Complete Framework

21 Backtracking on Boards and Grids



CHAPTER 18

Subsets and Subsequences: Take It or Leave It

18

IN THIS CHAPTER

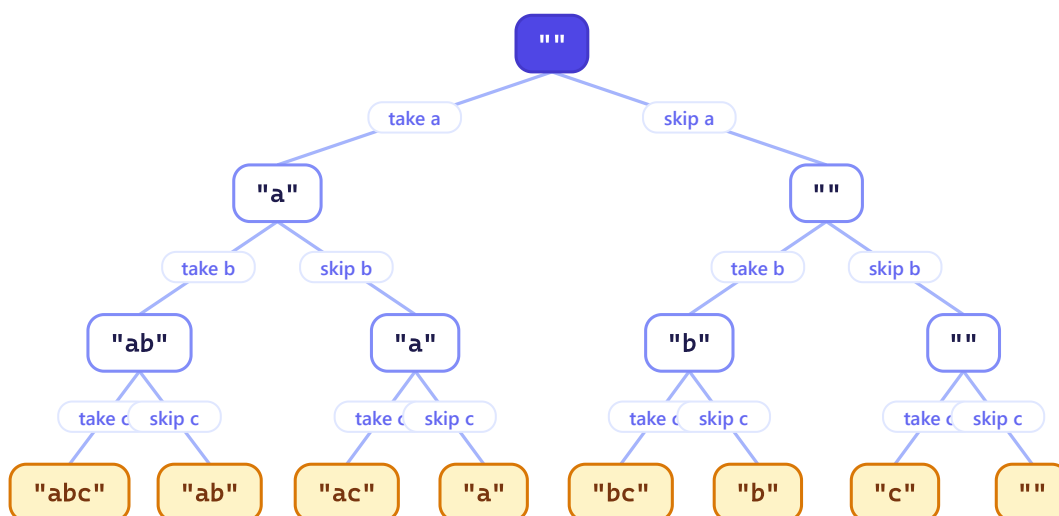
- Learn the most important pattern for generating possibilities: the take/skip decision
- Understand decision trees and how recursion walks them
- Generate all subsequences of a string and all subsets of an array
- Learn the copy rule and the undo rule for shared lists
- Count and check subsets with a given sum
- Handle duplicates correctly

18.1 The decision tree

A **subset** of a collection is any selection of its elements, including none and all. {1, 2, 3} has 8 subsets: {}, {1}, {2}, {3}, {1,2}, {1,3}, {2,3}, {1,2,3}. For strings, the equivalent is a **subsequence**: characters taken in their original order, possibly skipping some ("ac" is a subsequence of "abc"; "ca" is not).

How can we generate them all systematically? Think of it as making **one decision per element**: for each element, either **take** it or **skip** it. Every subset corresponds to exactly one sequence of decisions. For n elements there are $2 \times 2 \times \dots \times 2 = 2^n$ subsets.

These decisions form a **decision tree**:



Take/skip decision tree for "abc" -- every leaf is one subsequence

■ first call ■ final result

Each level of the tree is the decision about one character. Going left means “take”, going right means “skip”. Each leaf (in amber) is one complete set of decisions, one subsequence. Recursion walks this tree depth-first, and that’s all we need.

★ KEY IDEA

For each element, make a choice, then let recursion handle the remaining elements. The recursion tree is the decision tree. The base case is “no elements left to decide”, and it’s where you record a finished result.

18.2 All subsequences of a string

```
SubsetRecursion.java subsequences

static void subsequences(String s, int i, String built) {
    if (i == s.length()) {                // decided for every character
        System.out.print "\"" + built + "\" ";
        return;
    }
    subsequences(s, i + 1, built + s.charAt(i)); // choice 1: TAKE s[i]
    subsequences(s, i + 1, built);             // choice 2: SKIP s[i]
}
```

The parameters say it all: `i` is the index of the character we’re deciding about; `built` is what we’ve taken so far. Two recursive calls: one where `built` includes `s[i]`, one where it doesn’t. When `i` reaches the end, every decision has been made, and `built` is one subsequence.

Notice that we used `built + s.charAt(i)`, which creates a **new** String for the “take” branch. The “skip” branch still has the original `built`, untouched. Because Strings are immutable, each branch has its own copy, and we never need to undo anything. That’s convenient but costs a copy per call. With a mutable list, we must be more careful.

18.3 Subsets into a list: the copy rule and the undo rule

When the current subset is a mutable `List` shared by all calls, two rules become essential:

```

SubsetRecursion.java subsetsList

static void subsets(int[] nums, int i, List<Integer> current, List<List<Integer>> all) {
    if (i == nums.length) {
        all.add(new ArrayList<>(current)); // store a COPY, not the live list
        return;
    }
    current.add(nums[i]); // take nums[i]
    subsets(nums, i + 1, current, all);
    current.remove(current.size() - 1); // undo the take
    subsets(nums, i + 1, current, all); // skip nums[i]
}

```

§ THE TWO RULES OF SHARED MUTABLE STATE

- 1. The copy rule:** when recording a result, store a **copy** (`new ArrayList<>(current)`), never `current` itself. The same list object keeps changing after you record it; if you store the object itself, every stored “result” ends up being the same (usually empty) list.
- 2. The undo rule:** every change made before a recursive call must be **undone** after it (`current.remove(current.size() - 1)`), so that the next branch starts from the exact same state as this one did.

Together, “change → recurse → undo” is called **choose, explore, un-choose**, and it is the heart of backtracking. You’ll see it in every remaining chapter of this part.

✗ COMMON MISTAKE: ALL.ADD(CURRENT)

If you write `all.add(current)` instead of `all.add(new ArrayList<>(current))`, the output is `[[], [], [], [], [], [], [], []]`: eight references to the **same** list, which is empty by the time you print it, because every add was undone. This is the single most common backtracking bug.

18.4 Subset sums

Instead of collecting the subsets themselves, we can carry a running **sum** down the tree and record it at the leaves:

```

SubsetRecursion.java subsetSums

static void subsetSums(int[] a, int i, int sum, List<Integer> out) {
    if (i == a.length) {
        out.add(sum);
        return;
    }
    subsetSums(a, i + 1, sum + a[i], out); // include a[i]
    subsetSums(a, i + 1, sum, out);       // exclude a[i]
}

```

Since `sum` is an `int` (passed by value), each call has its own copy and nothing needs to be undone. For {3, 1, 2}, the 8 sums are 0, 1, 1, 2, 3, 3, 4, 5, 6 (sorted).

18.5 Counting subsets with a target sum

Now a question that returns a number instead of printing: *how many* subsets add up to exactly k ? This is the take/skip idea in functional style: the count is (the number of ways that take `a[i]`) plus (the number of ways that skip it).

```
SubsetRecursion.java countK

// How many subsets of a[i..] have sum exactly k? (all values positive)
static int countSubsetsWithSum(int[] a, int i, int k) {
    if (k == 0) return 1; // chosen set already hits k
    if (k < 0 || i == a.length) return 0; // overshoot, or ran out of items
    return countSubsetsWithSum(a, i + 1, k - a[i]) // take a[i]
        + countSubsetsWithSum(a, i + 1, k); // skip a[i]
}
```

The base cases matter here. If `k == 0`, the elements taken so far already sum to the target, which counts as one way (take nothing more). If `k < 0` (overshoot, possible because all values are positive) or we've run out of elements, there are zero ways. For {1, 2, 3, 4, 5} and $k = 5$, there are 3 ways: {5}, {1, 4}, {2, 3}.

A yes/no version uses `||` and gets **short-circuiting** for free: as soon as one branch finds a solution, Java doesn't even evaluate the other branch.

```
SubsetRecursion.java anyK

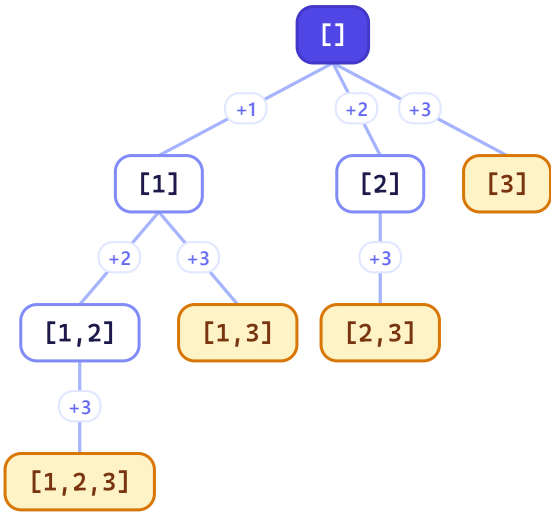
static boolean existsSubsetWithSum(int[] a, int i, int k) {
    if (k == 0) return true;
    if (k < 0 || i == a.length) return false;
    return existsSubsetWithSum(a, i + 1, k - a[i]) // short-circuits on first success
        || existsSubsetWithSum(a, i + 1, k);
}
```

18.6 Another shape: the “for-loop” style

There's a second way to generate subsets, and it's important because it's the shape used for combinations and many backtracking problems. Instead of asking “take or skip element i ?”, each call asks “which element comes next in the subset?” and loops over all candidates:

SubsetRecursion.java loopStyle

```
// "For-loop" style: at each level, pick which element comes NEXT.
static void subsetsLoop(int[] nums, int start,
                        List<Integer> current, List<List<Integer>> all) {
    all.add(new ArrayList<>(current));           // every node is a subset
    for (int j = start; j < nums.length; j++) {
        current.add(nums[j]);
        subsetsLoop(nums, j + 1, current, all);
        current.remove(current.size() - 1);
    }
}
```



For-loop style: every node (not just leaves) is a subset

■ first call ■ final result

In this tree, **every node** (not just the leaves) is a subset, so we record at the start of every call. Passing `j + 1` as the next start ensures elements are only added in increasing index order, so each subset is generated exactly once ([1,2] but never [2,1]).

Style	Each call decides	Results recorded	Branching
Take/skip	include element <code>i</code> or not	at leaves only	always 2
For-loop	which element comes next	at every node	varies (n – start)

Both generate the same 2^n subsets, just in a different order (compare the two lines in the output below).

18.7 Handling duplicates

If the input contains duplicates, like {1, 2, 2}, both styles above would generate [1, 2] twice (once with each 2). To avoid this:

- 1. **Sort** the input so equal values are adjacent.

2. In the for-loop style, at each level, **skip a value if it's equal to the previous value at the same level** (`j > start && nums[j] == nums[j - 1]`).

```

SubsetRecursion.java  subsetsDup

// Input may contain duplicates; output must not contain duplicate subsets.
static void subsetsWithDup(int[] nums, int start,
                          List<Integer> current, List<List<Integer>> all) {
    all.add(new ArrayList<>(current));
    for (int j = start; j < nums.length; j++) {
        if (j > start && nums[j] == nums[j - 1]) continue; // duplicate at this level
        current.add(nums[j]);
        subsetsWithDup(nums, j + 1, current, all);
        current.remove(current.size() - 1);
    }
}

```

Why does this work? At a given level, choosing the first 2 or the second 2 as “the next element” leads to identical subtrees. By only allowing the first of equal values at each level, we explore that subtree once. Note the condition is `j > start`, not `j > 0`: the second 2 **is** allowed when it's the first candidate at its level (that's how we get [1, 2, 2] and [2, 2]).

>_ OUTPUT OF SUBSETRECURSION.JAVA (ACTUAL RUN)

```

subsequences("abc"): "abc" "ab" "ac" "a" "bc" "b" "c" ""
subsets take/skip : [[1, 2, 3], [1, 2], [1, 3], [1], [2, 3], [2], [3], []]
subset sums {3,1,2}: [0, 1, 2, 3, 3, 4, 5, 6]
subsets of {1..5} with sum 5: 3
exists subset of {1..5} with sum 14? true
exists subset of {1..5} with sum 16? false
subsets loop style : [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
subsetsWithDup {1,2,2}: [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]

```

Σ CHAPTER SUMMARY

- Subsets come from **one take/skip decision per element**: 2^n results.
- The recursion tree is the **decision tree**; record results at the base case.
- With shared mutable lists: **copy when recording, undo after each recursive call**.
- Counting: take-count + skip-count. Existence: take || skip (short-circuits).
- For-loop style: “which element comes next?”; every node is a subset.
- Duplicates: **sort**, then skip equal values at the same level.

CHAPTER 19

Permutations: Every Possible Order

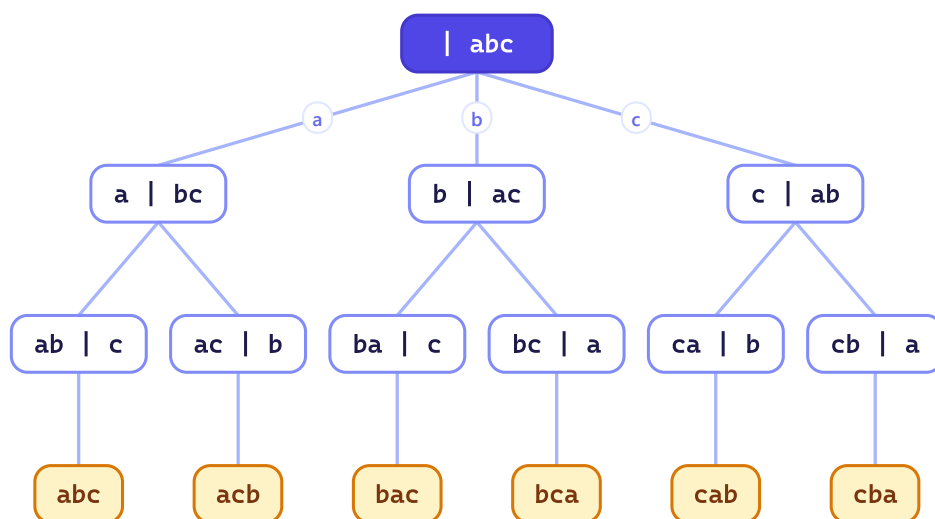
IN THIS CHAPTER

- Understand how permutations differ from subsets
- Generate permutations in three ways: pick-and-rest, `used[]` array, and swapping
- Generate unique permutations when the input has duplicates
- Compare subsets, combinations and permutations

19.1 From subsets to permutations

A **permutation** is an **arrangement** of all the elements in some order. "abc" has 6 permutations: abc, acb, bac, bca, cab, cba. For n distinct elements there are $n \times (n - 1) \times \dots \times 1 = n!$ permutations: n choices for the first position, $n - 1$ for the second, and so on.

The decision is no longer "take or skip"; it's "which of the remaining elements goes in the next position?"



Permutation tree for "abc": built | remaining

■ first call ■ final result

The root has 3 children (3 choices for the first letter), each of those has 2 children (2 remaining choices), and each of those has 1. Total: $3 \times 2 \times 1 = 6$ leaves.

19.2 Style 1: pick one, permute the rest

```

PermutationRecursion.java pickRest

// 'built' = letters already placed, 'rest' = letters still available.
static void permute(String built, String rest) {
    if (rest.isEmpty()) {
        System.out.print(built + " ");
        return;
    }
    for (int i = 0; i < rest.length(); i++) {
        char pick = rest.charAt(i);
        String remaining = rest.substring(0, i) + rest.substring(i + 1);
        permute(built + pick, remaining);    // place 'pick' next, permute the others
    }
}

```

`built` holds the letters placed so far and `rest` holds the letters still available. For each available letter, place it next and recursively permute what's left. When nothing is left, `built` is a complete permutation. Strings are immutable, so there's nothing to undo, but every call creates new strings.

19.3 Style 2: the used[] array

The standard, efficient approach uses one shared list for the current arrangement and a boolean array marking which elements are already placed:

```

PermutationRecursion.java usedArray

static void permuteUsed(int[] nums, boolean[] used,
                        List<Integer> current, List<List<Integer>> all) {
    if (current.size() == nums.length) {
        all.add(new ArrayList<>(current));
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;    // already placed in this permutation
        used[i] = true;           // choose
        current.add(nums[i]);
        permuteUsed(nums, used, current, all);    // explore
        current.remove(current.size() - 1);    // un-choose
        used[i] = false;
    }
}

```

This is choose, explore, un-choose in its textbook form:

- **Choose:** mark `used[i]` and append `nums[i]`.
- **Explore:** recurse to fill the next position.

- **Un-choose:** remove `nums[i]` and unmark `used[i]`, so the next iteration of the loop starts from a clean state.

19.4 Style 3: swapping in place

The most memory-efficient approach uses no extra list at all. It treats the array as having a **fixed prefix** (positions `0..index-1`, already decided) and a **free suffix**. For position `index`, it tries each element of the suffix by **swapping** it into place:

```

PermutationRecursion.java swap

// Positions [0, index) are fixed; try every candidate for position 'index'.
static void permuteSwap(int[] a, int index, List<String> out) {
    if (index == a.length) {
        out.add(Arrays.toString(a));
        return;
    }
    for (int i = index; i < a.length; i++) {
        swap(a, index, i);           // bring a[i] into position 'index'
        permuteSwap(a, index + 1, out);
        swap(a, index, i);           // put it back (backtrack)
    }
}

static void swap(int[] a, int i, int j) {
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
}

```

The second `swap` restores the array, which is the un-choose step. Note that this style produces permutations in a slightly different order (compare the output lines): after the swaps, “3 2 1” comes before “3 1 2”.

19.5 Unique permutations

For “aabc”, the methods above would generate each arrangement twice (swapping the two ‘a’s gives the “same” string). The fix is the same idea as for subsets with duplicates: **sort**, then skip a duplicate unless its previous twin is already used.

```

●●● PermutationRecursion.java  unique

static void permuteUnique(char[] chars, boolean[] used,
                          StringBuilder current, List<String> out) {
    if (current.length() == chars.length) {
        out.add(current.toString());
        return;
    }
    for (int i = 0; i < chars.length; i++) {
        if (used[i]) continue;
        // Among equal letters, only use them left-to-right; skip if the
        // previous equal letter is not used yet (it would create a duplicate).
        if (i > 0 && chars[i] == chars[i - 1] && !used[i - 1]) continue;
        used[i] = true;
        current.append(chars[i]);
        permuteUnique(chars, used, current, out);
        current.deleteCharAt(current.length() - 1);
        used[i] = false;
    }
}

```

The rule `chars[i] == chars[i - 1] && !used[i - 1] → skip` forces equal letters to be used **left to right**: the second 'a' can only be placed after the first 'a' has been placed. That way, the two 'a's can never swap roles, and each distinct arrangement appears once. "aabc" has $4! / 2! = 12$ unique permutations.

>_ OUTPUT OF PERMUTATIONRECURSION.JAVA (ACTUAL RUN)

```

permute("abc"): abc acb bac bca cab cba
used[] style   : [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
swap style     : [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 2, 1], [3, 1, 2]]
unique("aabc"): 12 -> [aabc, aacb, abac, abca, acab, acba, baac, baca, bcaa, caab, caba, cbaa]

```

19.6 Subsets vs. combinations vs. permutations

	Order matters?	Uses all elements?	Count (n distinct)	Recursion shape
Subsets	No	No (any number)	2^n	take/skip, or loop from <code>start</code>
Combinations (choose k)	No	Exactly k	$n! / (k!(n-k)!)$	loop from <code>start</code> , stop at size k
Permutations	Yes	Yes	$n!$	loop over all unused elements

★ KEY IDEA: START INDEX VS. USED ARRAY

If **order doesn't matter** (subsets, combinations), loop from a `start` index and pass `j + 1` : this prevents generating the same set in different orders. If **order matters** (permutations), loop over **all** elements and use a `used[]` array to prevent reusing an element.

Σ CHAPTER SUMMARY

- Permutations: **which remaining element goes next?** $n!$ results.
- Three styles: pick-and-rest (strings), `used[]` + list (standard), swap (in place).
- Every style is **choose** → **explore** → **un-choose**.
- Duplicates: sort, then only use equal elements left to right.

CHAPTER 20

Backtracking: The Complete Framework

20

IN THIS CHAPTER

- Understand backtracking as a systematic search with undo
- Learn the backtracking template and the four questions that define any problem
- Solve five classic problems with the same template
- Learn pruning: the technique that makes backtracking fast

20.1 What backtracking is

Imagine exploring a maze. At each junction, you pick a path and follow it. If you hit a dead end, you walk **back** to the last junction and try a different path. If all paths from that junction fail, you back up further. Eventually you either find the exit or prove there is none.

That's **backtracking**: build a solution **one choice at a time**; after each choice, recurse to make the next; if you reach a complete solution, record it; if a choice leads nowhere, **undo it** and try the next one. You've already been doing this: subsets and permutations are backtracking. This chapter makes the framework explicit so you can apply it to *any* problem.

20.2 The template

The backtracking template

```
void backtrack(State state, List<Solution> results) {
    if (isComplete(state)) {                // GOAL: a full solution has been built
        results.add(copyOf(state));         // record a COPY
        return;
    }
    for (Choice choice : choicesFor(state)) { // CHOICES available at this point
        if (!isValid(state, choice)) continue; // CONSTRAINTS: prune invalid choices early
        apply(state, choice);                 // choose
        backtrack(state, results);            // explore
        undo(state, choice);                  // un-choose
    }
}
```

To solve any backtracking problem, answer **four questions**:

§ THE FOUR QUESTIONS OF BACKTRACKING

1. **State:** What does a partial solution look like? (a list of chosen numbers, a partly filled board, a string being built...)
2. **Choices:** At each step, what are the options? (the next number, the next character, which column to put a queen in...)
3. **Constraints:** Which choices are invalid right now and can be skipped immediately?
4. **Goal:** When is the partial solution complete?

Let's answer those questions for five classic problems.

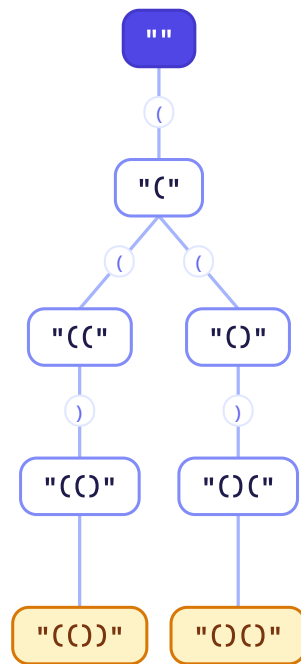
20.3 Generate balanced parentheses

Problem: generate all strings of n pairs of parentheses that are balanced (every `(` is closed, and no `)` comes before its `(`).

- **State:** the string built so far, plus the number of `(` and `)` used.
- **Choices:** add `(` or add `)`.
- **Constraints:** you may add `(` only if fewer than n have been used; you may add `)` only if it would close an open one (`close < open`).
- **Goal:** the string has length $2n$.

```
Backtracking.java  parentheses

static void generateParentheses(int n, int open, int close,
                                StringBuilder cur, List<String> out) {
    if (cur.length() == 2 * n) {           // used all n pairs
        out.add(cur.toString());
        return;
    }
    if (open < n) {                         // may open while pairs remain
        cur.append('(');
        generateParentheses(n, open + 1, close, cur, out);
        cur.deleteCharAt(cur.length() - 1);
    }
    if (close < open) {                    // may close only what is open
        cur.append(')');
        generateParentheses(n, open, close + 1, cur, out);
        cur.deleteCharAt(cur.length() - 1);
    }
}
```



generateParentheses(2): branches that break a rule are never created

■ first call ■ final result

Because the constraints are checked **before** recursing, invalid strings like `)()` are never even started. The tree for $n = 2$ has only 2 leaves, and every leaf is valid. There's no need to generate all $2^4 = 16$ strings of parentheses and filter them afterwards.

20.4 Combinations: choose k from n

Problem: list all ways to choose k numbers from $1..n$ (order doesn't matter).

- **State:** the list of chosen numbers.
- **Choices:** the next number, which must be larger than the last one chosen (so each combination is generated once, in increasing order).
- **Constraints:** there must be enough numbers left to reach k .
- **Goal:** k numbers chosen.

```

Backtracking.java combinations

// All ways to choose k numbers from 1..n (order does not matter).
static void combinations(int n, int k, int start,
                        List<Integer> cur, List<List<Integer>> out) {
    if (cur.size() == k) {
        out.add(new ArrayList<>(cur));
        return;
    }
    for (int x = start; x <= n; x++) {
        if (n - x + 1 < k - cur.size()) break;    // PRUNE: not enough numbers left
        cur.add(x);
        combinations(n, k, x + 1, cur, out);    // next pick must be larger than x
        cur.remove(cur.size() - 1);
    }
}

```

The line `if (n - x + 1 < k - cur.size()) break;` is **pruning**: if there aren't enough numbers from x to n to fill the remaining slots, no choice from here can succeed, so stop the loop entirely.

20.5 Combination sum

Problem: given distinct positive candidates and a target, find all combinations that sum to the target. Each candidate may be used **any number of times**. For candidates $\{2, 3, 6, 7\}$ and target 7, the answers are $[2, 2, 3]$ and $[7]$.

- **State:** the current list and the remaining target.
- **Choices:** any candidate at index $\geq$ `start` (to avoid $[2, 3, 2]$ and $[3, 2, 2]$ as duplicates of $[2, 2, 3]$).
- **Constraints:** the candidate must not exceed the remaining target.
- **Goal:** remaining target is exactly 0.

```

Backtracking.java combinationSum

// Each candidate may be reused any number of times. candidates must be sorted.
static void combinationSum(int[] c, int start, int remaining,
                          List<Integer> cur, List<List<Integer>> out) {
    if (remaining == 0) {
        out.add(new ArrayList<>(cur));
        return;
    }
    for (int i = start; i < c.length; i++) {
        if (c[i] > remaining) break;    // PRUNE: sorted, later ones too big
        cur.add(c[i]);
        combinationSum(c, i, remaining - c[i], cur, out);    // i, not i+1: reuse
        cur.remove(cur.size() - 1);
    }
}

```

Two details make this work:

- The recursive call passes `i`, not `i + 1`, as the next start. That allows the **same** candidate to be chosen again (reuse), while still never going back to earlier candidates.
- Because the candidates are **sorted**, once `c[i] > remaining`, every later candidate is also too big, so we `break` instead of `continue`. That's a stronger pruning.

✦ PRO TRICK: I VS. I + 1

In loop-style backtracking, the next start index controls reuse. Pass `i + 1` when each element can be used **at most once** (combinations, subsets). Pass `i` when elements can be **reused** (combination sum, coin change). Pass **0 with a** `used[]` array when **order matters** (permutations).

20.6 Palindrome partitioning

Problem: split a string into pieces such that every piece is a palindrome. List all such splits. For "aab": [a, a, b] and [aa, b].

- **State:** the list of pieces so far, and the index where the unsplit remainder starts.
- **Choices:** the end of the next piece: any position from `start` to the end of the string.
- **Constraints:** the piece `s[start..end]` must be a palindrome.
- **Goal:** `start` has reached the end of the string.

```
Backtracking.java  palindromePartition

static void partition(String s, int start, List<String> cur, List<List<String>> out) {
    if (start == s.length()) {
        out.add(new ArrayList<>(cur));
        return;
    }
    for (int end = start; end < s.length(); end++) {
        if (!isPalindrome(s, start, end)) continue;           // only cut off palindromes
        cur.add(s.substring(start, end + 1));
        partition(s, end + 1, cur, out);                       // partition what is left
        cur.remove(cur.size() - 1);
    }
}

static boolean isPalindrome(String s, int l, int r) {
    while (l < r) {
        if (s.charAt(l++) != s.charAt(r--)) return false;
    }
    return true;
}
```

This is the loop style again, but the "choice" is a **substring length** rather than an element. The same template handles it without any changes in structure.

20.7 Phone keypad letter combinations

Problem: on an old phone keypad, 2 = "abc", 3 = "def", and so on. Given a string of digits, list every letter string it could represent.

- **State:** the letters chosen so far, and which digit we're on.
- **Choices:** each letter on the current digit's key.
- **Constraints:** none.
- **Goal:** one letter chosen for every digit.

```
Backtracking.java keypad

static final String[] KEYS = {
    "", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"
};

static void letterCombinations(String digits, int i,
                               StringBuilder cur, List<String> out) {
    if (i == digits.length()) {
        out.add(cur.toString());
        return;
    }
    for (char ch : KEYS[digits.charAt(i) - '0'].toCharArray()) {
        cur.append(ch);
        letterCombinations(digits, i + 1, cur, out);
        cur.deleteCharAt(cur.length() - 1);
    }
}
```

>_ OUTPUT OF BACKTRACKING.JAVA (ACTUAL RUN)

```
parentheses(3)      : [((())) , (())() , (())() , ()()() , ()()()]
combinations(4, 2)  : [[1, 2], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]]
combinationSum(7)   : [[2, 2, 3], [7]]
partition("aab")    : [[a, a, b], [aa, b]]
keypad("23")        : [ad, ae, af, bd, be, bf, cd, ce, cf]
```

20.8 Pruning: the secret to fast backtracking

Backtracking explores a tree of possibilities, and that tree is often exponentially large. **Pruning** means cutting off branches **as early as possible**, before exploring them, when you can tell they can't lead to a valid solution.

Problem	Pruning rule	Effect
Parentheses	only add <code>)</code> if <code>close < open</code>	invalid strings never built
Combinations	stop if too few numbers remain	skips hopeless branches
Combination sum	<code>break</code> when candidate > remaining (sorted)	skips the rest of the loop
N-Queens (next chapter)	skip attacked squares	about 15,700 placements instead of $8^8 \approx 16.7$ million boards
Sudoku (next chapter)	only try digits allowed in the row, column, box	makes the search feasible at all

★ KEY IDEA

Check constraints as early as possible: before recursing, not at the base case. Generating everything and filtering at the end is correct but can be exponentially slower. Good pruning is often the difference between a program that finishes in milliseconds and one that never finishes.

Σ CHAPTER SUMMARY

- Backtracking = build a solution step by step, **choose** → **explore** → **un-choose**.
- Define any problem with four questions: **state**, **choices**, **constraints**, **goal**.
- Next-start index: `i + 1` (no reuse), `i` (reuse), `used[]` (order matters).
- **Prune early:** check constraints before recursing, and `break` when sorted input makes the rest hopeless.

CHAPTER 21

Backtracking on Boards and Grids

21

IN THIS CHAPTER

- Solve the N-Queens problem, the classic backtracking puzzle
- Find all paths through a maze with visited-marking and unmarking
- Search for words in a letter grid
- Count islands with flood fill (recursive depth-first search)
- Solve a real Sudoku with about 25 lines of recursion

Grids and boards are where backtracking really shines. The state is a 2D array, the choices are cells or values, and the constraints come from the rules of the puzzle.

21.1 N-Queens

Problem: place n queens on an $n \times n$ chessboard so that no two queens attack each other. A queen attacks along its row, its column and both diagonals.

- **State:** the board, with queens placed in rows `0..row-1`.
- **Choices:** a column for the queen in the current `row`. (Exactly one queen per row is forced, since two queens in a row would attack each other. That observation alone shrinks the search enormously.)
- **Constraints:** the square must not be attacked by any queen already placed.
- **Goal:** `row == n` (a queen in every row).

```

GridBacktracking.java  nQueens

static int solutions = 0;

static void placeQueens(char[][] board, int row, boolean print) {
    int n = board.length;
    if (row == n) {                                     // a queen sits in every row
        solutions++;
        if (print) printBoard(board);
        return;
    }
    for (int col = 0; col < n; col++) {
        if (!isSafe(board, row, col)) continue; // PRUNE attacked squares immediately
        board[row][col] = 'Q';                 // choose
        placeQueens(board, row + 1, print);     // explore the next row
        board[row][col] = '.';                 // un-choose
    }
}

// Only rows above 'row' contain queens, so only look upward.
static boolean isSafe(char[][] board, int row, int col) {
    int n = board.length;
    for (int r = row - 1; r >= 0; r--) {
        int d = row - r;                          // row distance
        if (board[r][col] == 'Q') return false;    // same column
        if (col - d >= 0 && board[r][col - d] == 'Q') return false; // up-left
        if (col + d < n && board[r][col + d] == 'Q') return false; // up-right
    }
    return true;
}

```

Because queens are placed row by row from the top, the safety check only needs to look **upward**: the same column, the up-left diagonal and the up-right diagonal. Rows below are still empty.

>_ OUTPUT OF GRIDBACKTRACKING.JAVA (ACTUAL RUN)

```

All solutions for 4 queens:
. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q
. Q . .

n=1:1  n=2:0  n=3:0  n=4:2  n=5:10  n=6:4  n=7:40  n=8:92

rat in maze paths: [DDRDRR, DRDDRR]
word "ABCCED": true
word "SEE"    : true
word "ABCB"   : false
islands: 3

```

The output shows both solutions for 4 queens, then the number of solutions for $n = 1$ to 8. The famous 8-queens puzzle has **92** solutions. Notice $n = 2$ and $n = 3$ have **none**: backtracking explores every possibility and proves it by exhaustion.

✦ **PRO TRICK: O(1) SAFETY CHECKS**

Instead of scanning the board, keep three boolean arrays: `cols[c]`, `diag1[row + col]` and `diag2[row - col + n - 1]`. Every square on the same "" diagonal has the same `row - col`, and every square on the same "/" diagonal has the same `row + col`. Then checking and updating safety is O(1). Remember to unset the three flags when un-choosing!

21.2 Rat in a maze

Problem: a rat starts at the top-left cell of an $n \times n$ maze and wants to reach the bottom-right cell. Cells with 1 are open, cells with 0 are walls. The rat can move down, left, right or up. List every path (as a string of moves).

THE MAZE (1 = OPEN, 0 = WALL)

```

col: 0  1  2  3
row 0:  1  0  0  0
row 1:  1  1  0  1
row 2:  1  1  0  0
row 3:  0  1  1  1

```

- **State:** the current cell, the path string, and which cells are on the current path.
- **Choices:** the four directions.

- **Constraints:** stay inside the grid, don't enter walls, don't revisit a cell already on the current path (or you'd walk in circles forever).
- **Goal:** the bottom-right cell.

```

GridBacktracking.java  ratMaze

static final int[] DR = {1, 0, 0, -1};           // Down, Left, Right, Up
static final int[] DC = {0, -1, 1, 0};
static final char[] DIR = {'D', 'L', 'R', 'U'};

static void ratInMaze(int[][] maze, int r, int c, boolean[][] visited,
                     StringBuilder path, List<String> out) {
    int n = maze.length;
    if (r == n - 1 && c == n - 1) {              // reached the exit
        out.add(path.toString());
        return;
    }
    visited[r][c] = true;                        // mark: don't walk in circles
    for (int d = 0; d < 4; d++) {
        int nr = r + DR[d], nc = c + DC[d];
        if (nr < 0 || nc < 0 || nr >= n || nc >= n) continue; // off the grid
        if (maze[nr][nc] == 0 || visited[nr][nc]) continue; // wall, or on path
        path.append(DIR[d]);
        ratInMaze(maze, nr, nc, visited, path, out);
        path.deleteCharAt(path.length() - 1);
    }
    visited[r][c] = false;                      // unmark: other paths may use it
}

```

★ KEY IDEA: MARK, EXPLORE, UNMARK

`visited[r][c] = true` before exploring and `visited[r][c] = false` afterwards is choose/un-choose applied to cells. Marking prevents cycles **within the current path**. Unmarking allows **other** paths to use the cell later. If you forget to unmark, you'll miss valid paths that share cells with earlier ones.

The two paths found are DDRDRR and DRDDRR (see output).

21.3 Word search

Problem: given a grid of letters, check whether a word can be formed by a path of adjacent cells (up, down, left, right), using each cell at most once.

```

GridBacktracking.java wordSearch

static boolean exist(char[][] board, String word) {
    for (int r = 0; r < board.length; r++) {
        for (int c = 0; c < board[0].length; c++) {
            if (match(board, word, 0, r, c)) return true;
        }
    }
    return false;
}

static boolean match(char[][] b, String word, int i, int r, int c) {
    if (i == word.length()) return true; // matched every letter
    if (r < 0 || c < 0 || r >= b.length || c >= b[0].length) return false;
    if (b[r][c] != word.charAt(i)) return false;
    char saved = b[r][c];
    b[r][c] = '#'; // mark as used
    boolean found = match(b, word, i + 1, r + 1, c) || match(b, word, i + 1, r - 1, c)
        || match(b, word, i + 1, r, c + 1) || match(b, word, i + 1, r, c - 1);
    b[r][c] = saved; // restore
    return found;
}

```

`match(b, word, i, r, c)` means: “can `word[i..]` be formed starting at cell (r, c) ?” If the cell matches `word[i]`, temporarily overwrite it with `#` (so the path can’t reuse it), try all four neighbors for `word[i+1..]`, then **restore** the letter. Overwriting the cell itself is a neat trick that avoids a separate `visited` array.

21.4 Flood fill: counting islands

Problem: in a grid of ‘1’ (land) and ‘0’ (water), count the islands (groups of land connected horizontally or vertically).

This is recursive **depth-first search** (DFS). Scan the grid; every time you find land that hasn’t been visited, that’s a new island, so count it and then **“sink”** its whole island (turn it into water) so it isn’t counted again. Sinking is recursive: sink this cell, then sink all four neighbors.

```

GridBacktracking.java islands

static int countIslands(char[][] grid) {
    int islands = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') {
                islands++;                // found an unvisited piece of land...
                sink(grid, r, c);          // ...erase its whole island
            }
        }
    }
    return islands;
}

static void sink(char[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1') return;
    g[r][c] = '0';
    sink(g, r + 1, c);
    sink(g, r - 1, c);
    sink(g, r, c + 1);
    sink(g, r, c - 1);
}

```

Notice there's **no un-choose** here. We *want* the cells to stay sunk, because we're not looking for all paths; we just want to visit every connected cell once. This is the difference between **DFS/flood fill** (mark permanently, visit each cell once, $O(\text{rows} \times \text{cols})$) and **backtracking** (mark temporarily, explore all paths, often exponential).

21.5 Sudoku solver

Problem: fill a 9×9 grid so that every row, every column and every 3×3 box contains the digits 1 to 9 exactly once.

- **State:** the board.
- **Choices:** a digit 1 to 9 for the first empty cell.
- **Constraints:** the digit must not already appear in the cell's row, column or box.
- **Goal:** no empty cells remain.

```

SudokuSolver.java solve

static boolean solve(int[][] board) {
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (board[r][c] != 0) continue; // already filled
            for (int d = 1; d <= 9; d++) {
                if (canPlace(board, r, c, d)) {
                    board[r][c] = d; // choose
                    if (solve(board)) return true; // explore: rest solvable? done!
                    board[r][c] = 0; // un-choose
                }
            }
            return false; // no digit fits here: backtrack
        }
    }
    return true; // no empty cell left: solved
}

static boolean canPlace(int[][] b, int r, int c, int d) {
    int boxRow = 3 * (r / 3), boxCol = 3 * (c / 3);
    for (int i = 0; i < 9; i++) {
        if (b[r][i] == d) return false; // row
        if (b[i][c] == d) return false; // column
        if (b[boxRow + i / 3][boxCol + i % 3] == d) return false; // 3x3 box
    }
    return true;
}

```

This method returns a `boolean`, which changes the shape slightly: instead of collecting all solutions, it **stops at the first one**. If the recursive call returns `true`, the board is solved, so we return `true` immediately **without undoing** (the board keeps the solution). If no digit works for the empty cell, we return `false`, and the caller undoes its choice and tries its next digit.

>_ OUTPUT OF SUDOKUSOLVER.JAVA (ACTUAL RUN)

Puzzle:

```

5 3 . | . 7 . | . . .
6 . . | 1 9 5 | . . .
. 9 8 | . . . | . 6 .
-----+-----+-----
8 . . | . 6 . | . . 3
4 . . | 8 . 3 | . . 1
7 . . | . 2 . | . . 6
-----+-----+-----
. 6 . | . . . | 2 8 .
. . . | 4 1 9 | . . 5
. . . | . 8 . | . 7 9

```

Solved:

```

5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
-----+-----+-----
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
-----+-----+-----
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9

```

✦ PRO TRICK: FIND ONE VS. FIND ALL

To find **all** solutions, return `void`, record at the goal, and always undo. To find **one** solution, return `boolean`: return `true` as soon as the goal is reached and propagate it upward immediately with `if (solve(...)) return true;`. This early return also saves a lot of time.

Σ CHAPTER SUMMARY

- **N-Queens**: one queen per row; check only upward; 92 solutions for $n = 8$.
- **Mazes and word search**: mark → explore → **unmark** to allow other paths.
- **Flood fill** is DFS without unmarking: visit each cell once.
- **Sudoku**: return `boolean` to stop at the first solution.

P A R T

VI

Making Recursion Fast

Memoization turns exponential recursion into fast algorithms; explicit stacks remove the depth limit.

22 Memoization: Recursion With a Memory

23 Recursion vs. Iteration



CHAPTER 22

Memoization: Recursion With a Memory

IN THIS CHAPTER

- Recognize overlapping subproblems, the cause of exponential slowness
- Add memoization to any recursive method with a three-step recipe
- Speed up Fibonacci from 331 million calls to 79
- Solve grid paths, 0/1 knapsack and longest common subsequence with memoized recursion
- Convert memoized recursion into bottom-up dynamic programming

22.1 Overlapping subproblems

Remember the Fibonacci tree from Chapter 13: `fib(5)` computes `fib(3)` twice and `fib(2)` three times. The same question is asked again and again, and each time it's answered from scratch. When a recursion keeps solving the **same subproblems** repeatedly, we say it has **overlapping subproblems**.

The fix is almost embarrassingly simple: **remember the answers**. The first time you compute `fib(3)`, write the answer down. The next time someone asks for `fib(3)`, look it up instead of recomputing it. This is called **memoization** (from "memo", a note to yourself; not "memorization").

★ KEY IDEA

A recursive method whose result depends **only on its parameters** will always return the same answer for the same parameters. So you can store the answer the first time and reuse it forever. If there are only a few distinct parameter values but many repeated calls, memoization makes the recursion dramatically faster.

22.2 The memoization recipe

§ THREE STEPS TO MEMOIZE ANY RECURSIVE METHOD

1. **Create a memo** (an array or a map) indexed by the method's parameters, initialized to "unknown" (for example `-1` or `null`).
2. **At the start of the method** (after the base cases), if the memo already has an answer for these parameters, **return it**.
3. **Before returning a computed answer, store it** in the memo.

Nothing else changes: same base cases, same recurrence, same leap of faith.

22.3 Fibonacci, memoized

Here are the naive and memoized versions side by side (both count their calls):

```
Memoization.java  fibNaive · fibMemo

static long fibNaive(int n) {
    calls++;
    if (n <= 1) return n;
    return fibNaive(n - 1) + fibNaive(n - 2);
}

static long fibMemo(int n, long[] memo) {
    calls++;
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];           // 1. already solved? reuse it
    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo); // 2. solve once, SAVE it
    return memo[n];
}
```

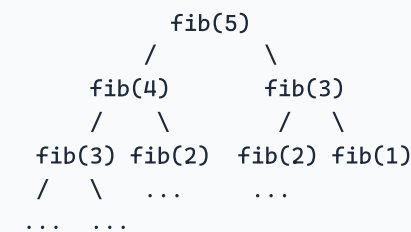
>_ OUTPUT OF MEMOIZATION.JAVA (ACTUAL RUN)

```
fibNaive(40) = 102334155    calls: 331,160,281
fibMemo(40)  = 102334155    calls: 79
fibBottomUp(90) = 2880067194370816120
gridPaths 18x18 = 2333606220
knapsack(cap=7) = 9
lcs("ABCB DAB", "BDCABA") = 4
```

For $n = 40$, the naive version makes **331,160,281** calls. The memoized version makes **79**. Same answer, over four million times fewer calls. Why 79? Each value from 2 to 40 is computed once (39 computations, each making 2 calls), plus the initial call: $2 \times 39 + 1 = 79$.

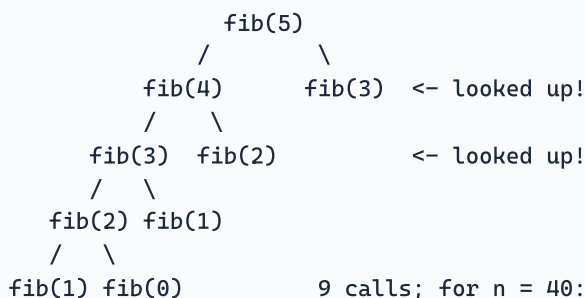
WHAT MEMOIZATION DOES TO THE FIB(5) TREE

without memo:



15 calls

with memo:



9 calls; for $n = 40$: 79 calls

The tree collapses into roughly a straight line: each `fib(k)` is fully computed only the first time; later requests return immediately. **Time goes from $O(1.618^n)$ to $O(n)$.** Space is $O(n)$ for the memo plus $O(n)$ stack.

✗ COMMON MISTAKE: MEMOIZING A METHOD WITH HIDDEN STATE

Memoization is only correct if the method's result depends **only on the parameters used as the memo key**. If the result also depends on a static variable, a shared list, or a parameter you left out of the key, the memo will return stale or wrong answers. Backtracking methods that mutate shared state are usually **not** memoizable as written.

22.4 Grid paths

Problem: in a grid with `rows × cols` cells, how many paths go from the top-left to the bottom-right cell if you can only move right or down?

Recurrence: from cell (r, c) , the first move is either down or right, so $\text{paths}(r, c) = \text{paths}(r + 1, c) + \text{paths}(r, c + 1)$. Base cases: at the target, 1 path; outside the grid, 0 paths.

The naive recursion is exponential: an 18×18 grid would make billions of calls. But there are only $18 \times 18 = 324$ distinct cells, so with a 2D memo, each cell is computed once:

```
Memoization.java  gridPaths

// Paths from (r, c) to the bottom-right cell, moving only right or down.
static long gridPaths(int r, int c, int rows, int cols, long[][] memo) {
    if (r == rows - 1 && c == cols - 1) return 1;    // at the target
    if (r >= rows || c >= cols) return 0;           // fell off the grid
    if (memo[r][c] != -1) return memo[r][c];
    return memo[r][c] = gridPaths(r + 1, c, rows, cols, memo) // go down
                        + gridPaths(r, c + 1, rows, cols, memo); // go right
}
```

The output shows 2,333,606,220 paths for an 18×18 grid, computed instantly. **Time: $O(\text{rows} \times \text{cols})$.**

✦ PRO TRICK: THE MEMO HAS ONE DIMENSION PER CHANGING PARAMETER

Look at which parameters **change** between recursive calls: those form the memo key. `fib(n)` → 1D array `memo[n]`. `gridPaths(r, c)` → 2D array `memo[r][c]`. `knapsack(i, cap)` → 2D array `memo[i][cap]`. Parameters that never change (like the input array or the grid size) are not part of the key.

22.5 0/1 knapsack

Problem: you have items with weights and values, and a bag with a weight capacity. Choose items (each at most once) to **maximize** the total value without exceeding the capacity.

This is the take/skip pattern from Chapter 18, but **optimizing** instead of listing:

- `knapsack(i, cap)` returns the best value achievable using items `i..n-1` with `cap` capacity left.
- **Skip** item `i`: the value is `knapsack(i + 1, cap)`.

- **Take** item i (only if it fits): `val[i] + knapsack(i + 1, cap - wt[i])`.
- The answer is the **maximum** of the two (“different choices $\rightarrow$ max”, from Chapter 14).

```

Memoization.java knapsack

// Best value using items i..n-1 with 'cap' capacity left. Each item used at most once.
static int knapsack(int[] wt, int[] val, int i, int cap, Integer[][] memo) {
    if (i == wt.length || cap == 0) return 0;
    if (memo[i][cap] != null) return memo[i][cap];
    int skip = knapsack(wt, val, i + 1, cap, memo);
    int take = 0;
    if (wt[i] <= cap) {
        take = val[i] + knapsack(wt, val, i + 1, cap - wt[i], memo);
    }
    return memo[i][cap] = Math.max(skip, take);
}

```

We use `Integer[][]` so that `null` means “not computed yet” (0 is a valid answer, so it can’t be used as the “unknown” marker). For weights {1, 3, 4, 5}, values {1, 4, 5, 7} and capacity 7, the best value is 9 (the items with weights 3 and 4). **Time:** $O(n \times \text{capacity})$ instead of $O(2^n)$.

22.6 Longest common subsequence

Problem: find the length of the longest sequence of characters that appears, in order, in both strings (not necessarily contiguously). For “ABCBDAB” and “BDCABA”, one LCS is “BCBA”, length 4.

- `lcs(i, j)` = LCS length of `a[i..]` and `b[j..]`.
- If `a[i] == b[j]`, that character can be part of the LCS: $1 + \text{lcs}(i + 1, j + 1)$.
- Otherwise, one of the two characters is not in the LCS; try dropping each and take the max.
- Base case: either string exhausted $\rightarrow 0$.

```

Memoization.java lcs

// Length of the longest common subsequence of a[i..] and b[j..].
static int lcs(String a, String b, int i, int j, Map<String, Integer> memo) {
    if (i == a.length() || j == b.length()) return 0;
    String key = i + "," + j;
    if (memo.containsKey(key)) return memo.get(key);
    int result;
    if (a.charAt(i) == b.charAt(j)) {
        result = 1 + lcs(a, b, i + 1, j + 1, memo); // match: use both chars
    } else {
        result = Math.max(lcs(a, b, i + 1, j, memo), // drop a char from a
                          lcs(a, b, i, j + 1, memo)); // or drop one from b
    }
    memo.put(key, result);
    return result;
}

```

This version uses a `HashMap` with a string key `"i,j"` to show that any structure that maps parameters to answers works as a memo. A 2D array would be faster.

22.7 From memoization to bottom-up dynamic programming

Memoized recursion is called **top-down dynamic programming**: it starts from the big problem and recurses down, computing subproblems on demand. You can also go **bottom-up**: solve the smallest subproblems first, in a loop, and fill a table until you reach the answer. Same recurrence, opposite direction, no recursion:

```
Memoization.java  fibBottomUp

static long fibBottomUp(int n) {
    if (n <= 1) return n;
    long[] dp = new long[n + 1];
    dp[0] = 0; // base cases become initial values
    dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2]; // same recurrence, smallest first
    }
    return dp[n];
}
```

The conversion recipe:

Memoized recursion	Bottom-up table
memo array	<code>dp</code> table of the same shape
base cases return values	initialize those table cells
recursive calls <code>f(n-1)</code> , <code>f(n-2)</code>	read <code>dp[i-1]</code> , <code>dp[i-2]</code> (already filled)
the recurrence	the loop body
call order decided by recursion	loop order: smallest subproblems first

★ KEY IDEA: THIS IS HOW DYNAMIC PROGRAMMING IS DISCOVERED

Most dynamic programming solutions are found in three steps: (1) write a plain recursive solution using the recipe from Chapter 4; (2) notice overlapping subproblems and **add memoization**; (3) optionally convert to a **bottom-up** loop to remove recursion overhead and depth limits. If you can write the recursion, you can do DP.

Σ CHAPTER SUMMARY

- **Overlapping subproblems** make naive recursion exponential.
- **Memoize**: check the memo first, store the answer before returning. Nothing else changes.
- fib(40): **331,160,281 calls** → **79 calls**. Grid paths, knapsack and LCS become polynomial.
- The memo key = the parameters that **change** between calls.
- Memoized recursion = **top-down DP**; converting to a table loop = **bottom-up DP**.

CHAPTER 23

Recursion vs. Iteration

23

IN THIS CHAPTER

- Understand that any recursion can be rewritten with an explicit stack
- Convert tree traversals to iterative code
- Simulate the Tower of Hanoi without recursion
- Decide when to use recursion and when to use loops

23.1 Anything recursive can be iterative

Recursion works because of the call stack. There's nothing magical about that stack: it's just a stack data structure managed by the JVM. So you can always replace recursion with **your own stack**, managed by your own loop. This is useful when the recursion would be too deep for the JVM stack (your stack lives on the heap, which is much larger), or when you need precise control over the order of processing.

Simple linear or tail recursion becomes a plain loop (Chapter 12). Tree recursion needs an explicit stack.

23.2 Preorder traversal with a stack

```

RecursionToIteration.java  preorderRecursive · preorderStack

static void preorderRecursive(Node node, StringBuilder out) {
    if (node == null) return;
    out.append(node.val).append(' ');
    preorderRecursive(node.left, out);
    preorderRecursive(node.right, out);
}

static String preorderIterative(Node root) {
    StringBuilder out = new StringBuilder();
    Deque<Node> stack = new ArrayDeque<>();           // OUR stack replaces the call stack
    if (root != null) stack.push(root);
    while (!stack.isEmpty()) {
        Node node = stack.pop();
        out.append(node.val).append(' ');
        if (node.right != null) stack.push(node.right); // pushed first => processed later
        if (node.left != null) stack.push(node.left);  // pushed last  => processed next
    }
    return out.toString();
}

```

The recursive version visits the node, then recurses left, then right. The iterative version pops a node, visits it, then pushes its children. The right child is pushed **first** so that the left child is on top and gets processed next, matching the recursive order.

23.3 Inorder traversal with a stack

Inorder is trickier, because the node must be processed **between** its left and right subtrees. The stack holds nodes whose left subtree is being processed and which still need to be visited:

```

RecursionToIteration.java  inorderStack

static String inorderIterative(Node root) {
    StringBuilder out = new StringBuilder();
    Deque<Node> stack = new ArrayDeque<>();
    Node cur = root;
    while (cur != null || !stack.isEmpty()) {
        while (cur != null) {                    // "recurse left" as far as possible
            stack.push(cur);
            cur = cur.left;
        }
        cur = stack.pop();                      // "return" to the parent frame
        out.append(cur.val).append(' ');
        cur = cur.right;                        // then "recurse right"
    }
    return out.toString();
}

```

Read the comments: going left as far as possible is the sequence of recursive calls on `node.left`; popping is returning to the parent frame; moving to `cur.right` is the second recursive call.

23.4 Tower of Hanoi with an explicit stack

For a general recursion, each stack entry represents a **pending call**. For Hanoi, a pending task is either “move n disks from X to Y ” or “move this one disk”. Because a stack is last-in, first-out, the three steps of each task are pushed in **reverse** order:

```

RecursionToIteration.java hanoiStack

// Tower of Hanoi without recursion: each stack entry is a pending "call".
record Task(int n, char from, char to, char via, boolean moveOnly) {}

static void hanoiIterative(int n) {
    Deque<Task> stack = new ArrayDeque<>();
    stack.push(new Task(n, 'A', 'C', 'B', false));
    while (!stack.isEmpty()) {
        Task t = stack.pop();
        if (t.n() == 0) continue;
        if (t.moveOnly()) {
            System.out.println("Move disk " + t.n() + " from " + t.from()
                               + " to " + t.to());
            continue;
        }
        // Push in REVERSE order of execution (a stack is last-in, first-out).
        stack.push(new Task(t.n() - 1, t.via(), t.to(), t.from(), false)); // step 3
        stack.push(new Task(t.n(), t.from(), t.to(), t.via(), true));       // step 2
        stack.push(new Task(t.n() - 1, t.from(), t.via(), t.to(), false)); // step 1
    }
}

```

>_ OUTPUT OF RECURSIONTOITERATION.JAVA (ACTUAL RUN)

```

preorder recursive : 1 2 4 5 3 6
preorder iterative : 1 2 4 5 3 6
inorder  iterative : 4 2 5 1 3 6
hanoi(3) with an explicit stack:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

```

The move sequence is identical to the recursive version from Chapter 13. This is essentially what the JVM does for you automatically when you write the recursive version.

I NOTE: RECORDS

`record Task(...)` is a compact way (Java 16+) to declare an immutable data class with a constructor and accessor methods like `t.n()` and `t.from()`. On older Java versions, use a small class with final fields.

23.5 When to use which

Situation	Prefer	Why
Trees, graphs (DFS), nested structures	Recursion	matches the data; much shorter and clearer
Backtracking, generating combinations	Recursion	the call stack handles choose/un-choose naturally
Divide and conquer	Recursion	depth is only $O(\log n)$; code mirrors the idea
Simple counting/summing over a range or array	Loop	no overhead, no depth limit, equally clear
Linear recursion that can be very deep (lists, big n)	Loop or explicit stack	Java would overflow (no TCO)
Performance-critical inner loops	Loop	method calls have a small overhead
Overlapping subproblems	Memoized recursion or bottom-up DP	avoids exponential blowup

★ KEY IDEA

Recursion and iteration are equally powerful: anything one can do, the other can. Choose based on **clarity** and **depth**. Use recursion when the problem is recursive in shape and the depth is manageable. Use loops when the problem is linear, or when depth could reach tens of thousands.

Σ CHAPTER SUMMARY

- The call stack is just a stack; any recursion can use **an explicit stack** instead.
- Push in **reverse** order of processing (last-in, first-out).
- Prefer recursion for trees, backtracking and divide and conquer; loops for linear, deep, or hot code.

P A R T

VII

Mastery

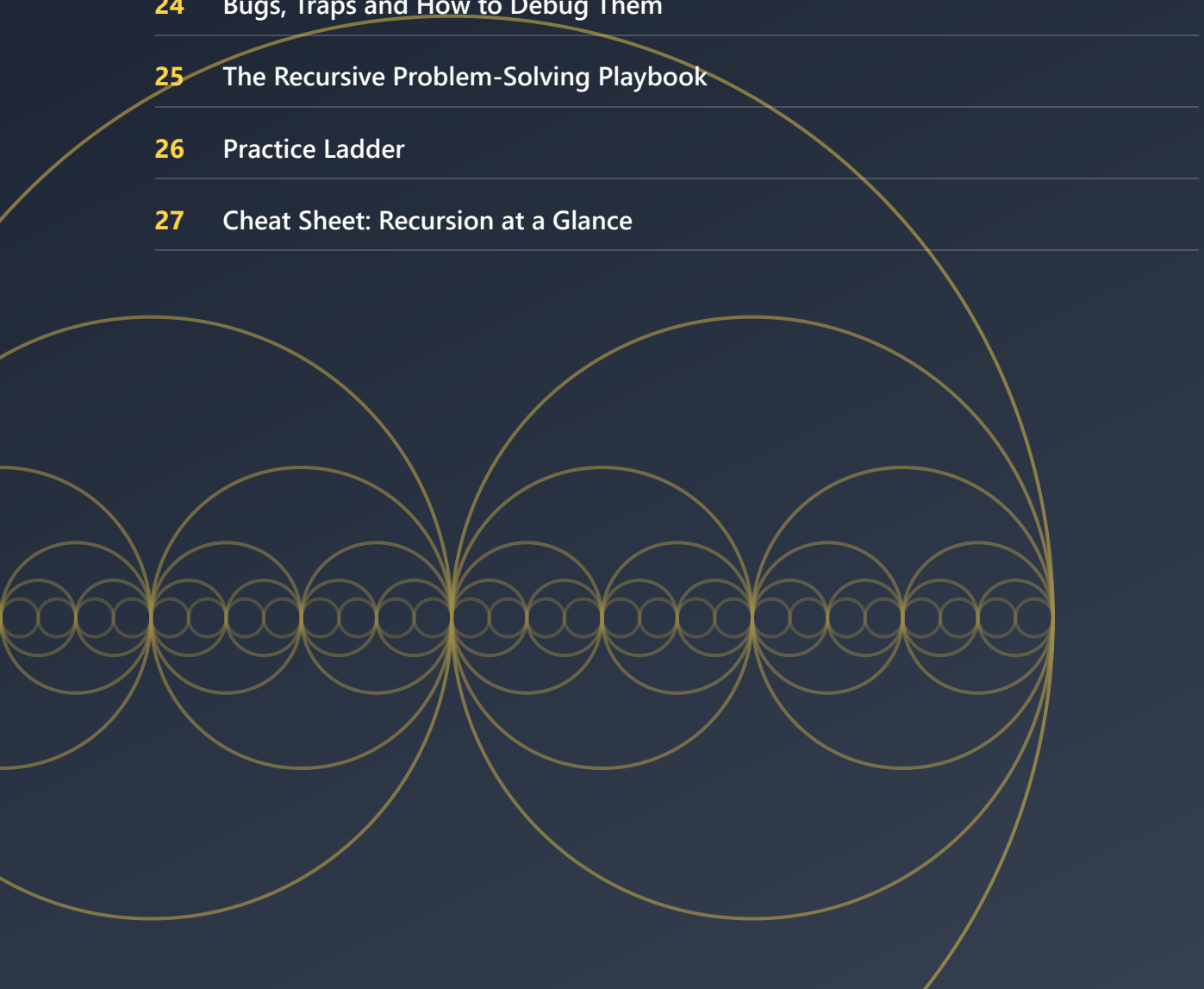
The bugs everyone makes, a playbook for any new problem, a practice ladder, and the whole book at a glance.

24 Bugs, Traps and How to Debug Them

25 The Recursive Problem-Solving Playbook

26 Practice Ladder

27 Cheat Sheet: Recursion at a Glance



CHAPTER 24

Bugs, Traps and How to Debug Them

24

IN THIS CHAPTER

- Recognize the twelve most common recursion bugs instantly
- See each bug next to its fix
- Learn a systematic debugging procedure for recursive code

Every bug below has been made by every programmer who has learned recursion. Learn to recognize them, and you'll save yourself hours.

24.1 Bug 1: missing base case

Java

X BUGGY

```
static int sum(int n) {
    return n + sum(n - 1);
}
```

Java

✓ FIXED

```
static int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n - 1);
}
```

Symptom: `StackOverflowError` for every input. **Fix:** always write the base case first, before the recursive case.

24.2 Bug 2: base case that can be skipped

Java

X BUGGY

```
static int sumEveryOther(int n) {
    if (n == 0) return 0;
    return n + sumEveryOther(n - 2); // odd n jumps from 1 to -1
}
```

Java

✓ FIXED

```
static int sumEveryOther(int n) {
    if (n <= 0) return 0;
    return n + sumEveryOther(n - 2);
}
```

Symptom: works for some inputs, overflows for others. **Fix:** use `<=` / `>=` when the input can jump past the exact base value.

24.3 Bug 3: `n--` instead of `n - 1`

Java

✗ BUGGY

```
static void countdown(int n) {
    if (n == 0) return;
    System.out.println(n);
    countdown(n--);    // passes the OLD value of n, then decrements the local copy
}
```

Java

✓ FIXED

```
static void countdown(int n) {
    if (n == 0) return;
    System.out.println(n);
    countdown(n - 1);
}
```

Symptom: infinite recursion. `n--` is a post-decrement: the call receives the current value of `n`, unchanged.

Fix: never use `++` or `--` in recursive arguments. Write `n - 1`, `i + 1`.

24.4 Bug 4: ignoring the result of the recursive call

Java

✗ BUGGY

```
static boolean contains(int[] a, int target, int i) {
    if (i == a.length) return false;
    if (a[i] == target) return true;
    contains(a, target, i + 1);    // result thrown away
    return false;
}
```

Java

✓ FIXED

```
static boolean contains(int[] a, int target, int i) {
    if (i == a.length) return false;
    if (a[i] == target) return true;
    return contains(a, target, i + 1);
}
```

Symptom: wrong answers (here: finds the target only at index 0). **Fix:** if the method returns a value, the recursive call's value must be **used**: returned or combined.

24.5 Bug 5: wrong base value

Java

✗ BUGGY

```
static long product(int[] a, int i) {
    if (i == a.length) return 0; // everything becomes 0
    return a[i] * product(a, i + 1);
}
```

Java

✓ FIXED

```
static long product(int[] a, int i) {
    if (i == a.length) return 1; // the neutral element for multiplication
    return a[i] * product(a, i + 1);
}
```

Fix: the base value must be the **neutral element** of the combine operation: 0 for `+`, 1 for `*`, `false` for `||`, `true` for `&&`, `Integer.MIN_VALUE` for `max`, `""` for string concatenation.

24.6 Bug 6: storing the live list instead of a copy

Java

✗ BUGGY

```
if (i == nums.length) {
    all.add(current); // the same object, which keeps changing
    return;
}
```

Java

✓ FIXED

```
if (i == nums.length) {
    all.add(new ArrayList<>(current)); // a snapshot of this moment
    return;
}
```

Symptom: the results list contains many identical (usually empty) lists. **Fix:** always store a copy of mutable state.

24.7 Bug 7: forgetting to undo

Java

X BUGGY

```
for (int i = 0; i < nums.length; i++) {
    if (used[i]) continue;
    used[i] = true;
    current.add(nums[i]);
    permute(nums, used, current, all);
    // forgot to undo: used[i] stays true, current keeps growing
}
```

Java

✓ FIXED

```
for (int i = 0; i < nums.length; i++) {
    if (used[i]) continue;
    used[i] = true;
    current.add(nums[i]);
    permute(nums, used, current, all);
    current.remove(current.size() - 1);
    used[i] = false;
}
```

Symptom: only the first solution is found, or results contain too many elements. **Fix:** every change before the recursive call gets a matching undo after it, in reverse order.

24.8 Bug 8: list.remove(int) vs. list.remove(Object)

Java

X BUGGY

```
List<Integer> path = new ArrayList<>();
path.add(node.val);
explore(...);
path.remove(node.val); // for a List<Integer>, this removes at INDEX node.val!
```

Java

✓ FIXED

```
path.add(node.val);
explore(...);
path.remove(path.size() - 1); // remove the last element, by index
```

Symptom: `IndexOutOfBoundsException`, or the wrong element removed. With `List<Integer>`, `remove(5)` means “remove index 5”, not “remove the value 5”. **Fix:** undo by removing the **last index**.

24.9 Bug 9: static state not reset

Java

X BUGGY

```
static int count = 0;
static void countNodes(Node node) {
    if (node == null) return;
    count++;
    countNodes(node.left);
    countNodes(node.right);
}
// calling countNodes twice gives double the answer the second time
```

Java

✓ FIXED

```
static int countNodes(Node node) {
    if (node == null) return 0;
    return 1 + countNodes(node.left) + countNodes(node.right);
}
```

Fix: prefer return values. If you must use shared state, create it fresh in a wrapper method and pass it as a parameter.

24.10 Bug 10: the same subproblem computed twice

Java

X BUGGY

```
static long fastPower(long x, int n) {
    if (n == 0) return 1;
    if (n % 2 == 0) return fastPower(x, n / 2) * fastPower(x, n / 2); // O(n), not O(log n)
    return x * fastPower(x, n - 1);
}
```

Java

✓ FIXED

```
static long fastPower(long x, int n) {
    if (n == 0) return 1;
    long half = fastPower(x, n / 2);
    return (n % 2 == 0) ? half * half : half * half * x;
}
```

Fix: store a recursive result in a variable if you need it more than once. If the duplication is spread across the tree, memoize.

24.11 Bug 11: validating a BST by comparing only with children

We covered this in Chapter 17: checking `left.val < node.val < right.val` at every node is not enough.

Fix: pass the allowed `(low, high)` range down.

24.12 Bug 12: expensive copying hidden in the arguments

Java

X BUGGY

```

static int sum(int[] a) {
    if (a.length == 0) return 0;
    return a[0] + sum(Arrays.copyOfRange(a, 1, a.length)); // copies n-1 elements per call:
    O(n²)
}

```

Java

✓ FIXED

```

static int sum(int[] a, int i) {
    if (i == a.length) return 0;
    return a[i] + sum(a, i + 1); // O(n)
}

```

Fix: pass indices, not copies (Chapter 9).

24.13 A systematic debugging procedure

When a recursive method misbehaves, go through these steps in order:

1. **Check the contract.** Can you state in one sentence what the method returns? Does every `return` statement honor that sentence?
2. **Check the base cases by hand.** Call the method on the smallest inputs (0, 1, empty, null) mentally. Are the answers right?
3. **Check progress.** Does every recursive call use a strictly smaller input? Is there any path (odd/even, negative, off-by-one) that skips the base case?
4. **Check the combination.** Assume the recursive call is correct. Is your combine step right? Say it in English.
5. **Check shared state.** Is anything mutable shared between calls (lists, arrays, static fields)? Is every change undone? Is every recorded result a copy?
6. **Trace with a depth parameter** (Chapter 5) on the smallest input that fails.

★ KEY IDEA

Nearly every recursion bug is in one of four places: **the base case**, **the progress**, **the combination**, or **shared state**. Check them in that order.

Σ CHAPTER SUMMARY

- Base case: missing, skippable, or wrong neutral value.
- Progress: `n--` in arguments, or steps that jump over the base case.
- Combination: ignored results, or the same subproblem computed twice.
- Shared state: live list stored, no undo, `remove(int)` confusion, static state not reset.

CHAPTER 25

The Recursive Problem-Solving Playbook

25

IN THIS CHAPTER

- Get a complete step-by-step playbook for attacking any new recursive problem
- Learn to recognize which pattern a problem needs from its wording
- Keep a library of templates for every pattern in the book
- Watch the playbook applied to two problems from scratch

25.1 The playbook

§ THE EIGHT-STEP PLAYBOOK

1. **Understand the problem with tiny examples.** Solve $n = 0, 1, 2, 3$ by hand. Draw them.
2. **Identify the pattern** using the recognition table below. (Linear? Tree? Take/skip? Permutation? Grid? Divide and conquer?)
3. **Define the method in one sentence**, including what every parameter means.
4. **Write the base cases**, including empty input and early exits.
5. **Write the recursive case with the leap of faith**: trust the smaller call(s), combine.
6. **Check progress** on every path.
7. **Test on your tiny examples**; trace with a depth parameter if the result is wrong.
8. **Analyze and optimize**: count calls and depth; memoize overlaps; prune backtracking; convert to a loop if the depth is too large.

25.2 Pattern recognition: from the problem statement to the technique

If the problem says or implies...	Pattern	Chapters
compute a value from n , $n-1$, $n-2$...	linear or multiple recursion with a recurrence	7, 13, 14
digits of a number	peel with <code>% 10</code> and <code>/ 10</code>	7
process an array or string element by element	index recursion: handle one, recurse on the rest	9, 10
check from both ends (palindrome, reverse)	two-pointer recursion	9, 10
sorted array, find something	binary search (divide and conquer)	15
"sort", "split in half", big power	divide and conquer	15
linked list	node + recursion on <code>next</code>	16
tree: height, size, sum, check a property	ask both children, combine	17
tree: path from root, bounds, depth	pass information down as parameters	17
"all subsets", "all subsequences", "pick any number of items"	take/skip, or loop from <code>start</code>	18
"all arrangements", "all orders"	permutations with <code>used[]</code>	19
"choose k ", "combinations that sum to"	loop from <code>start</code> , <code>i + 1</code> or <code>i</code>	20
"generate all valid..." with rules	backtracking with constraints and pruning	20
board puzzles, placing pieces	backtracking on a grid	21
maze, paths in a grid, words in a grid	DFS with mark/unmark	21
connected regions, islands	flood fill (mark without unmark)	21
"number of ways", "minimum cost", "maximum value" with repeated states	recursion + memoization	22

25.3 The template library

These are the skeletons of almost everything in this book. Recognizing which one a problem needs is half the solution.

 1. Linear recursion (numbers)

```
static int f(int n) {
    if (n == 0) return BASE;
    return combine(n, f(n - 1));
}
```

 2. Array/string index recursion

```
static int f(int[] a, int i) {
    if (i == a.length) return BASE;
    return combine(a[i], f(a, i + 1));
}
```

 3. Two pointers

```
static boolean f(char[] s, int left, int right) {
    if (left >= right) return true;
    if (s[left] != s[right]) return false;
    return f(s, left + 1, right - 1);
}
```

 4. Divide and conquer

```
static Result f(int[] a, int low, int high) {
    if (low >= high) return baseResult(a, low, high);
    int mid = low + (high - low) / 2;
    Result left = f(a, low, mid);
    Result right = f(a, mid + 1, high);
    return merge(left, right);
}
```

 5. Binary tree

```
static Result f(Node node) {
    if (node == null) return BASE;
    Result left = f(node.left);
    Result right = f(node.right);
    return combine(node.val, left, right);
}
```

6. Take/skip (subsets, knapsack)

```
static void f(int[] a, int i, List<Integer> cur, List<List<Integer>> out) {
    if (i == a.length) { out.add(new ArrayList<>(cur)); return; }
    cur.add(a[i]);
    f(a, i + 1, cur, out);           // take
    cur.remove(cur.size() - 1);
    f(a, i + 1, cur, out);           // skip
}
```

7. Loop-style backtracking (combinations; use i instead of j + 1 for reuse)

```
static void f(int[] a, int start, List<Integer> cur, List<List<Integer>> out) {
    if (isGoal(cur)) { out.add(new ArrayList<>(cur)); return; }
    for (int j = start; j < a.length; j++) {
        if (!isValid(cur, a[j])) continue;
        cur.add(a[j]);
        f(a, j + 1, cur, out);
        cur.remove(cur.size() - 1);
    }
}
```

8. Permutations

```
static void f(int[] a, boolean[] used, List<Integer> cur, List<List<Integer>> out) {
    if (cur.size() == a.length) { out.add(new ArrayList<>(cur)); return; }
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        used[i] = true; cur.add(a[i]);
        f(a, used, cur, out);
        cur.remove(cur.size() - 1); used[i] = false;
    }
}
```

9. Grid DFS / backtracking

```
static void f(char[][] g, int r, int c, boolean[][] seen) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length) return;
    if (seen[r][c] || g[r][c] == '#') return;
    seen[r][c] = true;
    f(g, r + 1, c, seen); f(g, r - 1, c, seen);
    f(g, r, c + 1, seen); f(g, r, c - 1, seen);
    seen[r][c] = false;           // omit this line for flood fill
}
```

10. Memoized recursion

```
static long f(int i, int j, Long[][] memo) {
    if (isBase(i, j)) return BASE;
    if (memo[i][j] != null) return memo[i][j];
    long result = combine(f(i + 1, j, memo), f(i, j + 1, memo));
    return memo[i][j] = result;
}
```

25.4 The playbook in action: counting islands

Let's replay how someone would arrive at the island-counting solution from Chapter 21 using the playbook.

- 1. Tiny examples.** A grid with one '1' has one island. Two '1's side by side: one island. Two '1's diagonal: two islands (only horizontal/vertical connections count).
- 2. Pattern.** "Connected regions" → flood fill (DFS on a grid, mark without unmark).
- 3. Define.** `sink(grid, r, c)` turns the entire island containing (r, c) into water. `countIslands(grid)` returns the number of islands.
- 4. Base cases.** `sink` stops if (r, c) is outside the grid or is not land.
- 5. Recursive case.** Sink this cell, then trust `sink` to sink the rest of the island from each of the four neighbors.
- 6. Progress.** Each call either stops immediately or turns one '1' into '0', and there are finitely many '1's, so it always terminates.
- 7. Test.** The example grid gives 3.
- 8. Analyze.** Each cell is sunk at most once: $O(\text{rows} \times \text{cols})$ time. Depth can be up to $\text{rows} \times \text{cols}$ for a snake-shaped island, so for huge grids, use an explicit stack or BFS.

25.5 The playbook in action: generating parentheses

- 1. Tiny examples.** $n = 1$: "()". $n = 2$: "(())", "()()". $n = 3$: five strings.
- 2. Pattern.** "Generate all valid..." with rules → backtracking with constraints.
- 3. Define.** `gen(n, open, close, cur, out)` adds to `out` every valid completion of `cur`, given `open` and `close` counts.
- 4. Base case.** `cur.length() == 2 * n` → record it.
- 5. Recursive case.** Two choices, each guarded by its rule: add `(` if `open < n`; add `)` if `close < open`. Choose → recurse → undo.
- 6. Progress.** Each call adds one character, and the length is capped at $2n$.
- 7. Test.** $n = 3$ gives the five strings in the output of Chapter 20.
- 8. Analyze.** The number of results is the n -th Catalan number (grows roughly like 4^n); pruning ensures we never build an invalid string.

Σ CHAPTER SUMMARY

- Follow the **eight-step playbook**: examples → pattern → define → base → leap of faith → progress → test → optimize.
- Use the **pattern recognition table** to map problem wording to a technique.
- Keep the **ten templates** in your head; most problems are one of them with a different combine step.

CHAPTER 26

Practice Ladder

26

IN THIS CHAPTER

- A graded list of problems to practice, from warm-ups to challenges
- Each with a one-line hint pointing to the right pattern

Do these in order. For each problem, apply the playbook **before** writing code: write the one-sentence contract, the base cases and the recurrence on paper. Most of these problems appear (under these or similar names) on LeetCode, GeeksforGeeks, HackerRank and in interviews.

26.1 Level 1: warm-up (Part II skills)

#	Problem	Hint
1	Print 1 to n and n to 1 without loops	work before vs. after the call
2	Sum of an array	<code>a[i] + sum(a, i + 1)</code>
3	Product of digits	<code>n % 10 * f(n / 10)</code> , base value 1
4	Count zeros in a number	watch out for the input 0 itself
5	Check whether a number is a power of two	<code>n % 2 == 0 && f(n / 2)</code> , base <code>n == 1</code>
6	Reverse a string	<code>f(rest) + first</code>
7	Check whether an array is a palindrome	two pointers
8	First uppercase letter in a string	early exit
9	Convert a string of digits to an int ("1234" → 1234)	<code>f(prefix) * 10 + lastDigit</code>
10	Print a pattern of stars (triangle) recursively	one recursion for rows, one for stars

26.2 Level 2: core (Part III and IV skills)

#	Problem	Hint
11	Fibonacci, then memoized	two calls; then add a memo array
12	Climbing stairs with 1, 2 or 3 steps	split by the first move: three terms
13	Recursive binary search (first and last occurrence)	keep searching left/right after a match
14	Fast exponentiation with negative exponents	$x^{-n} = 1 / x^n$
15	Merge sort and quick sort from memory	Chapter 15
16	Reverse a linked list	picture after the recursive call
17	Height, size and maximum of a binary tree	ask both children
18	Same tree / symmetric tree	compare two nodes at once: <code>f(a.left, b.right)</code>
19	Balanced binary tree in $O(n)$	return <code>-1</code> as a signal for “unbalanced”
20	Kth symbol in grammar	the answer depends on the parent in row $n-1$
21	Count paths in a grid with obstacles	grid paths + base case 0 for obstacles
22	Josephus problem	<code>(J(n-1) + k) % n</code>

26.3 Level 3: backtracking (Part V skills)

#	Problem	Hint
23	All subsets / subsets with duplicates	take/skip; sort + skip equals at the same level
24	All permutations / unique permutations	<code>used[]</code> ; skip twin if previous twin unused
25	Combinations (n choose k)	loop from <code>start</code> , prune when too few remain
26	Combination sum I (reuse) and II (no reuse, duplicates)	pass <code>i</code> vs. <code>i + 1</code> ; skip equals
27	Letter combinations of a phone number	one level per digit
28	Generate parentheses	<code>open < n</code> , <code>close < open</code>
29	Palindrome partitioning	loop over the end of the next piece
30	Letter case permutation ("a1b" → a1b, a1B, A1b, A1B)	take/skip with two letter cases
31	Word search	mark with '#', restore after
32	Rat in a maze / all paths in a grid	mark → explore → unmark
33	Number of islands / flood fill	mark without unmark
34	N-Queens (print all, count)	one queen per row; O(1) safety arrays
35	Sudoku solver	return <code>boolean</code> ; stop at the first solution

26.4 Level 4: challenge (Part VI and beyond)

#	Problem	Hint
36	0/1 knapsack, then bottom-up	take/skip with max; <code>memo[i][cap]</code>
37	Longest common subsequence	match → diagonal; else max of two drops
38	Edit distance	three choices: insert, delete, replace
39	Coin change (minimum coins, and number of ways)	min over coins; ways needs an index to avoid order duplicates
40	Word break	try every prefix that's a dictionary word; memoize on the index
41	Decode ways ("226" → 3)	take one digit or two digits; memoize
42	Diameter / maximum path sum of a binary tree	return depth, record best
43	Lowest common ancestor	both sides non-null → this node
44	Serialize and deserialize a binary tree	preorder with null markers
45	Expression add operators / target sum	backtracking with running value

✓ HOW TO PRACTICE EFFECTIVELY

- **Time-box:** give each problem 25 minutes before looking at hints.
- **Explain aloud:** after solving, explain your base case and leap of faith out loud as if teaching.
- **Re-solve:** redo a problem from memory three days later. If you can't, you memorized instead of understood; go back to its chapter.
- **Vary:** after solving a problem, change one rule (allow reuse, forbid duplicates, count instead of list) and solve it again.

CHAPTER 27

Cheat Sheet: Recursion at a Glance

27

27.1 The core idea

“

Solve a problem by solving a smaller version of the same problem, and combining the result with a little extra work. Stop at a problem so small the answer is obvious.

Concept	One-line reminder
Base case	the smallest input, answered directly; prefer the empty input; use <code><=</code> / <code>>=</code>
Progress	every call's input must be strictly closer to a base case
Leap of faith	assume the recursive call is correct; only design the combine step
Contract	one sentence stating exactly what the method returns
Call stack	one frame per active call, each with its own variables
Winding / unwinding	work before the call runs top-down; after the call, bottom-up (reversed)
Space	$O(\text{max depth})$, not $O(\text{total calls})$
Time	total calls $\times$ work per call; use recurrences or recursion trees
Tail call	nothing after the call; Java does not optimize it
Helper method	public clean signature, private helper with index/accumulator

27.2 The design recipe

1. Define the method in words (the contract).
2. Base case(s).
3. Assume the smaller problem is solved.
4. Combine.
5. Check progress.

27.3 Neutral base values

Combine with	Base value
<code>+</code> (sum, count)	<code>0</code>
<code>*</code> (product, ways in sequence)	<code>1</code>
<code>max</code>	<code>Integer.MIN_VALUE</code>
<code>min</code>	<code>Integer.MAX_VALUE</code>
logical OR (exists)	<code>false</code>
logical AND (all)	<code>true</code>
string concatenation	<code>""</code>

27.4 Common recurrences

Recurrence	Time
$T(n-1) + 1$	$O(n)$
$T(n/2) + 1$	$O(\log n)$
$T(n-1) + n$	$O(n^2)$
$2T(n/2) + 1$	$O(n)$
$2T(n/2) + n$	$O(n \log n)$
$2T(n-1) + 1$	$O(2^n)$
$n \cdot T(n-1)$	$O(n!)$

27.5 Backtracking in five lines

```

Java

if (goal) { record(copy); return; }
for (choice : choices) {
    if (!valid) continue;
    choose; recurse; unchoose;
}

```

- **Start index:** `i + 1` = no reuse, `i` = reuse, `used[]` = order matters.
- **Duplicates:** sort, then `if (j > start && a[j] == a[j-1]) continue;`

- **Find one:** return `boolean`, propagate `true` immediately.
- **Prune** before recursing, not after.

27.6 Memoization in three lines

```

Java
if (base) return baseValue;
if (memo[key] != null) return memo[key];
return memo[key] = recurrence(...);

```

27.7 Top 10 bugs

1. No base case
2. Base case can be skipped (use `<=`)
3. `n--` in the argument
4. Recursive result ignored
5. Wrong neutral base value
6. `all.add(current)` without copying
7. Forgot to undo
8. `list.remove(val)` on `List<Integer>`
9. Static state not reset
10. Same subproblem computed twice

27.8 Final words

You started this book with an idea: a problem that contains a smaller copy of itself. You end it with the whole toolkit: the call stack that makes recursion work, the leap of faith that lets you design it, the patterns (linear, index, two-pointer, divide and conquer, tree, take/skip, permutation, backtracking, grid DFS) that cover nearly every recursive problem, and memoization to make it fast.

The next step is practice. Take the practice ladder one rung at a time. Every time you solve a problem, say the leap of faith out loud: *“Assume the smaller problem is solved. How do I build my answer from it?”* After enough problems, you won’t need to say it anymore. You’ll just see it. That’s what it means to think recursively.

“

To understand recursion, you must first understand recursion. Now you do.